Mini Golf - Dokumentacja

Wygenerowano przez Doxygen 1.10.0

1 Indeks hierarchiczny 1
1.1 Hierarchia klas . . . 1

2 Indeks klas 3
2.1 Lista klas . . . 3

3 Indeks plików 5
3.1 Lista plików . . . 5

4 Dokumentacja klas 7
4.1 Dokumentacja klasy Ball . . . 7
4.1.1 Opis szczegółowy . . . 8
4.1.2 Dokumentacja konstruktora i destruktora . . . 9
4.1.2.1 Ball() . . . 9
4.1.3 Dokumentacja funkcji składowych . . . 9
4.1.3.1 applyImpulse() . . . 9
4.1.3.2 bounceX() . . . 9
4.1.3.3 bounceY() . . . 10
4.1.3.4 contains() . . . 10
4.1.3.5 draw() . . . 11
4.1.3.6 getPosition() . . . 11
4.1.3.7 getScale() . . . 12
4.1.3.8 getVelocity() . . . 12
4.1.3.9 invertVelocity() . . . 13
4.1.3.10 setPosition() . . . 13
4.1.3.11 setScale() . . . 13
4.1.3.12 stop() . . . 14
4.1.3.13 update() . . . 14
4.1.4 Dokumentacja atrybutów składowych . . . 15
4.1.4.1 player . . . 15
4.1.4.2 velocity . . . 15
4.2 Dokumentacja klasy FontManager . . . 15
4.2.1 Opis szczegółowy . . . 16
4.2.2 Dokumentacja konstruktora i destruktora . . . 16
4.2.2.1 FontManager() . . . 16
4.2.3 Dokumentacja funkcji składowych . . . 16
4.2.3.1 cleanup() . . . 16
4.2.3.2 get() . . . 17
4.2.3.3 load() . . . 17
4.2.4 Dokumentacja atrybutów składowych . . . 18
4.2.4.1 m_Font . . . 18
4.3 Dokumentacja klasy Game . . . 18
4.3.1 Opis szczegółowy . . . 19

4.3.2 Dokumentacja konstruktora i destruktora . . . 19
4.3.2.1 Game() . . . 19
4.3.3 Dokumentacja funkcji składowych . . . 20
4.3.3.1 run() . . . 20
4.3.4 Dokumentacja atrybutów składowych . . . 20
4.3.4.1 clock . . . 20
4.3.4.2 m_data . . . 20
4.3.4.3 timePerFrame . . . 20
4.4 Dokumentacja struktury GameData . . . 21
4.4.1 Opis szczegółowy . . . 21
4.4.2 Dokumentacja atrybutów składowych . . . 22
4.4.2.1 machine . . . 22
4.4.2.2 window . . . 22
4.5 Dokumentacja klasy GameOverState . . . 22
4.5.1 Opis szczegółowy . . . 25
4.5.2 Dokumentacja konstruktora i destruktora . . . 26
4.5.2.1 GameOverState() . . . 26
4.5.3 Dokumentacja funkcji składowych . . . 26
4.5.3.1 draw() . . . 26
4.5.3.2 handleInput() . . . 26
4.5.3.3 init() . . . 27
4.5.3.4 update() . . . 27
4.5.4 Dokumentacja atrybutów składowych . . . 27
4.5.4.1 m_alpha . . . 27
4.5.4.2 m_backgroundSprite . . . 27
4.5.4.3 m_bgTexture . . . 28
4.5.4.4 m_data . . . 28
4.5.4.5 m_dimmer . . . 28
4.5.4.6 m_fadeRect . . . 28
4.5.4.7 m_goToMenu . . . 28
4.5.4.8 m_isTransitioning . . . 28
4.5.4.9 m_level . . . 28
4.5.4.10 m_menuButton . . . 28
4.5.4.11 m_retryButton . . . 29
4.5.4.12 m_retryLevel . . . 29
4.5.4.13 m_title . . . 29
4.6 Dokumentacja klasy GameState . . . 29
4.6.1 Opis szczegółowy . . . 33
4.6.2 Dokumentacja konstruktora i destruktora . . . 34
4.6.2.1 GameState() . . . 34
4.6.3 Dokumentacja funkcji składowych . . . 34
4.6.3.1 draw() . . . 34

4.6.3.2 getTileAt() . . . 34
4.6.3.3 handleInput() . . . 35
4.6.3.4 init() . . . 36
4.6.3.5 initSounds() . . . 36
4.6.3.6 initUI() . . . 37
4.6.3.7 isWall() . . . 37
4.6.3.8 resetLevel() . . . 38
4.6.3.9 update() . . . 39
4.6.3.10 updateAimLine() . . . 40
4.6.3.11 updateHearts() . . . 40
4.6.4 Dokumentacja atrybutów składowych . . . 41
4.6.4.1 aimLine . . . 41
4.6.4.2 ball . . . 41
4.6.4.3 ballInHole . . . 41
4.6.4.4 forceVector . . . 41
4.6.4.5 heartSprites . . . 41
4.6.4.6 inputManager . . . 42
4.6.4.7 isJumping . . . 42
4.6.4.8 isSinking . . . 42
4.6.4.9 jumpTimer . . . 42
4.6.4.10 lastSafePos . . . 42
4.6.4.11 levelData . . . 42
4.6.4.12 levelFinishTimer . . . 42
4.6.4.13 lives . . . 42
4.6.4.14 m_alpha . . . 43
4.6.4.15 m_data . . . 43
4.6.4.16 m_fadeRect . . . 43
4.6.4.17 m_hitSound . . . 43
4.6.4.18 m_level . . . 43
4.6.4.19 m_shadow . . . 43
4.6.4.20 m_splashSound . . . 43
4.6.4.21 m_wallHitSound . . . 43
4.6.4.22 m_winSound . . . 44
4.6.4.23 maxJumpTime . . . 44
4.6.4.24 scoreText . . . 44
4.6.4.25 strokes . . . 44
4.6.4.26 tileMap . . . 44
4.6.4.27 wasInWater . . . 44
4.6.4.28 winText . . . 44
4.7 Dokumentacja klasy InputManager . . . 45
4.7.1 Opis szczegółowy . . . 46
4.7.2 Dokumentacja konstruktora i destruktora . . . 46

4.7.2.1 InputManager() . . . 46
4.7.3 Dokumentacja funkcji składowych . . . 46
4.7.3.1 getCurrentForce() . . . 46
4.7.3.2 getForceVector() . . . 47
4.7.3.3 handleEvent() . . . 47
4.7.3.4 isDragging() . . . 48
4.7.3.5 isReady() . . . 49
4.7.4 Dokumentacja atrybutów składowych . . . 49
4.7.4.1 ballStartPos . . . 49
4.7.4.2 currentMousePos . . . 49
4.7.4.3 force . . . 49
4.7.4.4 isGrabbed . . . 49
4.7.4.5 ready . . . 50
4.8 Dokumentacja klasy LeaderboardState . . . 50
4.8.1 Opis szczegółowy . . . 53
4.8.2 Dokumentacja konstruktora i destruktora . . . 53
4.8.2.1 LeaderboardState() . . . 53
4.8.3 Dokumentacja funkcji składowych . . . 54
4.8.3.1 draw() . . . 54
4.8.3.2 handleInput() . . . 54
4.8.3.3 init() . . . 54
4.8.3.4 update() . . . 54
4.8.4 Dokumentacja atrybutów składowych . . . 55
4.8.4.1 m_alpha . . . 55
4.8.4.2 m_backButton . . . 55
4.8.4.3 m_data . . . 55
4.8.4.4 m_fadeRect . . . 55
4.8.4.5 m_isFadingIn . . . 55
4.8.4.6 m_isGoingBack . . . 55
4.8.4.7 m_scoreTexts . . . 55
4.8.4.8 m_title . . . 56
4.9 Dokumentacja klasy LevelCompleteState . . . 56
4.9.1 Opis szczegółowy . . . 60
4.9.2 Dokumentacja konstruktora i destruktora . . . 60
4.9.2.1 LevelCompleteState() . . . 60
4.9.3 Dokumentacja funkcji składowych . . . 60
4.9.3.1 draw() . . . 60
4.9.3.2 handleInput() . . . 61
4.9.3.3 init() . . . 61
4.9.3.4 update() . . . 61
4.9.4 Dokumentacja atrybutów składowych . . . 62
4.9.4.1 m_alpha . . . 62

4.9.4.2 m_backgroundSprite . . . 62
4.9.4.3 m_bgTexture . . . 62
4.9.4.4 m_currentLevel . . . 62
4.9.4.5 m_data . . . 62
4.9.4.6 m_dimmer . . . 62
4.9.4.7 m_fadeRect . . . 62
4.9.4.8 m_goToMenu . . . 63
4.9.4.9 m_isTransitioning . . . 63
4.9.4.10 m_menuButton . . . 63
4.9.4.11 m_nextLevelButton . . . 63
4.9.4.12 m_retryButton . . . 63
4.9.4.13 m_retryLevel . . . 63
4.9.4.14 m_scoreText . . . 63
4.9.4.15 m_strokes . . . 63
4.9.4.16 m_title . . . 64
4.10 Dokumentacja klasy LevelManager . . . 64
4.10.1 Opis szczegółowy . . . 65
4.10.2 Dokumentacja funkcji składowych . . . 65
4.10.2.1 colorToTileID() . . . 65
4.10.2.2 loadLevelFromImage() . . . 67
4.10.3 Dokumentacja atrybutów składowych . . . 69
4.10.3.1 startPosition . . . 69
4.11 Dokumentacja klasy LevelSelectState . . . 69
4.11.1 Opis szczegółowy . . . 72
4.11.2 Dokumentacja konstruktora i destruktora . . . 72
4.11.2.1 LevelSelectState() . . . 72
4.11.3 Dokumentacja funkcji składowych . . . 73
4.11.3.1 draw() . . . 73
4.11.3.2 handleInput() . . . 73
4.11.3.3 init() . . . 73
4.11.3.4 update() . . . 74
4.11.4 Dokumentacja atrybutów składowych . . . 74
4.11.4.1 m_alpha . . . 74
4.11.4.2 m_backButton . . . 74
4.11.4.3 m_data . . . 75
4.11.4.4 m_fadeRect . . . 75
4.11.4.5 m_isFadingIn . . . 75
4.11.4.6 m_isTransitioning . . . 75
4.11.4.7 m_levelButtons . . . 75
4.11.4.8 m_nextLevelToLoad . . . 75
4.11.4.9 m_title . . . 75
4.12 Dokumentacja klasy MainMenuState . . . 76

4.12.1 Opis szczegółowy . . . 78
4.12.2 Dokumentacja konstruktora i destruktora . . . 79
4.12.2.1 MainMenuState() . . . 79
4.12.3 Dokumentacja funkcji składowych . . . 79
4.12.3.1 draw() . . . 79
4.12.3.2 handleInput() . . . 79
4.12.3.3 init() . . . 79
4.12.3.4 update() . . . 80
4.12.4 Dokumentacja atrybutów składowych . . . 80
4.12.4.1 m_alpha . . . 80
4.12.4.2 m_background . . . 80
4.12.4.3 m_data . . . 80
4.12.4.4 m_exitButton . . . 81
4.12.4.5 m_fadeRect . . . 81
4.12.4.6 m_goToLeaderboard . . . 81
4.12.4.7 m_isFadingIn . . . 81
4.12.4.8 m_isTransitioning . . . 81
4.12.4.9 m_leaderboardButton . . . 81
4.12.4.10 m_playButton . . . 81
4.12.4.11 m_title . . . 82
4.13 Dokumentacja klasy PauseMenuState . . . 82
4.13.1 Opis szczegółowy . . . 85
4.13.2 Dokumentacja konstruktora i destruktora . . . 86
4.13.2.1 PauseMenuState() . . . 86
4.13.3 Dokumentacja funkcji składowych . . . 87
4.13.3.1 draw() . . . 87
4.13.3.2 handleInput() . . . 87
4.13.3.3 init() . . . 87
4.13.3.4 update() . . . 88
4.13.4 Dokumentacja atrybutów składowych . . . 88
4.13.4.1 m_alpha . . . 88
4.13.4.2 m_backgroundSprite . . . 88
4.13.4.3 m_bgTexture . . . 88
4.13.4.4 m_currentLevel . . . 88
4.13.4.5 m_data . . . 89
4.13.4.6 m_dimmer . . . 89
4.13.4.7 m_fadeRect . . . 89
4.13.4.8 m_isGoingToMenu . . . 89
4.13.4.9 m_isReseting . . . 89
4.13.4.10 m_menuButton . . . 89
4.13.4.11 m_pauseTitle . . . 89
4.13.4.12 m_resetButton . . . 89

4.13.4.13 m_resumeButton . . . 89
4.14 Dokumentacja klasy ResourceManager . . . 90
4.14.1 Opis szczegółowy . . . 90
4.14.2 Dokumentacja konstruktora i destruktora . . . 90
4.14.2.1 ResourceManager() . . . 90
4.14.3 Dokumentacja funkcji składowych . . . 91
4.14.3.1 cleanUpAllResources() . . . 91
4.14.3.2 loadAllResources() . . . 91
4.15 Dokumentacja klasy SaveManager . . . 92
4.15.1 Opis szczegółowy . . . 94
4.15.2 Dokumentacja funkcji składowych . . . 94
4.15.2.1 getBestScore() . . . 94
4.15.2.2 getUnlockedLevel() . . . 95
4.15.2.3 init() . . . 95
4.15.2.4 load() . . . 96
4.15.2.5 save() . . . 97
4.15.2.6 submitScore() . . . 97
4.15.3 Dokumentacja atrybutów składowych . . . 98
4.15.3.1 bestScores . . . 98
4.15.3.2 fileName . . . 98
4.15.3.3 unlockedLevel . . . 98
4.16 Dokumentacja klasy SoundManager . . . 99
4.16.1 Opis szczegółowy . . . 99
4.16.2 Dokumentacja konstruktora i destruktora . . . 100
4.16.2.1 SoundManager() . . . 100
4.16.3 Dokumentacja funkcji składowych . . . 100
4.16.3.1 cleanup() . . . 100
4.16.3.2 get() . . . 100
4.16.3.3 load() . . . 101
4.16.4 Dokumentacja atrybutów składowych . . . 101
4.16.4.1 m_Sounds . . . 101
4.17 Dokumentacja klasy State . . . 101
4.17.1 Opis szczegółowy . . . 103
4.17.2 Dokumentacja konstruktora i destruktora . . . 103
4.17.2.1 ~State() . . . 103
4.17.3 Dokumentacja funkcji składowych . . . 103
4.17.3.1 draw() . . . 103
4.17.3.2 handleInput() . . . 103
4.17.3.3 init() . . . 104
4.17.3.4 pause() . . . 104
4.17.3.5 resume() . . . 104
4.17.3.6 update() . . . 104

4.18 Dokumentacja klasy StateManager . . . 105
4.18.1 Opis szczegółowy . . . 106
4.18.2 Dokumentacja konstruktora i destruktora . . . 106
4.18.2.1 StateManager() . . . 106
4.18.2.2 ∼StateManager() . . . 106
4.18.3 Dokumentacja funkcji składowych . . . 106
4.18.3.1 addState() . . . 106
4.18.3.2 getActiveState() . . . 107
4.18.3.3 processStateChanges() . . . 107
4.18.3.4 removeState() . . . 107
4.18.3.5 switchState() . . . 107
4.18.4 Dokumentacja atrybutów składowych . . . 107
4.18.4.1 m_isAdding . . . 107
4.18.4.2 m_isClearing . . . 107
4.18.4.3 m_isRemoving . . . 108
4.18.4.4 m_isReplacing . . . 108
4.18.4.5 m_newState . . . 108
4.18.4.6 m_states . . . 108
4.19 Dokumentacja klasy TextureManager . . . 108
4.19.1 Opis szczegółowy . . . 109
4.19.2 Dokumentacja konstruktora i destruktora . . . 109
4.19.2.1 TextureManager() . . . 109
4.19.3 Dokumentacja funkcji składowych . . . 109
4.19.3.1 cleanup() . . . 109
4.19.3.2 get() . . . 109
4.19.3.3 load() . . . 110
4.19.4 Dokumentacja atrybutów składowych . . . 111
4.19.4.1 m_Textures . . . 111
4.20 Dokumentacja klasy TileMap . . . 111
4.20.1 Opis szczegółowy . . . 112
4.20.2 Dokumentacja funkcji składowych . . . 113
4.20.2.1 draw() . . . 113
4.20.2.2 load() . . . 113
4.20.3 Dokumentacja atrybutów składowych . . . 114
4.20.3.1 m_tileset . . . 114
4.20.3.2 m_vertices . . . 114

5 Dokumentacja plików **115**
5.1 Dokumentacja pliku Ball.cpp . . . 115
5.2 Dokumentacja pliku Ball.h . . . 115
5.3 Ball.h . . . 116
5.4 Dokumentacja pliku DEFINITIONS.h . . . 117

5.4.1 Opis szczegółowy . . . 119
5.4.2 Dokumentacja definicji . . . 119
5.4.2.1 BALL_TEXTURE_PATH . . . 119
5.4.2.2 FONT_PATH . . . 119
5.4.2.3 HEART_PATH . . . 119
5.4.2.4 HEART_WHITE_PATH . . . 119
5.4.2.5 HIT_PATH . . . 120
5.4.2.6 LEVEL_HEIGHT . . . 120
5.4.2.7 LEVEL_WIDTH . . . 120
5.4.2.8 SCREEN_HEIGHT . . . 120
5.4.2.9 SCREEN_WIDTH . . . 120
5.4.2.10 SPLASH_PATH . . . 120
5.4.2.11 TILE_SIZE . . . 120
5.4.2.12 TILESET_PATH . . . 120
5.4.2.13 TITLE . . . 121
5.4.2.14 WALL_HIT_PATH . . . 121
5.4.2.15 WIN_PATH . . . 121
5.4.3 Dokumentacja funkcji . . . 121
5.4.3.1 COLOR_BALL() . . . 121
5.4.3.2 COLOR_CONCRETE() . . . 121
5.4.3.3 COLOR_CORNER_LD() . . . 122
5.4.3.4 COLOR_CORNER_LU() . . . 122
5.4.3.5 COLOR_CORNER_RD() . . . 122
5.4.3.6 COLOR_CORNER_RU() . . . 123
5.4.3.7 COLOR_GRASS() . . . 123
5.4.3.8 COLOR_HOLE() . . . 123
5.4.3.9 COLOR_RAMP_DOWN() . . . 124
5.4.3.10 COLOR_RAMP_L() . . . 124
5.4.3.11 COLOR_RAMP_R() . . . 124
5.4.3.12 COLOR_RAMP_UP() . . . 125
5.4.3.13 COLOR_SAND() . . . 125
5.4.3.14 COLOR_WALL_DOWN() . . . 125
5.4.3.15 COLOR_WALL_L() . . . 126
5.4.3.16 COLOR_WALL_R() . . . 126
5.4.3.17 COLOR_WALL_UP() . . . 126
5.4.3.18 COLOR_WATER() . . . 127
5.5 DEFINITIONS.h . . . 127
5.6 Dokumentacja pliku FontManager.cpp . . . 128
5.7 Dokumentacja pliku FontManager.h . . . 128
5.8 FontManager.h . . . 129
5.9 Dokumentacja pliku Game.cpp . . . 129
5.10 Dokumentacja pliku Game.h . . . 130

5.10.1 Dokumentacja definicji typów . . . 131
5.10.1.1 GameDataRef . . . 131
5.11 Game.h . . . 131
5.12 Dokumentacja pliku GameOverState.cpp . . . 132
5.13 Dokumentacja pliku GameOverState.h . . . 132
5.14 GameOverState.h . . . 133
5.15 Dokumentacja pliku GameState.cpp . . . 134
5.16 Dokumentacja pliku GameState.h . . . 134
5.17 GameState.h . . . 135
5.18 Dokumentacja pliku InputManager.cpp . . . 136
5.19 Dokumentacja pliku InputManager.h . . . 137
5.20 InputManager.h . . . 138
5.21 Dokumentacja pliku LeaderBoardState.cpp . . . 138
5.22 Dokumentacja pliku LeaderBoardState.h . . . 139
5.23 LeaderBoardState.h . . . 140
5.24 Dokumentacja pliku LevelCompleteState.cpp . . . 140
5.25 Dokumentacja pliku LevelCompleteState.h . . . 140
5.26 LevelCompleteState.h . . . 141
5.27 Dokumentacja pliku LevelManager.cpp . . . 142
5.28 Dokumentacja pliku LevelManager.h . . . 143
5.29 LevelManager.h . . . 143
5.30 Dokumentacja pliku LevelSelectState.cpp . . . 144
5.31 Dokumentacja pliku LevelSelectState.h . . . 144
5.32 LevelSelectState.h . . . 145
5.33 Dokumentacja pliku main.cpp . . . 146
5.33.1 Dokumentacja funkcji . . . 146
5.33.1.1 main() . . . 146
5.34 Dokumentacja pliku MainMenuState.cpp . . . 147
5.35 Dokumentacja pliku MainMenuState.h . . . 147
5.36 MainMenuState.h . . . 148
5.37 Dokumentacja pliku PauseMenuState.cpp . . . 148
5.38 Dokumentacja pliku PauseMenuState.h . . . 149
5.39 PauseMenuState.h . . . 150
5.40 Dokumentacja pliku ResourceManager.cpp . . . 150
5.41 Dokumentacja pliku ResourceManager.h . . . 150
5.42 ResourceManager.h . . . 151
5.43 Dokumentacja pliku SaveManager.cpp . . . 152
5.44 Dokumentacja pliku SaveManager.h . . . 152
5.45 SaveManager.h . . . 153
5.46 Dokumentacja pliku SoundManager.cpp . . . 154
5.47 Dokumentacja pliku SoundManager.h . . . 154
5.48 SoundManager.h . . . 155

5.49 Dokumentacja pliku State.h 156
5.50 State.h 156
5.51 Dokumentacja pliku StateManager.cpp 157
5.52 Dokumentacja pliku StateManager.h 157
5.52.1 Dokumentacja definicji typów 159
5.52.1.1 StateRef 159
5.53 StateManager.h 159
5.54 Dokumentacja pliku TextureManager.cpp 160
5.55 Dokumentacja pliku TextureManager.h 160
5.56 TextureManager.h 161
5.57 Dokumentacja pliku TileMap.cpp 161
5.58 Dokumentacja pliku TileMap.h 162
5.59 TileMap.h 163

Skorowidz **165**

Rozdział 1

Indeks hierarchiczny

1.1 Hierarchia klas

Ta lista dziedziczenia posortowana jest z grubsza, choć nie całkowicie, alfabetycznie:

Ball 7
sf::Drawable
  TileMap 111
FontManager 15
Game 18
GameData 21
InputManager 45
LevelManager 64
ResourceManager 90
SaveManager 92
SoundManager 99
State 101
  GameOverState 22
  GameState 29
  LeaderboardState 50
  LevelCompleteState 56
  LevelSelectState 69
  MainMenuState 76
  PauseMenuState 82
StateManager 105
TextureManager 108
sf::Transformable
  TileMap 111

Rozdział 2

Indeks klas

2.1 Lista klas

Tutaj znajdują się klasy, struktury, unie i interfejsy wraz z ich krótkimi opisami:

Ball
Klasa reprezentująca piłkę golfową (obiekt gracza) 7
FontManager
Klasa statyczna zarządzająca zasobami czcionek w grze 15
Game
Główna klasa silnika gry 18
GameData
Struktura przechowująca współdzielone dane gry 21
GameOverState
Stan gry wyświetlany po przegranej (Game Over) 22
GameState
Główny stan rozgrywki 29
InputManager
Klasa obsługująca sterowanie (mechanika Drag & Shoot) 45
LeaderboardState
Stan wyświetlający tabelę najlepszych wyników 50
LevelCompleteState
Stan gry wyświetlany po pomyślnym ukończeniu poziomu 56
LevelManager
Klasa statyczna odpowiedzialna za wczytywanie poziomów 64
LevelSelectState
Stan wyboru poziomu 69
MainMenuState
Stan początkowy gry (Menu Główne) 76
PauseMenuState
Stan pauzy nakładany na stan gry 82
ResourceManager
Fasada zarządzająca ładowaniem wszystkich zasobów aplikacji 90
SaveManager
Klasa statyczna obsługująca trwałość danych gry (system zapisu/odczytu) 92
SoundManager
Statyczny menedżer zasobów dla buforów dźwiękowych 99
State
Abstrakcyjna klasa bazowa reprezentująca pojedynczy stan gry (np. Menu, Gra, Pauza) 101
StateManager
Zarządza stosem stanów gry 105

TextureManager
Statyczny menedżer zasobów dla tekstur SFML 108
TileMap
Klasa odpowiedzialna za renderowanie poziomu gry przy użyciu Vertex Array 111

Rozdział 3

Indeks plików

3.1 Lista plików

Tutaj znajduje się lista wszystkich plików z ich krótkimi opisami:

Ball.cpp 115
Ball.h 115
DEFINITIONS.h
Plik nagłówkowy zawierający globalne definicje, stałe oraz makra używane w projekcie 117
FontManager.cpp 128
FontManager.h 128
Game.cpp 129
Game.h 130
GameOverState.cpp 132
GameOverState.h 132
GameState.cpp 134
GameState.h 134
InputManager.cpp 136
InputManager.h 137
LeaderBoardState.cpp 138
LeaderBoardState.h 139
LevelCompleteState.cpp 140
LevelCompleteState.h 140
LevelManager.cpp 142
LevelManager.h 143
LevelSelectState.cpp 144
LevelSelectState.h 144
main.cpp 146
MainMenuState.cpp 147
MainMenuState.h 147
PauseMenuState.cpp 148
PauseMenuState.h 149
ResourceManager.cpp 150
ResourceManager.h 150
SaveManager.cpp 152
SaveManager.h 152
SoundManager.cpp 154
SoundManager.h 154
State.h 156
StateManager.cpp 157

StateManager.h 157
TextureManager.cpp 160
TextureManager.h 160
TileMap.cpp 161
TileMap.h 162

Rozdział 4

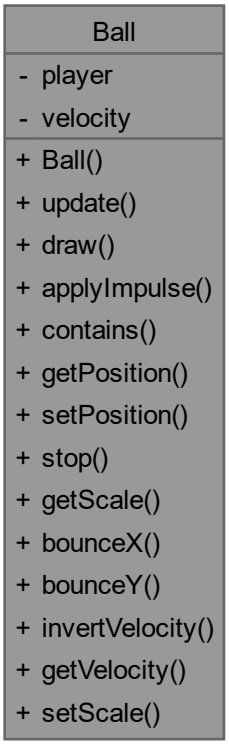
Dokumentacja klas

4.1 Dokumentacja klasy Ball

Klasa reprezentująca piłkę golfową (obiekt gracza).

```
#include <Ball.h>
```

Diagram współpracy dla Ball:



Metody publiczne

- Ball ()

  Domyślny konstruktor. Inicjalizuje teksturę i origin piłki.
- void update (float dt, float friction)

  Aktualizuje pozycję piłki na podstawie prędkości i czasu.
- void draw (sf::RenderWindow &window)

  Rysuje piłkę w oknie gry.
- void applyImpulse (const sf::Vector2f &impulse)

  Nadaje piłce natychmiastowy wektor prędkości (uderzenie).
- bool contains (const sf::Vector2f &point) const

  Sprawdza, czy dany punkt (np. myszka) znajduje się w obrębie piłki.
- sf::Vector2f getPosition () const

  Pobiera aktualną pozycję piłki.
- void setPosition (const sf::Vector2f &pos)

  Ustawia piłkę w konkretnym miejscu (np. na starcie poziomu).
- void stop ()

  Całkowicie zatrzymuje piłkę (zeruje prędkość).
- sf::Vector2f getScale () const

  Pobiera aktualną skalę obiektu.
- void bounceX ()

  Obsługuje fizykę odbicia od pionowej ściany (oś X). Odwraca wektor X i redukuje prędkość (symulacja utraty energii).
- void bounceY ()

  Obsługuje fizykę odbicia od poziomej ściany (oś Y). Odwraca wektor Y i redukuje prędkość.
- void invertVelocity ()

  Odwraca całkowicie wektor prędkości.
- sf::Vector2f getVelocity () const

  Pobiera aktualny wektor prędkości.
- void setScale (float factor)

  Ustawia skalę sprite'a (używane np. przy animacji skoku lub wpadania do wody).

Atrybuty prywatne

- sf::Sprite player

  Sprite graficzny piłki.
- sf::Vector2f velocity

  Aktualny wektor prędkości piłki.

4.1.1 Opis szczegółowy

Klasa reprezentująca piłkę golfową (obiekt gracza).

Obsługuje fizykę ruchu, kolizje (odbicia), renderowanie oraz logikę interakcji z otoczeniem.

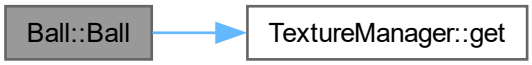
4.1.2 Dokumentacja konstruktora i destruktora

4.1.2.1 Ball()

```
Ball::Ball ( )
```

Domyślny konstruktor. Inicjalizuje teksturę i origin piłki.

Oto graf wywołań dla tej funkcji:



4.1.3 Dokumentacja funkcji składowych

4.1.3.1 applyImpulse()

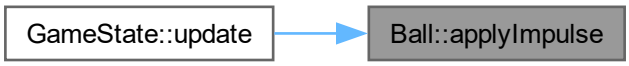
```
void Ball::applyImpulse (
            const sf::Vector2f & impulse )
```

Nadaje piłce natychmiastowy wektor prędkości (uderzenie).

Parametry

impulse	Wektor siły uderzenia.

Oto graf wywoływań tej funkcji:



4.1.3.2 bounceX()

```
void Ball::bounceX ( )
```

Obsługuje fizykę odbicia od pionowej ściany (oś X). Odwraca wektor X i redukuje prędkość (symulacja utraty energii).

Oto graf wywoływań tej funkcji:

4.1.3.3 bounceY()

```
void Ball::bounceY ( )
```

Obsługuje fizykę odbicia od poziomej ściany (oś Y). Odwraca wektor Y i redukuje prędkość.

Oto graf wywoływań tej funkcji:

4.1.3.4 contains()

```
bool Ball::contains (
            const sf::Vector2f & point ) const
```

Sprawdza, czy dany punkt (np. myszka) znajduje się w obrębie piłki.

Parametry

point	Współrzędne punktu.

Zwraca

true Jeśli punkt zawiera się w granicach sprite'a.

Oto graf wywoływań tej funkcji:

4.1.3.5 draw()

```
void Ball::draw (
            sf::RenderWindow & window )
```

Rysuje piłkę w oknie gry.

Parametry

window	Referencja do okna renderowania.

Oto graf wywoływań tej funkcji:

4.1.3.6 getPosition()

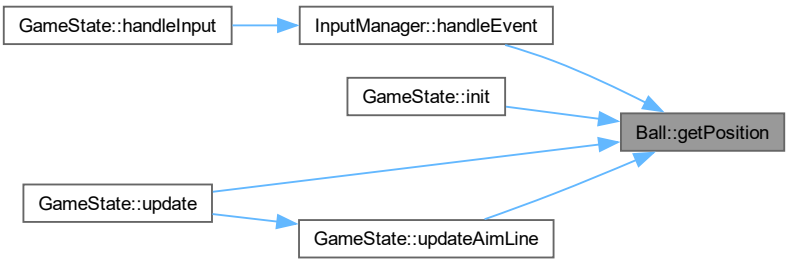
```
sf::Vector2f Ball::getPosition ( ) const
```

Pobiera aktualną pozycję piłki.

Zwraca

Wektor pozycji (x, y).

Oto graf wywoływań tej funkcji:



4.1.3.7 getScale()

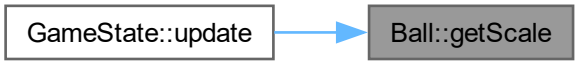
```
sf::Vector2f Ball::getScale ( ) const  [inline]
```

Pobiera aktualną skalę obiektu.

Zwraca

Wektor skali (x, y).

Oto graf wywoływań tej funkcji:



4.1.3.8 getVelocity()

```
sf::Vector2f Ball::getVelocity ( ) const
```

Pobiera aktualny wektor prędkości.

Zwraca

Wektor prędkości (vx, vy).

Oto graf wywoływań tej funkcji:

4.1.3.9 invertVelocity()

```
void Ball::invertVelocity ( )
```

Odwraca całkowicie wektor prędkości.

4.1.3.10 setPosition()

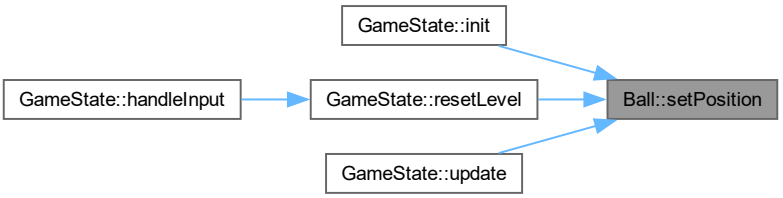
```
void Ball::setPosition (
            const sf::Vector2f & pos )
```

Ustawia piłkę w konkretnym miejscu (np. na starcie poziomu).

Parametry

pos	Nowa pozycja.

Oto graf wywoływań tej funkcji:



4.1.3.11 setScale()

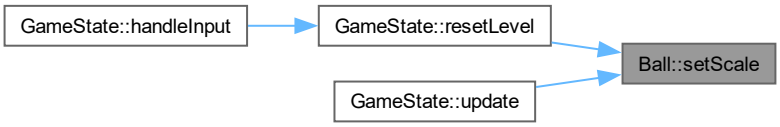
```
void Ball::setScale (
            float factor )
```

Ustawia skalę sprite'a (używane np. przy animacji skoku lub wpadania do wody).

Parametry

factor	Współczynnik skali (1.0 = normalny rozmiar).

Oto graf wywoływań tej funkcji:



4.1.3.12 stop()

```
void Ball::stop ( )
```

Całkowicie zatrzymuje piłkę (zeruje prędkość).

Oto graf wywoływań tej funkcji:

4.1.3.13 update()

```
void Ball::update (
            float dt,
            float friction )
```

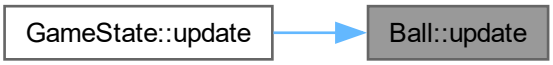
Aktualizuje pozycję piłki na podstawie prędkości i czasu.

Parametry

dt	Czas, który upłynął od ostatniej klatki (Delta Time).
friction	Współczynnik tarcia (zwalnia piłkę w każdej klatce).

-

Oto graf wywoływań tej funkcji:



4.1.4 Dokumentacja atrybutów składowych

4.1.4.1 player

```
sf::Sprite Ball::player [private]
```

Sprite graficzny piłki.

4.1.4.2 velocity

```
sf::Vector2f Ball::velocity [private]
```

Aktualny wektor prędkości piłki.

Dokumentacja dla tej klasy została wygenerowana z plików:

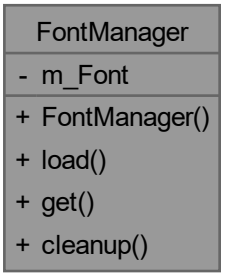
- Ball.h
- Ball.cpp

4.2 Dokumentacja klasy FontManager

Klasa statyczna zarządzająca zasobami czcionek w grze.

```
#include <FontManager.h>
```

Diagram współpracy dla FontManager:



Metody publiczne

- FontManager ()=delete

  Konstruktor usunięty - klasa jest w pełni statyczna.

Statyczne metody publiczne

- static void load (const std::string &key, const std::string &path)

  Ładuje czcionkę z pliku i przypisuje ją do klucza.
- static const sf::Font & get (const std::string &key)

  Pobiera załadowaną wcześniej czcionkę.
- static void cleanup ()

  Usuwa wszystkie załadowane czcionki i zwalnia pamięć.

Statyczne atrybuty prywatne

- static std::map< std::string, sf::Font > m_Font

  Mapa przechowująca pary: nazwa czcionki -> obiekt czcionki.

4.2.1 Opis szczegółowy

Klasa statyczna zarządzająca zasobami czcionek w grze.

Odpowiada za ładowanie, przechowywanie i udostępnianie obiektów sf::Font. Zapobiega wielokrotnemu ładowaniu tych samych plików z dysku.

4.2.2 Dokumentacja konstruktora i destruktora

4.2.2.1 FontManager()

```
FontManager::FontManager ( )  [delete]
```

Konstruktor usunięty - klasa jest w pełni statyczna.

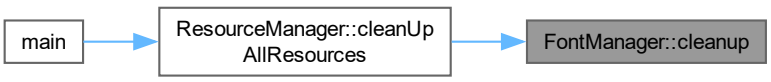
4.2.3 Dokumentacja funkcji składowych

4.2.3.1 cleanup()

```
void FontManager::cleanup ( )  [static]
```

Usuwa wszystkie załadowane czcionki i zwalnia pamięć.

Oto graf wywoływań tej funkcji:



4.2.3.2 get()

```
const sf::Font & FontManager::get (
            const std::string & key )  [static]
```

Pobiera załadowaną wcześniej czcionkę.

Parametry

key	Klucz pod którym czcionka została zapisana.

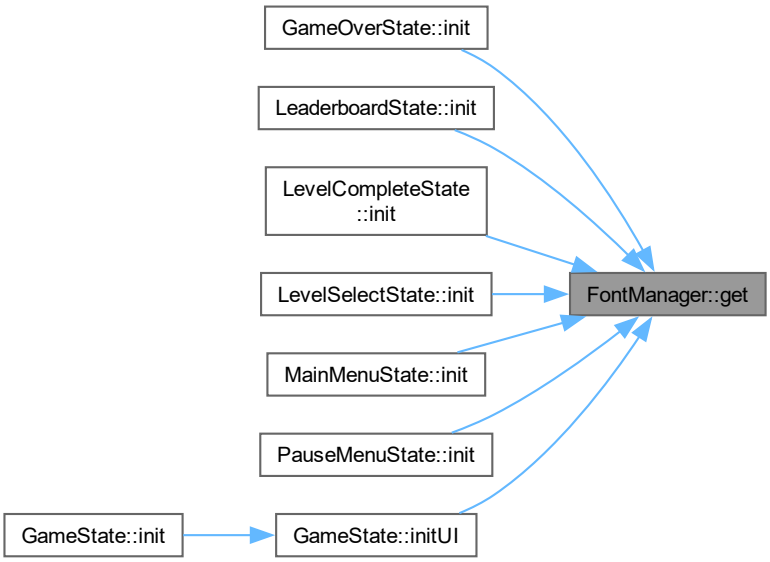
Zwraca

- Stała referencja do obiektu sf::Font.

Wyjątki

std::out_of_range	Jeśli klucz nie istnieje w mapie.

Oto graf wywoływań tej funkcji:



4.2.3.3 load()

```
void FontManager::load (
            const std::string & key,
            const std::string & path )  [static]
```

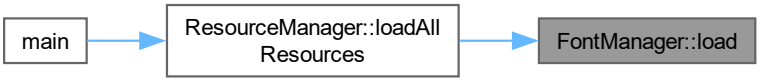
Ładuje czcionkę z pliku i przypisuje ją do klucza.

Parametry

key	Unikalny identyfikator (nazwa) czcionki, np. "MainFont".
path	Ścieżka do pliku czcionki (np. .ttf).

-

Oto graf wywoływań tej funkcji:



4.2.4 Dokumentacja atrybutów składowych

4.2.4.1 m_Font

`std::map< std::string, sf::Font > FontManager::m_Font [static], [private]`

Mapa przechowująca pary: nazwa czcionki -> obiekt czcionki.

Dokumentacja dla tej klasy została wygenerowana z plików:

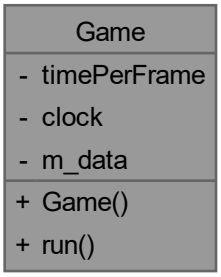
- FontManager.h
- FontManager.cpp

4.3 Dokumentacja klasy Game

Główna klasa silnika gry.

`#include <Game.h>`

Diagram współpracy dla Game:



Metody publiczne

- Game (int width, int height, std::string title)

  Konstruktor inicjalizujący grę.
- void run ()

  Uruchamia główną pętlę gry.

Atrybuty prywatne

- const sf::Time timePerFrame = sf::seconds(1.f / 60.f)

  Stały czas trwania jednej klatki logicznej (1/60 sekundy).
- sf::Clock clock

  Zegar do mierzenia czasu rzeczywistego.
- GameDataRef m_data = std::make_shared<GameData>()

  Współdzielone dane gry (okno, maszyna stanów).

4.3.1 Opis szczegółowy

Główna klasa silnika gry.

Odpowiada za inicjalizację okna, zarządzanie czasem (Game Loop) oraz uruchomienie maszyny stanów.

4.3.2 Dokumentacja konstruktora i destruktora

4.3.2.1 Game()

```
Game::Game (
            int width,
            int height,
            std::string title )
```

Konstruktor inicjalizujący grę.

- Tworzy okno aplikacji i ustawia pierwszy stan (MainMenuState).

Parametry

width	Szerokość okna.
height	Wysokość okna.
title	Tytuł okna.

-

4.3.3 Dokumentacja funkcji składowych

4.3.3.1 run()

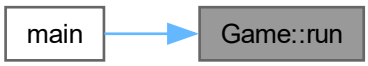
```
void Game::run ( )
```

Uruchamia główną pętlę gry.

Pętla obsługuje:

- Zmiany stanów (processStateChanges)
- Aktualizację logiki (update) ze stałym krokiem czasowym (fixed time step).
- Renderowanie (draw) z interpolacją.

Oto graf wywoływań tej funkcji:



4.3.4 Dokumentacja atrybutów składowych

4.3.4.1 clock

```
sf::Clock Game::clock  [private]
```

Zegar do mierzenia czasu rzeczywistego.

4.3.4.2 m_data

```
GameDataRef Game::m_data = std::make_shared<GameData>()  [private]
```

Współdzielone dane gry (okno, maszyna stanów).

4.3.4.3 timePerFrame

```
const sf::Time Game::timePerFrame = sf::seconds(1.f / 60.f)  [private]
```

Stały czas trwania jednej klatki logicznej (1/60 sekundy).

Dokumentacja dla tej klasy została wygenerowana z plików:

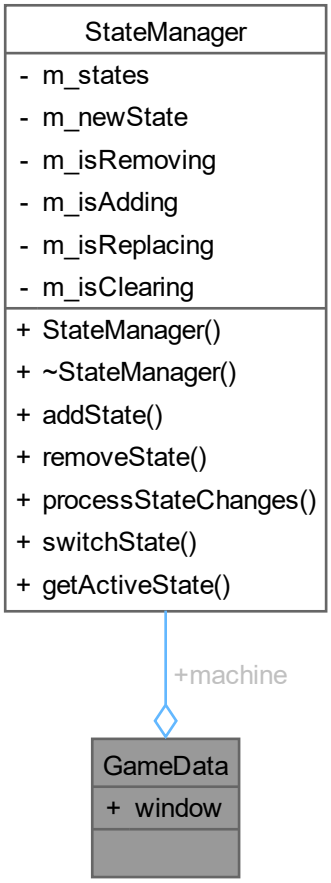
- Game.h
- Game.cpp

4.4 Dokumentacja struktury GameData

Struktura przechowująca współdzielone dane gry.

```
#include <Game.h>
```

Diagram współpracy dla GameData:



Atrybuty publiczne

- StateManager machine

  Maszyna stanów zarządzająca przejściami (Menu -> Gra -> Pauza).
- sf::RenderWindow window

  Główne okno renderowania SFML.

4.4.1 Opis szczegółowy

Struktura przechowująca współdzielone dane gry.

Obiekt tej struktury jest przekazywany jako Smart Pointer (GameDataRef) do wszystkich stanów gry, umożliwiając im dostęp do okna i maszyny stanów.

4.4.2 Dokumentacja atrybutów składowych

4.4.2.1 machine

```
StateManager GameData::machine
```

Maszyna stanów zarządzająca przejściami (Menu -> Gra -> Pauza).

4.4.2.2 window

```
sf::RenderWindow GameData::window
```

Główne okno renderowania SFML.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Game.h

4.5 Dokumentacja klasy GameOverState

Stan gry wyświetlany po przegranej (Game Over).

```
#include <GameOverState.h>
```

Diagram dziedziczenia dla GameOverState

Diagram współpracy dla GameOverState:

Metody publiczne

- GameOverState (GameDataRef data, sf::Sprite bgSprite, std::shared_ptr< sf::Texture > bgTexture, int level)

  Konstruktor stanu GameOver.
- void init () override

  Inicjalizuje elementy interfejsu (teksty, przyciski, tło).
- void handleInput (sf::Event &event) override

Obsługuje wejście od użytkownika (kliknięcia myszką).

- void update (float dt) override

  Aktualizuje logikę stanu (np. animacje przycisków, efekt fade-in).
- void draw (float dt) override

  Rysuje wszystkie elementy stanu na ekranie.

Metody publiczne dziedziczone z State

- virtual ~State ()

  Wirtualny destruktor.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

Atrybuty prywatne

- GameDataRef m_data

  Wskaźnik do danych gry.
- sf::Sprite m_backgroundSprite

  Sprite tła (screenshot gry).
- std::shared_ptr< sf::Texture > m_bgTexture

  Tekstura tła.
- int m_level

  Numer poziomu do ewentualnego restartu.
- sf::RectangleShape m_dimmer

  Prostokąt przyciemniający tło.
- sf::Text m_title

  Tekst "GAME OVER".
- sf::Text m_retryButton

  Przycisk "TRY AGAIN".
- sf::Text m_menuButton

  Przycisk "MAIN MENU".
- sf::RectangleShape m_fadeRect

  Prostokąt do efektu zanikania/pojawiania.
- float m_alpha

  Aktualna przezroczystość fadera.
- bool m_isTransitioning

  Flaga czy trwa przejście między stanami.
- bool m_goToMenu

  Flaga wyboru powrotu do menu.
- bool m_retryLevel

  Flaga wyboru restartu poziomu.

4.5.1 Opis szczegółowy

Stan gry wyświetlany po przegranej (Game Over).

Klasa dziedzicząca po State. Wyświetla komunikat o końcu gry, przyciemnia tło i pozwala na restart poziomu lub powrót do menu.

4.5.2 Dokumentacja konstruktora i destruktora

4.5.2.1 GameOverState()

```
GameOverState::GameOverState (
            GameDataRef data,
            sf::Sprite bgSprite,
            std::shared_ptr< sf::Texture > bgTexture,
            int level )
```

Konstruktor stanu GameOver.

Parametry

data	Współdzielone dane gry.
bgSprite	Zrzut ekranu z momentu przegranej (tło).
bgTexture	Tekstura tła (przechowywana, aby wskaźnik był ważny).
level	Numer poziomu, na którym nastąpiła przegrana (do restartu).

-

4.5.3 Dokumentacja funkcji składowych

4.5.3.1 draw()

```
void GameOverState::draw (
            float dt )  [override], [virtual]
```

Rysuje wszystkie elementy stanu na ekranie.

Parametry

dt	Interpolacja klatki (nieużywana w tym stanie, ale wymagana przez interfejs).

Implementuje State.

4.5.3.2 handleInput()

```
void GameOverState::handleInput (
            sf::Event & event )  [override], [virtual]
```

Obsługuje wejście od użytkownika (kliknięcia myszką).

Parametry

event	Zdarzenie SFML.

Implementuje State.

4.5.3.3 init()

```
void GameOverState::init ( ) [override], [virtual]
```

Inicjalizuje elementy interfejsu (teksty, przyciski, tło).

Implementuje State.

Oto graf wywołań dla tej funkcji:

4.5.3.4 update()

```
void GameOverState::update (
            float dt ) [override], [virtual]
```

Aktualizuje logikę stanu (np. animacje przycisków, efekt fade-in).

Parametry

dt	Czas klatki (Delta Time).

Implementuje State.

4.5.4 Dokumentacja atrybutów składowych

4.5.4.1 m_alpha

```
float GameOverState::m_alpha [private]
```

Aktualna przezroczystość fadera.

4.5.4.2 m_backgroundSprite

```
sf::Sprite GameOverState::m_backgroundSprite [private]
```

Sprite tła (screenshot gry).

4.5.4.3 m_bgTexture

```
std::shared_ptr<sf::Texture> GameOverState::m_bgTexture [private]
```

Tekstura tła.

4.5.4.4 m_data

```
GameDataRef GameOverState::m_data [private]
```

Wskaźnik do danych gry.

4.5.4.5 m_dimmer

```
sf::RectangleShape GameOverState::m_dimmer [private]
```

Prostokąt przyciemniający tło.

4.5.4.6 m_fadeRect

```
sf::RectangleShape GameOverState::m_fadeRect [private]
```

Prostokąt do efektu zanikania/pojawiania.

4.5.4.7 m_goToMenu

```
bool GameOverState::m_goToMenu [private]
```

Flaga wyboru powrotu do menu.

4.5.4.8 m_isTransitioning

```
bool GameOverState::m_isTransitioning [private]
```

Flaga czy trwa przejście między stanami.

4.5.4.9 m_level

```
int GameOverState::m_level [private]
```

Numer poziomu do ewentualnego restartu.

4.5.4.10 m_menuButton

```
sf::Text GameOverState::m_menuButton [private]
```

Przycisk "MAIN MENU".

4.5.4.11 m_retryButton

```
sf::Text GameOverState::m_retryButton [private]
```

Przycisk "TRY AGAIN".

4.5.4.12 m_retryLevel

```
bool GameOverState::m_retryLevel [private]
```

Flaga wyboru restartu poziomu.

4.5.4.13 m_title

```
sf::Text GameOverState::m_title [private]
```

Tekst "GAME OVER".

Dokumentacja dla tej klasy została wygenerowana z plików:

- GameOverState.h
- GameOverState.cpp

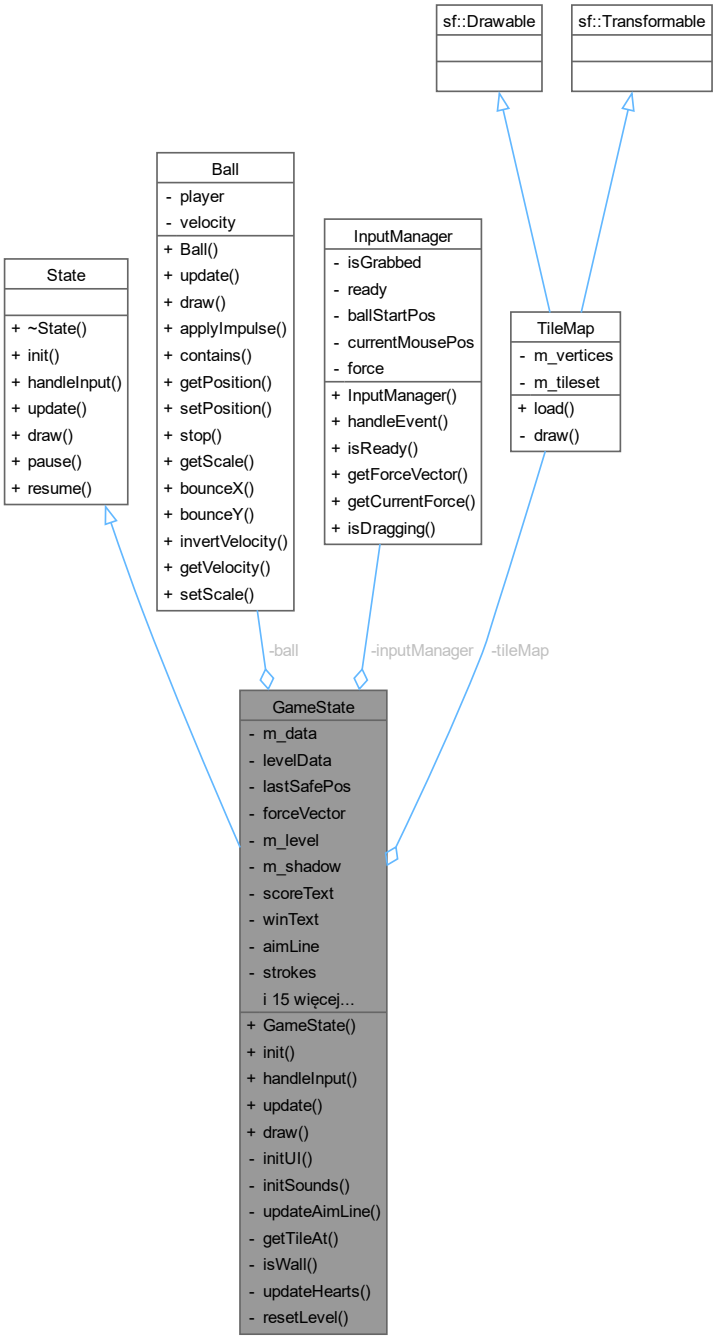
4.6 Dokumentacja klasy GameState

Główny stan rozgrywki.

```
#include <GameState.h>
```

Diagram dziedziczenia dla GameState

Diagram współpracy dla GameState:



Metody publiczne

- GameState (GameDataRef data, int level)

  Konstruktor stanu gry.
- void init () override

  Inicjalizuje poziom: ładuje mapę, ustawia piłkę, UI i dźwięki.
- void handleInput (sf::Event &event) override

Obsługuje wejście (strzał, pauza, reset).

- void update (float dt) override

  Główna pętla fizyki i logiki gry. Wykonuje kroki symulacji fizycznej, sprawdza warunki zwycięstwa/porażki.
- void draw (float dt) override

  Rysuje mapę, piłkę, UI i efekty.

Metody publiczne dziedziczone z State

- virtual ~State ()

  Wirtualny destruktor.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

Metody prywatne

- void initUI ()

  Konfiguracja interfejsu użytkownika.
- void initSounds ()

  Ładowanie i konfiguracja dźwięków.
- void updateAimLine ()

  Aktualizacja wizualizacji linii celowania.
- int getTileAt (sf::Vector2f position)

  Pobiera ID kafelka na danej pozycji w świecie gry.
- bool isWall (int tileId)

  Sprawdza, czy dane ID kafelka jest ścianą.
- void updateHearts ()

  Aktualizacja tekstur serc (pełne/puste).
- void resetLevel ()

  Resetuje stan poziomu po utracie życia lub restarcie.

Atrybuty prywatne

- GameDataRef m_data

  Dane gry.
- Ball ball

  Obiekt gracza (piłka).
- InputManager inputManager

  Menedżer sterowania (myszka).
- TileMap tileMap

  Mapa kafelkowa poziomu.
- std::vector< int > levelData

  Surowe dane mapy (ID kafelków).
- sf::Vector2f lastSafePos

  Ostatnia bezpieczna pozycja (do respawnu).
- sf::Vector2f forceVector

  Wektor siły uderzenia.
- int m_level

Numer aktualnego poziomu.

- sf::CircleShape m_shadow

  Cień pod piłką (używany przy skokach).
- sf::Text scoreText

  Licznik uderzeń.
- sf::Text winText

  Tekst wygranej (opcjonalny).
- sf::RectangleShape aimLine

  Linia celownicza.
- int strokes

  Liczba wykonanych uderzeń.
- bool ballInHole

  Czy piłka wpadła do dołka.
- bool wasInWater

  Czy odegrano dźwięk wpadnięcia do wody.
- bool isSinking

  Czy trwa animacja tonięcia.
- bool isJumping

  Czy piłka jest w powietrzu (skok z rampy).
- float jumpTimer

  Czas pozostały do końca skoku.
- float levelFinishTimer

  Opóźnienie po wpadnięciu do dołka przed zmianą stanu.
- float maxJumpTime

  Całkowity czas trwania aktualnego skoku (do obliczeń skali).
- sf::Sound m_hitSound

  Dźwięk uderzenia.
- sf::Sound m_splashSound

  Dźwięk wpadnięcia do wody.
- sf::Sound m_wallHitSound

  Dźwięk uderzenia o ścianę.
- sf::Sound m_winSound

  Dźwięk zwycięstwa.
- sf::RectangleShape m_fadeRect
- float m_alpha
- int lives

  Liczba żyć (serc).
- std::vector< sf::Sprite > heartSprites

  Graficzna reprezentacja żyć.

4.6.1 Opis szczegółowy

Główny stan rozgrywki.

Klasa ta implementuje pętlę gry, w tym:

- Fizykę ruchu piłki (tarcie, odbicia, grawitacja przy skokach).
- Detekcję kolizji z mapą kafelkową (TileMap).
- Logikę interakcji z elementami specjalnymi (woda, piasek, rampy, dołek).
- Zarządzanie cyklem życia (śmierć, wygrana, pauza).

4.6.2 Dokumentacja konstruktora i destruktora

4.6.2.1 GameState()

```
GameState::GameState (
            GameDataRef data,
            int level )
```

Konstruktor stanu gry.

Parametry

data	Dane gry.
level	Numer poziomu do załadowania.

4.6.3 Dokumentacja funkcji składowych

4.6.3.1 draw()

```
void GameState::draw (
            float dt )  [override], [virtual]
```

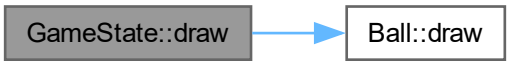
Rysuje mapę, piłkę, UI i efekty.

Parametry

dt	Interpolacja klatki.

Implementuje State.

Oto graf wywołań dla tej funkcji:



4.6.3.2 getTileAt()

```
int GameState::getTileAt (
            sf::Vector2f position )  [private]
```

Pobiera ID kafelka na danej pozycji w świecie gry.

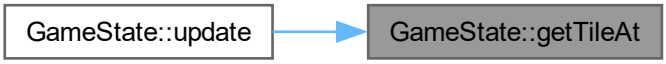
Parametry

position	Współrzędne (x, y) w pikselach.

Zwraca

ID kafelka (int). Zwraca ID ściany (3) jeśli poza zakresem.

Oto graf wywoływań tej funkcji:



4.6.3.3 handleInput()

```
void GameState::handleInput (
            sf::Event & event )  [override], [virtual]
```

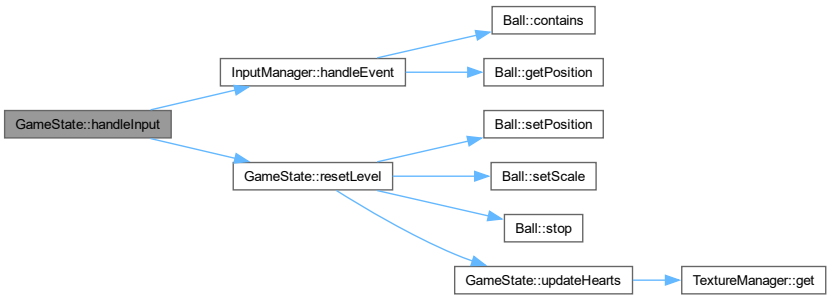
Obsługuje wejście (strzał, pauza, reset).

Parametry

event	Zdarzenie SFML.

Implementuje State.

Oto graf wywołań dla tej funkcji:



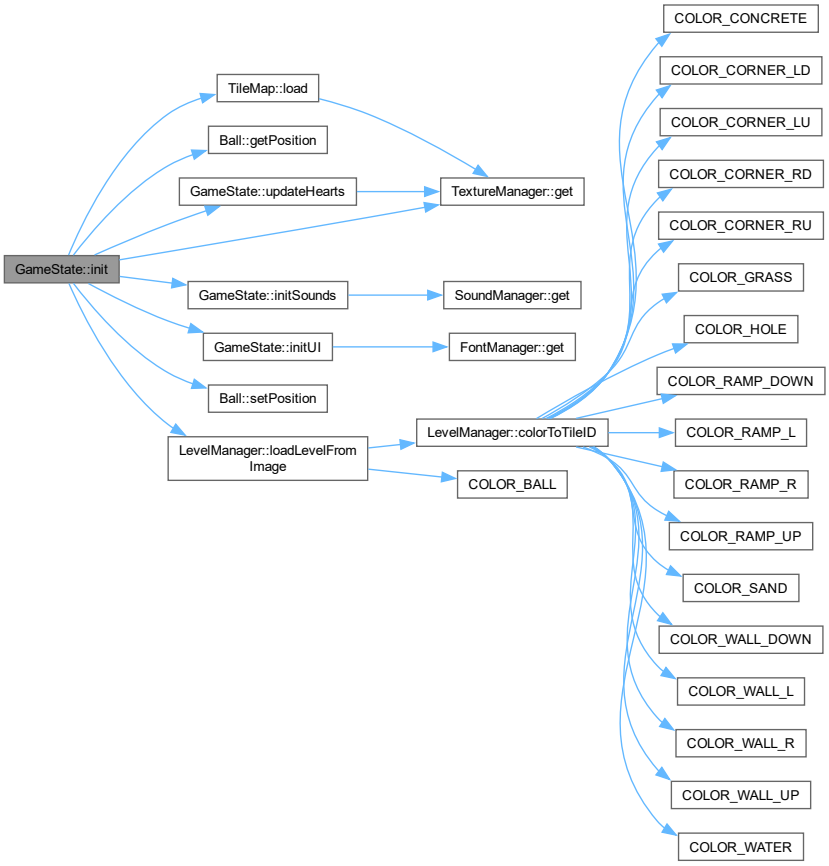
4.6.3.4 init()

```
void GameState::init ( ) [override], [virtual]
```

Inicjalizuje poziom: ładuje mapę, ustawia piłkę, UI i dźwięki.

Implementuje State.

Oto graf wywołań dla tej funkcji:



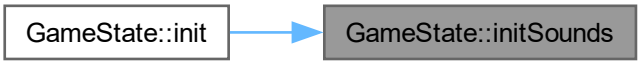
4.6.3.5 initSounds()

```
void GameState::initSounds ( ) [private]
```

Ładowanie i konfiguracja dźwięków.

Oto graf wywołań dla tej funkcji:

Oto graf wywoływań tej funkcji:

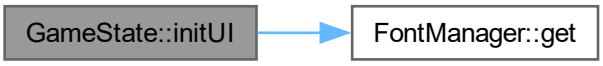


4.6.3.6 initUI()

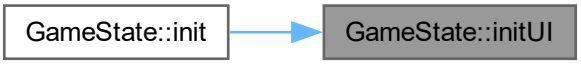
```
void GameState::initUI ( ) [private]
```

Konfiguracja interfejsu użytkownika.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



4.6.3.7 isWall()

```
bool GameState::isWall (
            int tileId ) [private]
```

Sprawdza, czy dane ID kafelka jest ścianą.

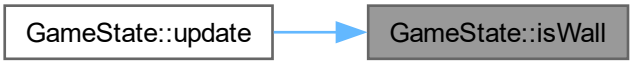
Parametry

tileId	ID kafelka.

Zwraca

true jeśli kafelek jest przeszkodą stałą.

Oto graf wywoływań tej funkcji:

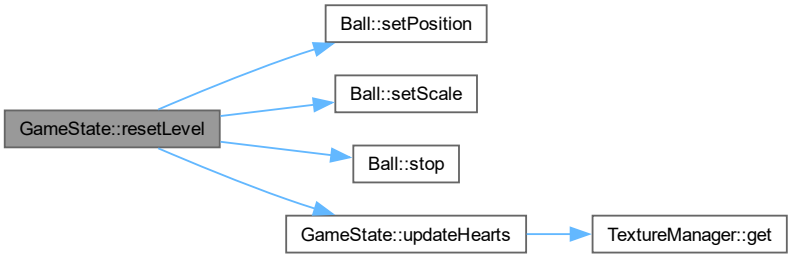


4.6.3.8 resetLevel()

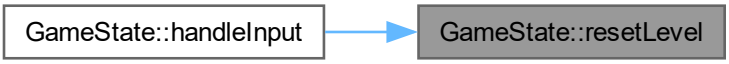
```
void GameState::resetLevel ( ) [private]
```

Resetuje stan poziomu po utracie życia lub restarcie.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



4.6.3.9 update()

```
void GameState::update (
            float dt )  [override], [virtual]
```

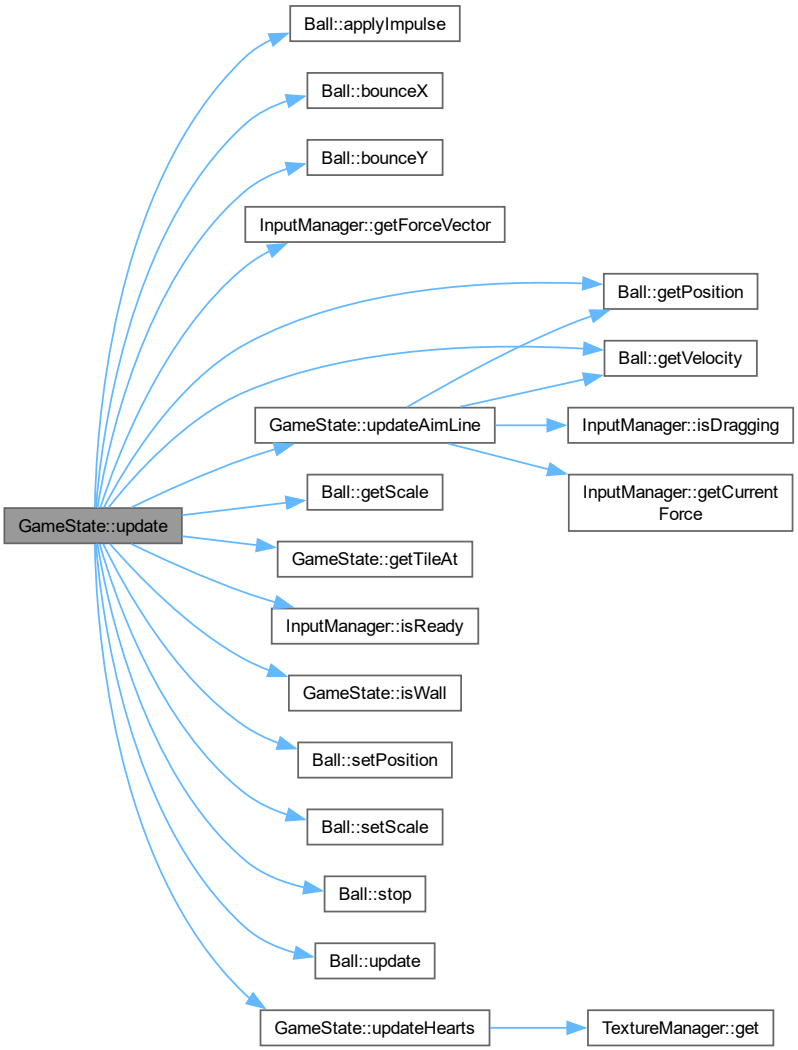
Główna pętla fizyki i logiki gry. Wykonuje kroki symulacji fizycznej, sprawdza warunki zwycięstwa/porażki.

Parametry

dt	Czas klatki (Delta Time).

Implementuje State.

Oto graf wywołań dla tej funkcji:

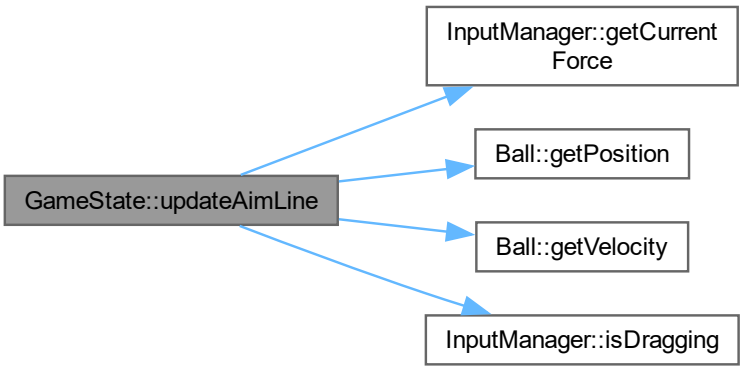


4.6.3.10 updateAimLine()

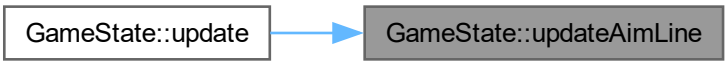
```
void GameState::updateAimLine ( ) [private]
```

Aktualizacja wizualizacji linii celowania.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



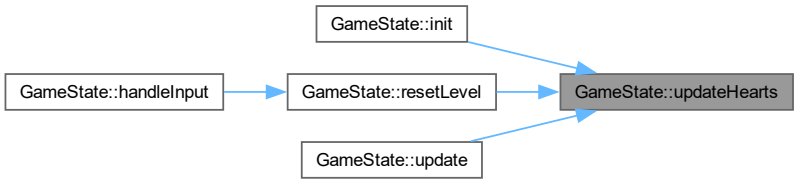
4.6.3.11 updateHearts()

```
void GameState::updateHearts ( ) [private]
```

Aktualizacja tekstur serc (pełne/puste).

Oto graf wywołań dla tej funkcji:

Oto graf wywoływań tej funkcji:



4.6.4 Dokumentacja atrybutów składowych

4.6.4.1 aimLine

```
sf::RectangleShape GameState::aimLine [private]
```

Linia celownicza.

4.6.4.2 ball

```
Ball GameState::ball [private]
```

Obiekt gracza (piłka).

4.6.4.3 ballInHole

```
bool GameState::ballInHole [private]
```

Czy piłka wpadła do dołka.

4.6.4.4 forceVector

```
sf::Vector2f GameState::forceVector [private]
```

Wektor siły uderzenia.

4.6.4.5 heartSprites

```
std::vector<sf::Sprite> GameState::heartSprites [private]
```

Graficzna reprezentacja żyć.

4.6.4.6 inputManager

```
InputManager GameState::inputManager [private]
```

Menedżer sterowania (myszka).

4.6.4.7 isJumping

```
bool GameState::isJumping [private]
```

Czy piłka jest w powietrzu (skok z rampy).

4.6.4.8 isSinking

```
bool GameState::isSinking [private]
```

Czy trwa animacja tonięcia.

4.6.4.9 jumpTimer

```
float GameState::jumpTimer [private]
```

Czas pozostały do końca skoku.

4.6.4.10 lastSafePos

```
sf::Vector2f GameState::lastSafePos [private]
```

Ostatnia bezpieczna pozycja (do respawnu).

4.6.4.11 levelData

```
std::vector<int> GameState::levelData [private]
```

Surowe dane mapy (ID kafelków).

4.6.4.12 levelFinishTimer

```
float GameState::levelFinishTimer [private]
```

Opóźnienie po wpadnięciu do dołka przed zmianą stanu.

4.6.4.13 lives

```
int GameState::lives [private]
```

Liczba żyć (serc).

4.6.4.14 m_alpha

```
float GameState::m_alpha [private]
```

4.6.4.15 m_data

```
GameDataRef GameState::m_data [private]
```

Dane gry.

4.6.4.16 m_fadeRect

```
sf::RectangleShape GameState::m_fadeRect [private]
```

4.6.4.17 m_hitSound

```
sf::Sound GameState::m_hitSound [private]
```

Dźwięk uderzenia.

4.6.4.18 m_level

```
int GameState::m_level [private]
```

Numer aktualnego poziomu.

4.6.4.19 m_shadow

```
sf::CircleShape GameState::m_shadow [private]
```

Cień pod piłką (używany przy skokach).

4.6.4.20 m_splashSound

```
sf::Sound GameState::m_splashSound [private]
```

Dźwięk wpadnięcia do wody.

4.6.4.21 m_wallHitSound

```
sf::Sound GameState::m_wallHitSound [private]
```

Dźwięk uderzenia o ścianę.

4.6.4.22 m_winSound

```
sf::Sound GameState::m_winSound [private]
```

Dźwięk zwycięstwa.

4.6.4.23 maxJumpTime

```
float GameState::maxJumpTime [private]
```

Całkowity czas trwania aktualnego skoku (do obliczeń skali).

4.6.4.24 scoreText

```
sf::Text GameState::scoreText [private]
```

Licznik uderzeń.

4.6.4.25 strokes

```
int GameState::strokes [private]
```

Liczba wykonanych uderzeń.

4.6.4.26 tileMap

```
TileMap GameState::tileMap [private]
```

Mapa kafelkowa poziomu.

4.6.4.27 wasInWater

```
bool GameState::wasInWater [private]
```

Czy odegrano dźwięk wpadnięcia do wody.

4.6.4.28 winText

```
sf::Text GameState::winText [private]
```

Tekst wygranej (opcjonalny).

Dokumentacja dla tej klasy została wygenerowana z plików:

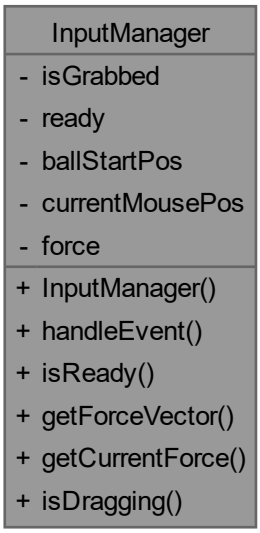
- GameState.h
- GameState.cpp

4.7 Dokumentacja klasy InputManager

Klasa obsługująca sterowanie (mechanika Drag & Shoot).

```
#include <InputManager.h>
```

Diagram współpracy dla InputManager:



Metody publiczne

- InputManager ()

  Konstruktor inicjalizujący stan wejścia.
- void handleEvent (const sf::Event &event, Ball &ball, sf::RenderWindow &window)

  Przetwarza zdarzenia wejścia SFML.
- bool isReady () const

  Sprawdza, czy gracz wykonał strzał (puścił przycisk myszy).
- sf::Vector2f getForceVector ()

  Oblicza i zwraca wektor siły uderzenia.
- sf::Vector2f getCurrentForce (const sf::RenderWindow &window) const

  Oblicza aktualny wektor siły (do wizualizacji linii celowania). Nie resetuje stanu 'ready'.
- bool isDragging () const

  Sprawdza, czy gracz jest w trakcie celowania (trzyma przycisk).

Atrybuty prywatne

- bool isGrabbed

  Czy gracz trzyma piłkę (lewy przycisk myszy wciśnięty).
- bool ready

  Czy strzał jest gotowy do wykonania (puszczenie przycisku).
- sf::Vector2f ballStartPos

  Pozycja piłki w momencie rozpoczęcia celowania.
- sf::Vector2f currentMousePos

  Aktualna pozycja myszy.
- float force

  Obliczona siła uderzenia (nieużywana bezpośrednio, obliczana w wektorze).

4.7.1 Opis szczegółowy

Klasa obsługująca sterowanie (mechanika Drag & Shoot).

Odpowiada za wykrywanie kliknięć na piłce, przeciąganie myszką (celowanie) oraz obliczanie wektora siły uderzenia.

4.7.2 Dokumentacja konstruktora i destruktora

4.7.2.1 InputManager()

```
InputManager::InputManager ( )
```

Konstruktor inicjalizujący stan wejścia.

4.7.3 Dokumentacja funkcji składowych

4.7.3.1 getCurrentForce()

```
sf::Vector2f InputManager::getCurrentForce (
            const sf::RenderWindow & window ) const
```

Oblicza aktualny wektor siły (do wizualizacji linii celowania). Nie resetuje stanu 'ready'.

Parametry

window	Okno gry.

Zwraca

Wektor siły.

Oto graf wywoływań tej funkcji:

4.7.3.2 getForceVector()

```
sf::Vector2f InputManager::getForceVector ( )
```

Oblicza i zwraca wektor siły uderzenia.

Wektor jest obliczany na podstawie różnicy między pozycją startową a pozycją myszy. Siła jest limitowana do maksymalnej wartości. Po pobraniu wektora flaga 'ready' jest resetowana.

Zwraca

Wektor siły (sf::Vector2f).

Oto graf wywoływań tej funkcji:

4.7.3.3 handleEvent()

```
void InputManager::handleEvent (
            const sf::Event & event,
            Ball & ball,
            sf::RenderWindow & window )
```

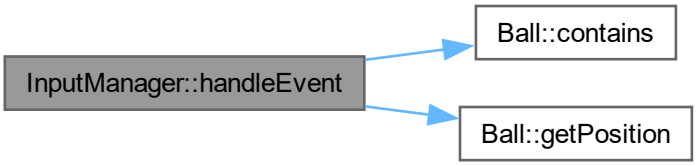
Przetwarza zdarzenia wejścia SFML.

Obsługuje MouseButtonPressed (chwycenie), MouseMoved (celowanie) i MouseButtonReleased (strzał).

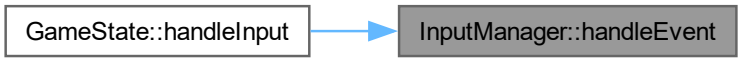
Parametry

event	Zdarzenie SFML.
ball	Referencja do obiektu piłki (do sprawdzenia czy kliknięto w nią).
window	Referencja do okna (do mapowania współrzędnych).

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



4.7.3.4 isDragging()

```
bool InputManager::isDragging ( ) const
```

Sprawdza, czy gracz jest w trakcie celowania (trzyma przycisk).

Zwraca

true jeśli trwa celowanie.

Oto graf wywoływań tej funkcji:

4.7.3.5 isReady()

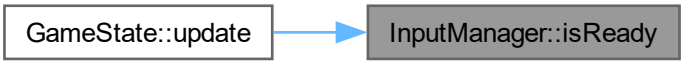
```
bool InputManager::isReady ( ) const
```

Sprawdza, czy gracz wykonał strzał (puścił przycisk myszy).

Zwraca

true jeśli strzał jest gotowy.

Oto graf wywoływań tej funkcji:



4.7.4 Dokumentacja atrybutów składowych

4.7.4.1 ballStartPos

```
sf::Vector2f InputManager::ballStartPos [private]
```

Pozycja piłki w momencie rozpoczęcia celowania.

4.7.4.2 currentMousePos

```
sf::Vector2f InputManager::currentMousePos [private]
```

Aktualna pozycja myszy.

4.7.4.3 force

```
float InputManager::force [private]
```

Obliczona siła uderzenia (nieużywana bezpośrednio, obliczana w wektorze).

4.7.4.4 isGrabbed

```
bool InputManager::isGrabbed [private]
```

Czy gracz trzyma piłkę (lewy przycisk myszy wciśnięty).

4.7.4.5 ready

```
bool InputManager::ready [private]
```

Czy strzał jest gotowy do wykonania (puszczenie przycisku).

Dokumentacja dla tej klasy została wygenerowana z plików:

- InputManager.h
- InputManager.cpp

4.8 Dokumentacja klasy LeaderboardState

Stan wyświetlający tabelę najlepszych wyników.

```
#include <LeaderBoardState.h>
```

Diagram dziedziczenia dla LeaderboardState

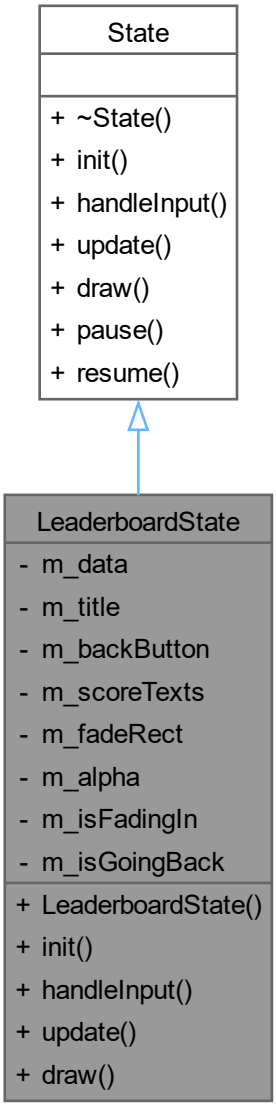
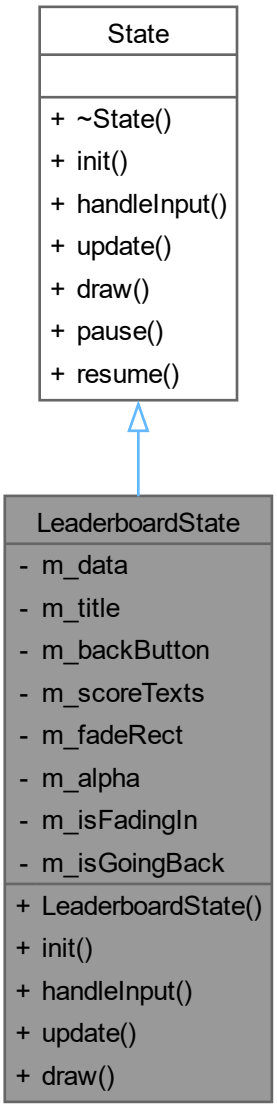


Diagram współpracy dla LeaderboardState:



Metody publiczne

- LeaderboardState (GameDataRef data)

  Konstruktor stanu tabeli wyników.
- void init () override

  Inicjalizuje widok, wczytuje wyniki i tworzy obiekty tekstowe.
- void handleInput (sf::Event &event) override

  Obsługuje przycisk powrotu (BACK).
- void update (float dt) override

  Aktualizuje efekty wizualne (fade-in, hover).
- void draw (float dt) override

  Rysuje tabelę wyników.

Metody publiczne dziedziczone z State

- virtual ~State ()

  Wirtualny destruktor.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

Atrybuty prywatne

- GameDataRef m_data

  Dane gry.
- sf::Text m_title

  Tytuł "BEST SCORES".
- sf::Text m_backButton

  Przycisk powrotu.
- std::vector< sf::Text > m_scoreTexts

  Wektor tekstów z wynikami dla każdego poziomu.
- sf::RectangleShape m_fadeRect

  Prostokąt przejścia.
- float m_alpha

  Przezroczystość przejścia.
- bool m_isFadingIn

  Czy trwa animacja wejścia.
- bool m_isGoingBack

  Czy trwa powrót do menu.

4.8.1 Opis szczegółowy

Stan wyświetlający tabelę najlepszych wyników.

Pobiera dane z SaveManager i prezentuje je w formie listy tekstowej.

4.8.2 Dokumentacja konstruktora i destruktora

4.8.2.1 LeaderboardState()

```
LeaderboardState::LeaderboardState (
            GameDataRef data )
```

Konstruktor stanu tabeli wyników.

Parametry

data	Dane gry.

4.8.3 Dokumentacja funkcji składowych

4.8.3.1 draw()

```
void LeaderboardState::draw (
            float dt )  [override], [virtual]
```

Rysuje tabelę wyników.

Implementuje State.

4.8.3.2 handleInput()

```
void LeaderboardState::handleInput (
            sf::Event & event )  [override], [virtual]
```

Obsługuje przycisk powrotu (BACK).

Implementuje State.

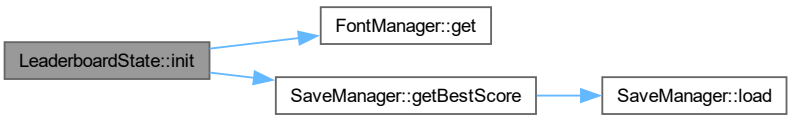
4.8.3.3 init()

```
void LeaderboardState::init ( )  [override], [virtual]
```

Inicjalizuje widok, wczytuje wyniki i tworzy obiekty tekstowe.

Implementuje State.

Oto graf wywołań dla tej funkcji:



4.8.3.4 update()

```
void LeaderboardState::update (
            float dt )  [override], [virtual]
```

Aktualizuje efekty wizualne (fade-in, hover).

Implementuje State.

4.8.4 Dokumentacja atrybutów składowych

4.8.4.1 m_alpha

```
float LeaderboardState::m_alpha [private]
```

Przezroczystość przejścia.

4.8.4.2 m_backButton

```
sf::Text LeaderboardState::m_backButton [private]
```

Przycisk powrotu.

4.8.4.3 m_data

```
GameDataRef LeaderboardState::m_data [private]
```

Dane gry.

4.8.4.4 m_fadeRect

```
sf::RectangleShape LeaderboardState::m_fadeRect [private]
```

Prostokąt przejścia.

4.8.4.5 m_isFadingIn

```
bool LeaderboardState::m_isFadingIn [private]
```

Czy trwa animacja wejścia.

4.8.4.6 m_isGoingBack

```
bool LeaderboardState::m_isGoingBack [private]
```

Czy trwa powrót do menu.

4.8.4.7 m_scoreTexts

```
std::vector<sf::Text> LeaderboardState::m_scoreTexts [private]
```

Wektor tekstów z wynikami dla każdego poziomu.

4.8.4.8 m_title

```
sf::Text LeaderboardState::m_title [private]
```

Tytuł "BEST SCORES".

Dokumentacja dla tej klasy została wygenerowana z plików:

- LeaderBoardState.h
- LeaderBoardState.cpp

4.9 Dokumentacja klasy LevelCompleteState

Stan gry wyświetlany po pomyślnym ukończeniu poziomu.

```
#include <LevelCompleteState.h>
```

Diagram dziedziczenia dla LevelCompleteState

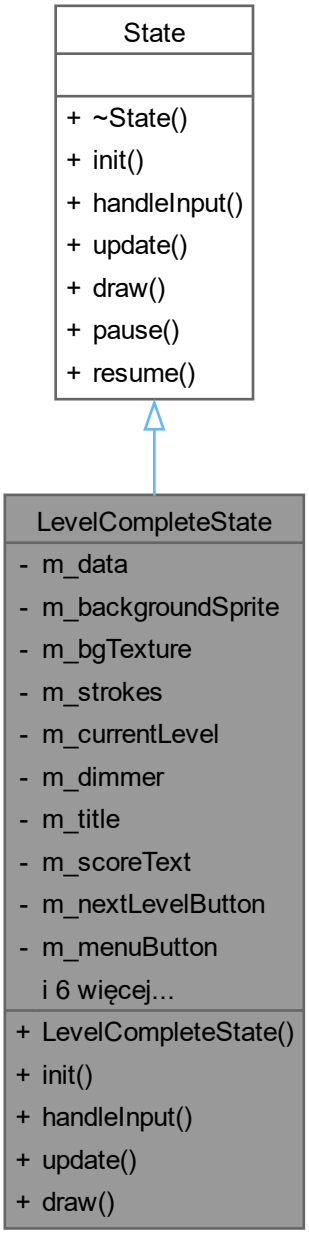
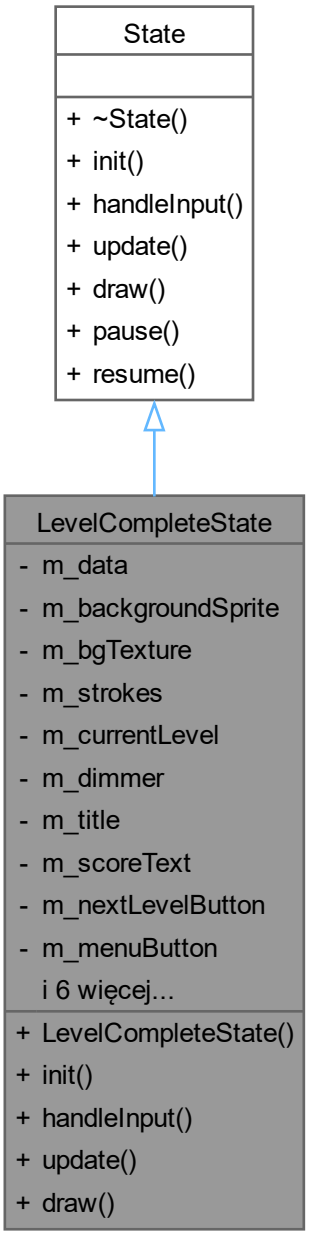


Diagram współpracy dla LevelCompleteState:



Metody publiczne

- LevelCompleteState (GameDataRef data, sf::Sprite bgSprite, std::shared_ptr< sf::Texture > bgTexture, int strokes, int currentLevel)

  Konstruktor stanu ukończenia poziomu.
- void init () override

  Inicjalizuje elementy interfejsu (teksty, przyciski) oraz zapisuje wynik.
- void handleInput (sf::Event &event) override

Obsługuje interakcję użytkownika z przyciskami (Next Level, Retry, Menu).

- void update (float dt) override

  Aktualizuje logikę stanu (animacje przycisków, efekt przejścia).
- void draw (float dt) override

  Renderuje elementy stanu na ekranie.

Metody publiczne dziedziczone z State

- virtual ~State ()

  Wirtualny destruktor.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

Atrybuty prywatne

- GameDataRef m_data

  Wskaźnik do danych gry.
- sf::Sprite m_backgroundSprite

  Tło (screenshot z gry).
- std::shared_ptr< sf::Texture > m_bgTexture

  Tekstura tła.
- int m_strokes

  Wynik gracza.
- int m_currentLevel

  Numer ukończonego poziomu.
- sf::RectangleShape m_dimmer

  Prostokąt przyciemniający tło.
- sf::Text m_title

  Nagłówek "LEVEL COMPLETE".
- sf::Text m_scoreText

  Tekst wyświetlający liczbę uderzeń.
- sf::Text m_nextLevelButton

  Przycisk przejścia do kolejnego poziomu.
- sf::Text m_menuButton

  Przycisk powrotu do menu.
- sf::Text m_retryButton

  Przycisk powtórzenia poziomu.
- sf::RectangleShape m_fadeRect

  Prostokąt do efektu zanikania.
- float m_alpha

  Aktualna przezroczystość fadera.
- bool m_isTransitioning

  Flaga trwania animacji wyjścia.
- bool m_goToMenu

  Cel przejścia: Menu.
- bool m_retryLevel

  Cel przejścia: Restart.

4.9.1 Opis szczegółowy

Stan gry wyświetlany po pomyślnym ukończeniu poziomu.

Klasa ta odpowiada za:

- Wyświetlenie podsumowania poziomu (wynik/ilość uderzeń).
- Zapisanie postępów gracza (odblokowanie kolejnego poziomu).
- Nawigację do następnego poziomu, powtórzenia obecnego lub powrotu do menu.

4.9.2 Dokumentacja konstruktora i destruktora

4.9.2.1 LevelCompleteState()

```
LevelCompleteState::LevelCompleteState (
            GameDataRef data,
            sf::Sprite bgSprite,
            std::shared_ptr< sf::Texture > bgTexture,
            int strokes,
            int currentLevel )
```

Konstruktor stanu ukończenia poziomu.

Parametry

data	Współdzielone dane gry (okno, maszyna stanów).
bgSprite	Zrzut ekranu z gry (tło).
bgTexture	Tekstura zrzutu ekranu.
strokes	Liczba uderzeń wykonana w zakończonym poziomie.
currentLevel	Numer ukończonego poziomu.

4.9.3 Dokumentacja funkcji składowych

4.9.3.1 draw()

```
void LevelCompleteState::draw (
            float dt )  [override], [virtual]
```

Renderuje elementy stanu na ekranie.

Parametry

dt	Interpolacja (nieużywana).

Implementuje State.

4.9.3.2 handleInput()

```
void LevelCompleteState::handleInput (
            sf::Event & event )  [override], [virtual]
```

Obsługuje interakcję użytkownika z przyciskami (Next Level, Retry, Menu).

Parametry

event	Zdarzenie SFML.

Implementuje State.

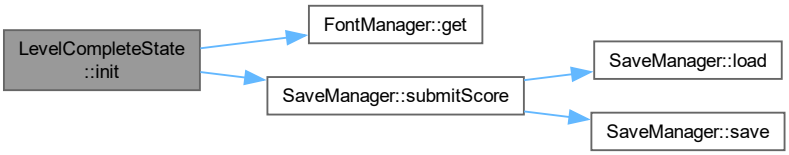
4.9.3.3 init()

```
void LevelCompleteState::init ( )  [override], [virtual]
```

Inicjalizuje elementy interfejsu (teksty, przyciski) oraz zapisuje wynik.

Implementuje State.

Oto graf wywołań dla tej funkcji:



4.9.3.4 update()

```
void LevelCompleteState::update (
            float dt )  [override], [virtual]
```

Aktualizuje logikę stanu (animacje przycisków, efekt przejścia).

Parametry

dt	Czas klatki (Delta Time).

Implementuje State.

4.9.4 Dokumentacja atrybutów składowych

4.9.4.1 m_alpha

```
float LevelCompleteState::m_alpha [private]
```

Aktualna przezroczystość fadera.

4.9.4.2 m_backgroundSprite

```
sf::Sprite LevelCompleteState::m_backgroundSprite [private]
```

Tło (screenshot z gry).

4.9.4.3 m_bgTexture

```
std::shared_ptr<sf::Texture> LevelCompleteState::m_bgTexture [private]
```

Tekstura tła.

4.9.4.4 m_currentLevel

```
int LevelCompleteState::m_currentLevel [private]
```

Numer ukończonego poziomu.

4.9.4.5 m_data

```
GameDataRef LevelCompleteState::m_data [private]
```

Wskaźnik do danych gry.

4.9.4.6 m_dimmer

```
sf::RectangleShape LevelCompleteState::m_dimmer [private]
```

Prostokąt przyciemniający tło.

4.9.4.7 m_fadeRect

```
sf::RectangleShape LevelCompleteState::m_fadeRect [private]
```

Prostokąt do efektu zanikania.

4.9.4.8 m_goToMenu

```
bool LevelCompleteState::m_goToMenu [private]
```

Cel przejścia: Menu.

4.9.4.9 m_isTransitioning

```
bool LevelCompleteState::m_isTransitioning [private]
```

Flaga trwania animacji wyjścia.

4.9.4.10 m_menuButton

```
sf::Text LevelCompleteState::m_menuButton [private]
```

Przycisk powrotu do menu.

4.9.4.11 m_nextLevelButton

```
sf::Text LevelCompleteState::m_nextLevelButton [private]
```

Przycisk przejścia do kolejnego poziomu.

4.9.4.12 m_retryButton

```
sf::Text LevelCompleteState::m_retryButton [private]
```

Przycisk powtórzenia poziomu.

4.9.4.13 m_retryLevel

```
bool LevelCompleteState::m_retryLevel [private]
```

Cel przejścia: Restart.

4.9.4.14 m_scoreText

```
sf::Text LevelCompleteState::m_scoreText [private]
```

Tekst wyświetlający liczbę uderzeń.

4.9.4.15 m_strokes

```
int LevelCompleteState::m_strokes [private]
```

Wynik gracza.

4.9.4.16 m_title

```
sf::Text LevelCompleteState::m_title [private]
```

Nagłówek "LEVEL COMPLETE".

Dokumentacja dla tej klasy została wygenerowana z plików:

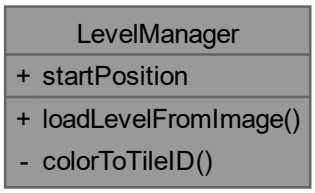
- LevelCompleteState.h
- LevelCompleteState.cpp

4.10 Dokumentacja klasy LevelManager

Klasa statyczna odpowiedzialna za wczytywanie poziomów.

```
#include <LevelManager.h>
```

Diagram współpracy dla LevelManager:



Statyczne metody publiczne

- static std::vector< int > loadLevelFromImage (int levelNumber)

  Wczytuje poziom z pliku obrazu.

Statyczne atrybuty publiczne

- static sf::Vector2f startPosition = sf::Vector2f(400.f, 300.f)

  Pozycja startowa gracza wykryta podczas ładowania ostatniego poziomu.

Statyczne metody prywatne

- static int colorToTileID (sf::Color color)

  Konwertuje kolor piksela na ID kafelka.

4.10.1 Opis szczegółowy

Klasa statyczna odpowiedzialna za wczytywanie poziomów.

Implementuje parsowanie plików graficznych (PNG) na strukturę mapy kafelkowej. Mapuje kolory pikseli na identyfikatory kafelków (Tile ID).

4.10.2 Dokumentacja funkcji składowych

4.10.2.1 colorToTileID()

```
int LevelManager::colorToTileID (
            sf::Color color )  [static], [private]
```

Konwertuje kolor piksela na ID kafelka.

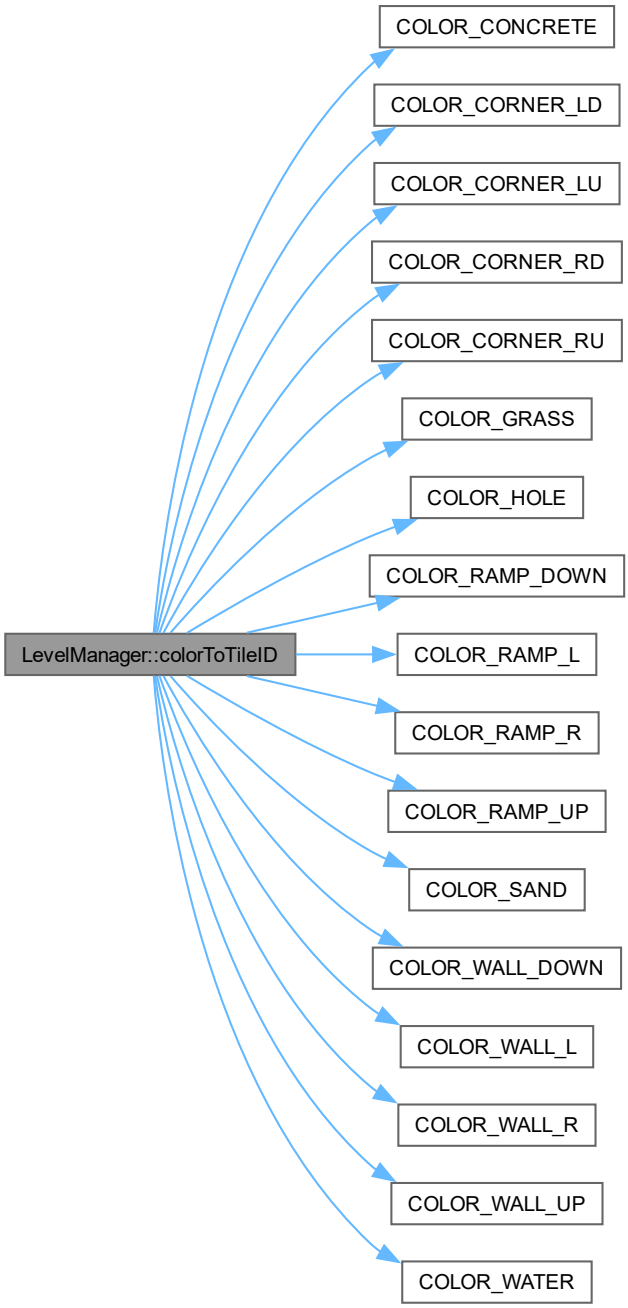
Parametry

color	Kolor piksela z mapy źródłowej.

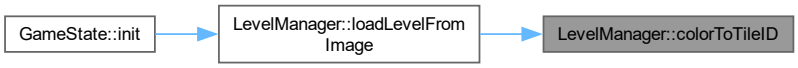
Zwraca

ID kafelka zgodne z definicjami w DEFINITIONS.h.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



4.10.2.2 loadLevelFromImage()

```
std::vector< int > LevelManager::loadLevelFromImage (
            int levelNumber ) [static]
```

Wczytuje poziom z pliku obrazu.

Iteruje po pikselach obrazu i na podstawie ich koloru generuje tablicę ID kafelków. Wykrywa również pozycję startową gracza (biały piksel).

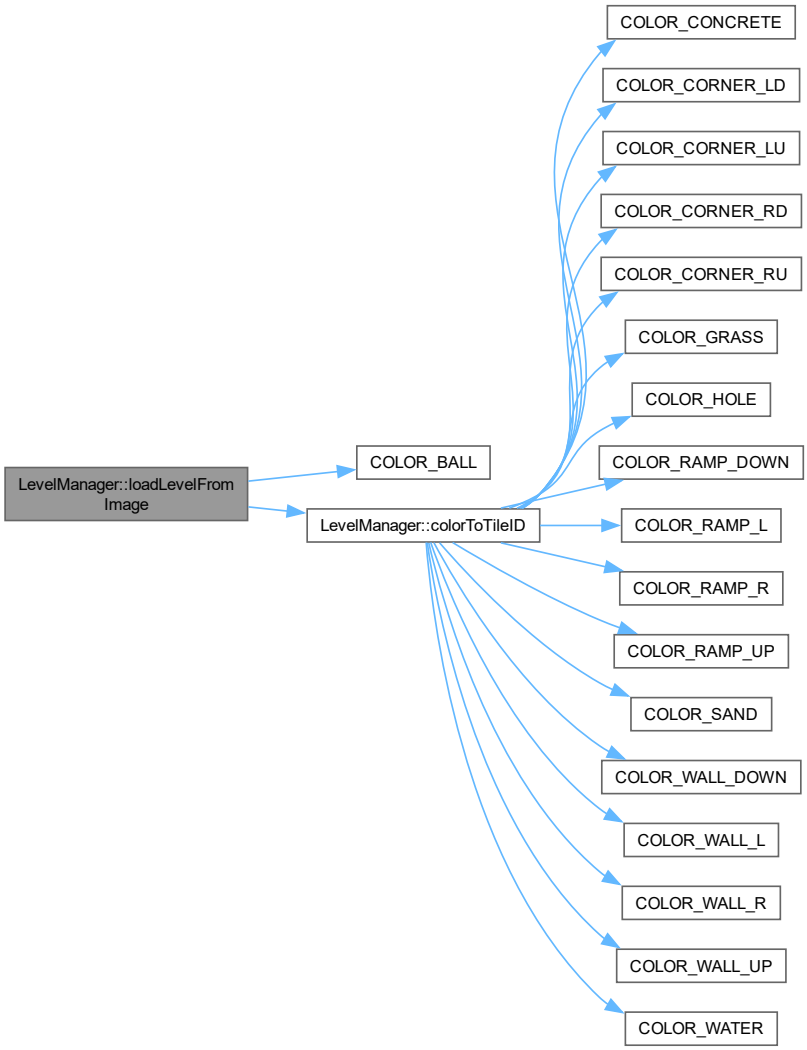
Parametry

levelNumber	Numer poziomu do wczytania (np. 1 dla "level1.png").

Zwraca

Wektor liczb całkowitych reprezentujący mapę kafelków.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

4.10.3 Dokumentacja atrybutów składowych

4.10.3.1 startPosition

```
sf::Vector2f LevelManager::startPosition = sf::Vector2f(400.f, 300.f)  [static]
```

Pozycja startowa gracza wykryta podczas ładowania ostatniego poziomu.

Dokumentacja dla tej klasy została wygenerowana z plików:

- LevelManager.h
- LevelManager.cpp

4.11 Dokumentacja klasy LevelSelectState

Stan wyboru poziomu.

```
#include <LevelSelectState.h>
```

Diagram dziedziczenia dla LevelSelectState

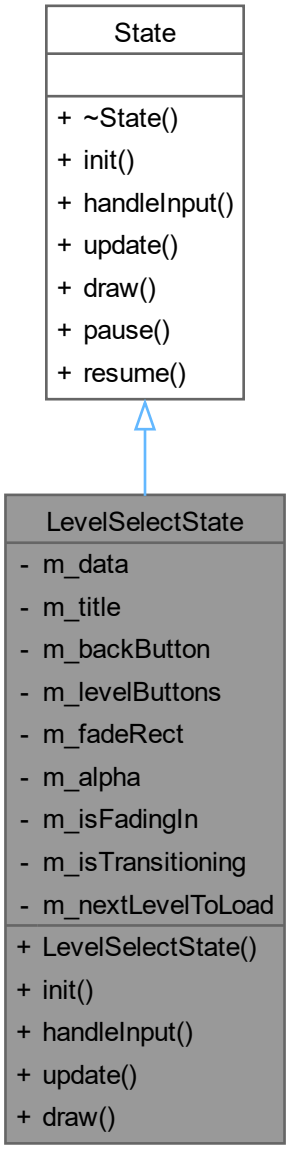
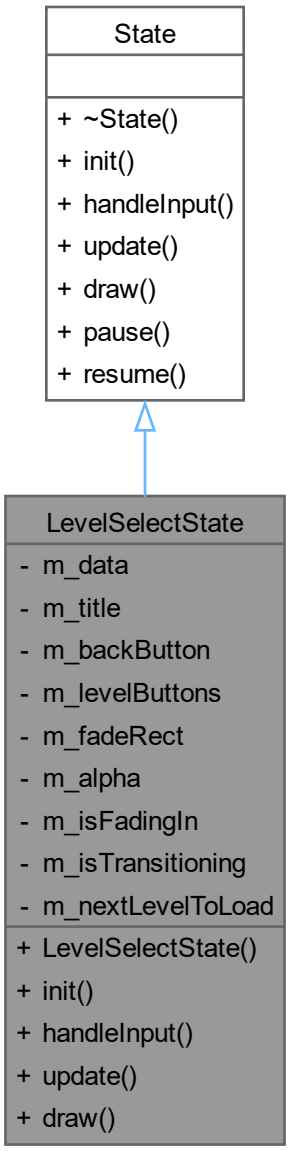


Diagram współpracy dla LevelSelectState:



Metody publiczne

- LevelSelectState (GameDataRef data)

  Konstruktor stanu wyboru poziomu.
- void init () override

  Tworzy listę przycisków na podstawie liczby poziomów i postępu gracza.
- void handleInput (sf::Event &event) override

  Obsługuje wybór poziomu lub powrót do menu. Blokuje kliknięcia w nieodblokowane poziomy.
- void update (float dt) override

  Aktualizuje logikę podświetlania dostępnych poziomów.

- void draw (float dt) override

  Rysuje listę poziomów.

Metody publiczne dziedziczone z State

- virtual ~State ()

  Wirtualny destruktor.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

Atrybuty prywatne

- GameDataRef m_data
- sf::Text m_title

  Nagłówek "SELECT LEVEL".
- sf::Text m_backButton

  Przycisk powrotu.
- std::vector< sf::Text > m_levelButtons

  Dynamiczna lista przycisków poziomów.
- sf::RectangleShape m_fadeRect
- float m_alpha
- bool m_isFadingIn
- bool m_isTransitioning
- int m_nextLevelToLoad

  Numer wybranego poziomu (-1 oznacza powrót).

4.11.1 Opis szczegółowy

Stan wyboru poziomu.

Wyświetla listę dostępnych poziomów. Komunikuje się z SaveManagerem, aby zablokować poziomy, do których gracz nie ma jeszcze dostępu (oznaczone jako LOCKED).

4.11.2 Dokumentacja konstruktora i destruktora

4.11.2.1 LevelSelectState()

```
LevelSelectState::LevelSelectState (
            GameDataRef data )
```

Konstruktor stanu wyboru poziomu.

Parametry

data	Dane gry.

4.11.3 Dokumentacja funkcji składowych

4.11.3.1 draw()

```
void LevelSelectState::draw (
            float dt )  [override], [virtual]
```

Rysuje listę poziomów.

Parametry

dt	Interpolacja.

Implementuje State.

4.11.3.2 handleInput()

```
void LevelSelectState::handleInput (
            sf::Event & event )  [override], [virtual]
```

Obsługuje wybór poziomu lub powrót do menu. Blokuje kliknięcia w nieodblokowane poziomy.

Parametry

event	Zdarzenie SFML.

Implementuje State.

Oto graf wywołań dla tej funkcji:

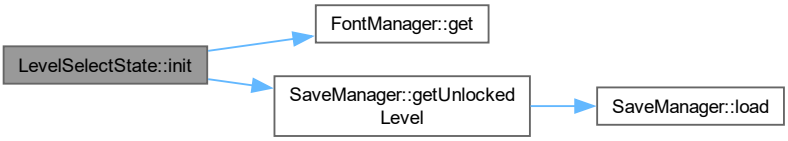
4.11.3.3 init()

```
void LevelSelectState::init ( )  [override], [virtual]
```

Tworzy listę przycisków na podstawie liczby poziomów i postępu gracza.

Implementuje State.

Oto graf wywołań dla tej funkcji:



4.11.3.4 update()

```
void LevelSelectState::update (
            float dt )  [override], [virtual]
```

Aktualizuje logikę podświetlania dostępnych poziomów.

Parametry

dt	Czas klatki.

Implementuje State.

Oto graf wywołań dla tej funkcji:

4.11.4 Dokumentacja atrybutów składowych

4.11.4.1 m_alpha

```
float LevelSelectState::m_alpha  [private]
```

4.11.4.2 m_backButton

```
sf::Text LevelSelectState::m_backButton  [private]
```

Przycisk powrotu.

4.11.4.3 m_data

```
GameDataRef LevelSelectState::m_data [private]
```

4.11.4.4 m_fadeRect

```
sf::RectangleShape LevelSelectState::m_fadeRect [private]
```

4.11.4.5 m_isFadingIn

```
bool LevelSelectState::m_isFadingIn [private]
```

4.11.4.6 m_isTransitioning

```
bool LevelSelectState::m_isTransitioning [private]
```

4.11.4.7 m_levelButtons

```
std::vector<sf::Text> LevelSelectState::m_levelButtons [private]
```

Dynamiczna lista przycisków poziomów.

4.11.4.8 m_nextLevelToLoad

```
int LevelSelectState::m_nextLevelToLoad [private]
```

Numer wybranego poziomu (-1 oznacza powrót).

4.11.4.9 m_title

```
sf::Text LevelSelectState::m_title [private]
```

Nagłówek "SELECT LEVEL".

Dokumentacja dla tej klasy została wygenerowana z plików:

- LevelSelectState.h
- LevelSelectState.cpp

4.12 Dokumentacja klasy MainMenuState

Stan początkowy gry (Menu Główne).

```
#include <MainMenuState.h>
```

Diagram dziedziczenia dla MainMenuState

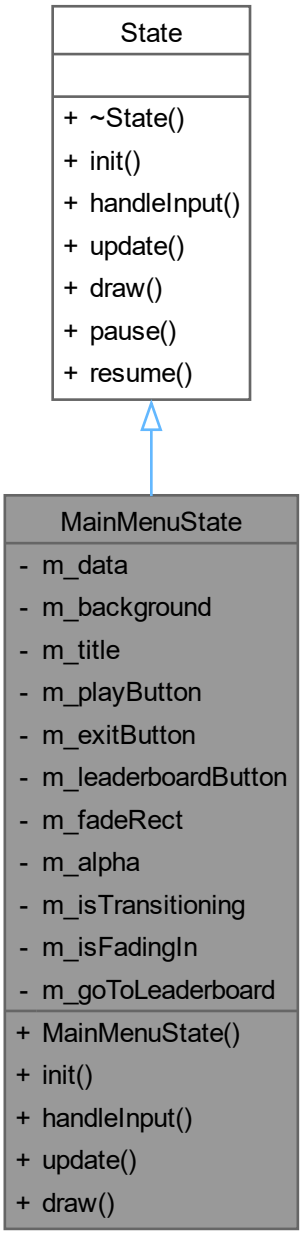
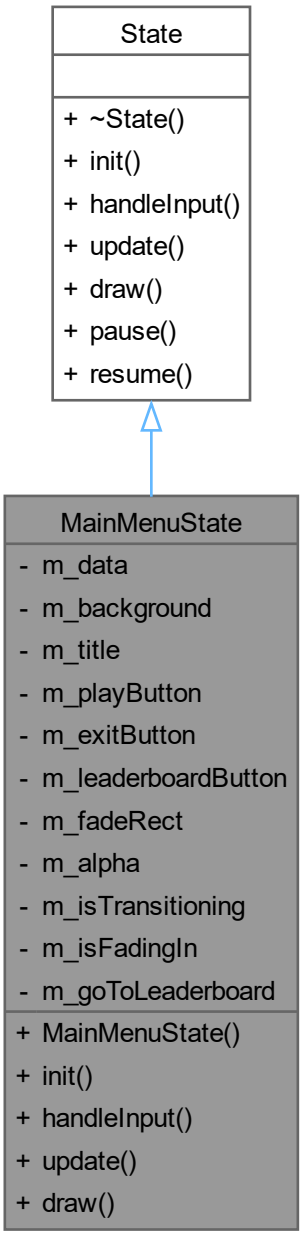


Diagram współpracy dla MainMenuState:



Metody publiczne

- MainMenuState (GameDataRef data)

  Konstruktor stanu Menu.
- void init () override

  Inicjalizuje widok menu, ładuje czcionki i ustawia pozycje przycisków. Inicjalizuje również system zapisu (SaveManager).
- void handleInput (sf::Event &event) override

Obsługuje kliknięcia w przyciski menu.

- void update (float dt) override

  Aktualizuje efekty wizualne (podświetlenie przycisków, animacje wejścia/wyjścia).
- void draw (float dt) override

  Rysuje interfejs menu na ekranie.

Metody publiczne dziedziczone z State

- virtual ~State ()

  Wirtualny destruktor.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

Atrybuty prywatne

- GameDataRef m_data

  Dane gry.
- sf::Sprite m_background

  Tło menu (opcjonalne, w kodzie .cpp używany jest jednolity kolor).
- sf::Text m_title

  Tytuł gry "MINI GOLF".
- sf::Text m_playButton

  Przycisk "PLAY".
- sf::Text m_exitButton

  Przycisk "EXIT".
- sf::Text m_leaderboardButton

  Przycisk "LEADERBOARD".
- sf::RectangleShape m_fadeRect

  Prostokąt przyciemniający ekran.
- float m_alpha

  Kanał alpha dla efektu fade.
- bool m_isTransitioning

  Czy trwa animacja wyjścia do innego stanu.
- bool m_isFadingIn

  Czy trwa animacja wejścia (rozjaśnianie).
- bool m_goToLeaderboard

  Flaga decydująca o celu przejścia (Leaderboard vs LevelSelect).

4.12.1 Opis szczegółowy

Stan początkowy gry (Menu Główne).

Klasa odpowiada za wyświetlenie głównego interfejsu użytkownika po uruchomieniu aplikacji. Umożliwia nawigację do:

- Wyboru poziomów (Play).
- Tabeli wyników (Leaderboard).
- Wyjścia z aplikacji (Exit).

4.12.2 Dokumentacja konstruktora i destruktora

4.12.2.1 MainMenuState()

```
MainMenuState::MainMenuState (
            GameDataRef data )
```

Konstruktor stanu Menu.

Parametry

data	Współdzielone dane gry.

4.12.3 Dokumentacja funkcji składowych

4.12.3.1 draw()

```
void MainMenuState::draw (
            float dt )  [override], [virtual]
```

Rysuje interfejs menu na ekranie.

Parametry

dt	Interpolacja (nieużywana).

Implementuje State.

4.12.3.2 handleInput()

```
void MainMenuState::handleInput (
            sf::Event & event )  [override], [virtual]
```

Obsługuje kliknięcia w przyciski menu.

Parametry

event	Zdarzenie SFML.

Implementuje State.

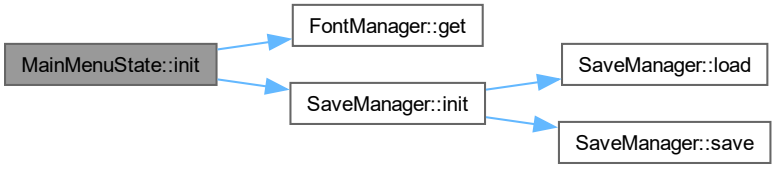
4.12.3.3 init()

```
void MainMenuState::init ( )  [override], [virtual]
```

Inicjalizuje widok menu, ładuje czcionki i ustawia pozycje przycisków. Inicjalizuje również system zapisu (SaveManager).

Implementuje State.

Oto graf wywołań dla tej funkcji:



4.12.3.4 update()

```
void MainMenuState::update (
            float dt )  [override], [virtual]
```

Aktualizuje efekty wizualne (podświetlenie przycisków, animacje wejścia/wyjścia).

Parametry

dt	Czas klatki (Delta Time).

Implementuje State.

4.12.4 Dokumentacja atrybutów składowych

4.12.4.1 m_alpha

```
float MainMenuState::m_alpha  [private]
```

Kanał alpha dla efektu fade.

4.12.4.2 m_background

```
sf::Sprite MainMenuState::m_background  [private]
```

Tło menu (opcjonalne, w kodzie .cpp używany jest jednolity kolor).

4.12.4.3 m_data

```
GameDataRef MainMenuState::m_data  [private]
```

Dane gry.

4.12.4.4 m_exitButton

```
sf::Text MainMenuState::m_exitButton [private]
```

Przycisk "EXIT".

4.12.4.5 m_fadeRect

```
sf::RectangleShape MainMenuState::m_fadeRect [private]
```

Prostokąt przyciemniający ekran.

4.12.4.6 m_goToLeaderboard

```
bool MainMenuState::m_goToLeaderboard [private]
```

Flaga decydująca o celu przejścia (Leaderboard vs LevelSelect).

4.12.4.7 m_isFadingIn

```
bool MainMenuState::m_isFadingIn [private]
```

Czy trwa animacja wejścia (rozjaśnianie).

4.12.4.8 m_isTransitioning

```
bool MainMenuState::m_isTransitioning [private]
```

Czy trwa animacja wyjścia do innego stanu.

4.12.4.9 m_leaderboardButton

```
sf::Text MainMenuState::m_leaderboardButton [private]
```

Przycisk "LEADERBOARD".

4.12.4.10 m_playButton

```
sf::Text MainMenuState::m_playButton [private]
```

Przycisk "PLAY".

4.12.4.11 m_title

```
sf::Text MainMenuState::m_title [private]
```

Tytuł gry "MINI GOLF".

Dokumentacja dla tej klasy została wygenerowana z plików:

- MainMenuState.h
- MainMenuState.cpp

4.13 Dokumentacja klasy PauseMenuState

Stan pauzy nakładany na stan gry.

```
#include <PauseMenuState.h>
```

Diagram dziedziczenia dla PauseMenuState

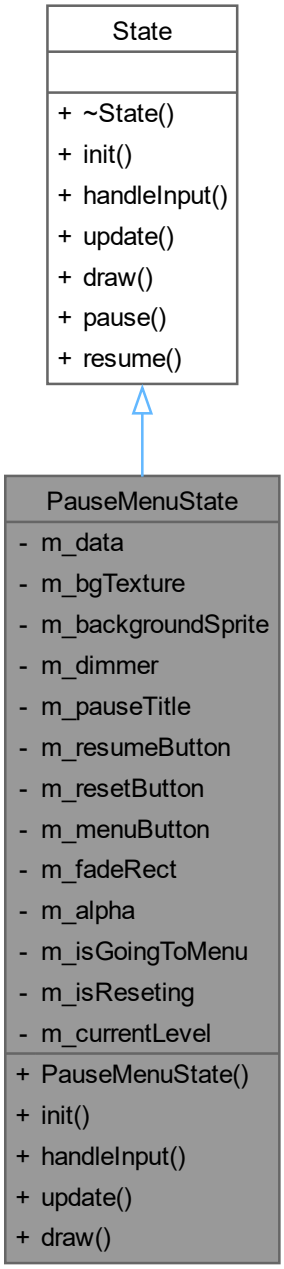
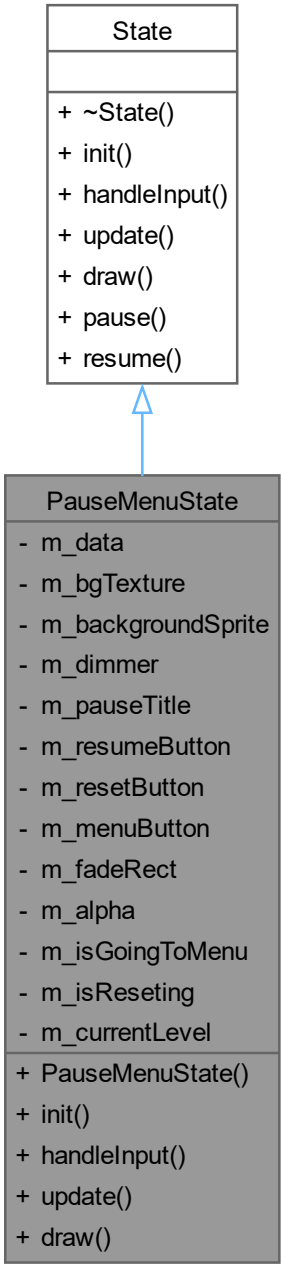


Diagram współpracy dla PauseMenuState:



Metody publiczne

- PauseMenuState (GameDataRef data, sf::Sprite bgSprite, std::shared_ptr< sf::Texture > bgTexture, int levelNumber)

  Konstruktor stanu pauzy.
- void init () override

  Inicjalizuje przyciski Resume, Reset, Menu.

- void handleInput (sf::Event &event) override

  Obsługuje kliknięcia w menu pauzy oraz klawisz ESC (wznowienie).
- void update (float dt) override

  Aktualizuje efekty wizualne przycisków.
- void draw (float dt) override

  Rysuje zamrożone tło gry, przyciemnienie i menu pauzy.

Metody publiczne dziedziczone z State

- virtual ~State ()

  Wirtualny destruktor.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

Atrybuty prywatne

- GameDataRef m_data
- std::shared_ptr< sf::Texture > m_bgTexture

  Przechowywana tekstura tła.
- sf::Sprite m_backgroundSprite

  Sprite tła (zamrożona gra).
- sf::RectangleShape m_dimmer

  Półprzezroczysty prostokąt przyciemniający.
- sf::Text m_pauseTitle
- sf::Text m_resumeButton

  Wznawia grę (dejmuje stan ze stosu).
- sf::Text m_resetButton

  Resetuje poziom (podmienia stan gry).
- sf::Text m_menuButton

  Wraca do menu głównego.
- sf::RectangleShape m_fadeRect
- float m_alpha
- bool m_isGoingToMenu
- bool m_isReseting
- int m_currentLevel

4.13.1 Opis szczegółowy

Stan pauzy nakładany na stan gry.

Jest to stan typu "overlay" (nakładka). Nie usuwa stanu gry ze stosu, lecz rysuje się "na nim", wykorzystując zrzut ekranu jako tło.

4.13.2 Dokumentacja konstruktora i destruktora

4.13.2.1 PauseMenuState()

```
PauseMenuState::PauseMenuState (
            GameDataRef data,
            sf::Sprite bgSprite,
            std::shared_ptr< sf::Texture > bgTexture,
            int levelNumber )
```

Konstruktor stanu pauzy.

Parametry

data	Dane gry.
bgSprite	Sprite ze zrzutem ekranu gry w momencie pauzy.
bgTexture	Wskaźnik do tekstury zrzutu (shared_ptr zapewnia, że tekstura nie zostanie usunięta).
levelNumber	Numer aktualnego poziomu (potrzebny do opcji Restart).

4.13.3 Dokumentacja funkcji składowych

4.13.3.1 draw()

```
void PauseMenuState::draw (
            float dt )  [override], [virtual]
```

Rysuje zamrożone tło gry, przyciemnienie i menu pauzy.

Parametry

dt	Interpolacja.

Implementuje State.

4.13.3.2 handleInput()

```
void PauseMenuState::handleInput (
            sf::Event & event )  [override], [virtual]
```

Obsługuje kliknięcia w menu pauzy oraz klawisz ESC (wznowienie).

Parametry

event	Zdarzenie SFML.

Implementuje State.

4.13.3.3 init()

```
void PauseMenuState::init ( )  [override], [virtual]
```

Inicjalizuje przyciski Resume, Reset, Menu.

Implementuje State.

Oto graf wywołań dla tej funkcji:

4.13.3.4 update()

```
void PauseMenuState::update (
            float dt )  [override], [virtual]
```

Aktualizuje efekty wizualne przycisków.

Parametry

dt	Czas klatki.

Implementuje State.

4.13.4 Dokumentacja atrybutów składowych

4.13.4.1 m_alpha

```
float PauseMenuState::m_alpha  [private]
```

4.13.4.2 m_backgroundSprite

```
sf::Sprite PauseMenuState::m_backgroundSprite  [private]
```

Sprite tła (zamrożona gra).

4.13.4.3 m_bgTexture

```
std::shared_ptr<sf::Texture> PauseMenuState::m_bgTexture  [private]
```

Przechowywana tekstura tła.

4.13.4.4 m_currentLevel

```
int PauseMenuState::m_currentLevel  [private]
```

4.13.4.5 m_data

```
GameDataRef PauseMenuState::m_data [private]
```

4.13.4.6 m_dimmer

```
sf::RectangleShape PauseMenuState::m_dimmer [private]
```

Półprzezroczysty prostokąt przyciemniający.

4.13.4.7 m_fadeRect

```
sf::RectangleShape PauseMenuState::m_fadeRect [private]
```

4.13.4.8 m_isGoingToMenu

```
bool PauseMenuState::m_isGoingToMenu [private]
```

4.13.4.9 m_isReseting

```
bool PauseMenuState::m_isReseting [private]
```

4.13.4.10 m_menuButton

```
sf::Text PauseMenuState::m_menuButton [private]
```

Wraca do menu głównego.

4.13.4.11 m_pauseTitle

```
sf::Text PauseMenuState::m_pauseTitle [private]
```

4.13.4.12 m_resetButton

```
sf::Text PauseMenuState::m_resetButton [private]
```

Resetuje poziom (podmienia stan gry).

4.13.4.13 m_resumeButton

```
sf::Text PauseMenuState::m_resumeButton [private]
```

Wznawia grę (dejmuje stan ze stosu).

Dokumentacja dla tej klasy została wygenerowana z plików:

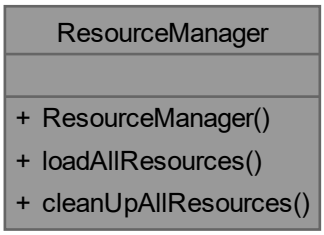
- PauseMenuState.h
- PauseMenuState.cpp

4.14 Dokumentacja klasy ResourceManager

Fasada zarządzająca ładowaniem wszystkich zasobów aplikacji.

```
#include <ResourceManager.h>
```

Diagram współpracy dla ResourceManager:



Metody publiczne

- ResourceManager ()=delete

Statyczne metody publiczne

- static void loadAllResources ()

  Ładuje wszystkie tekstury, czcionki i dźwięki zdefiniowane w DEFINITIONS.h. Powinna być wywołana na początku funkcji main().
- static void cleanUpAllResources ()

  Czyści pamięć wszystkich menedżerów zasobów.

4.14.1 Opis szczegółowy

Fasada zarządzająca ładowaniem wszystkich zasobów aplikacji.

Klasa statyczna, która agreguje wywołania do poszczególnych menedżerów (Texture, Font, Sound) w celu jednorazowego załadowania lub zwolnienia wszystkich assetów gry.

4.14.2 Dokumentacja konstruktora i destruktora

4.14.2.1 ResourceManager()

```
ResourceManager::ResourceManager ( )  [delete]
```

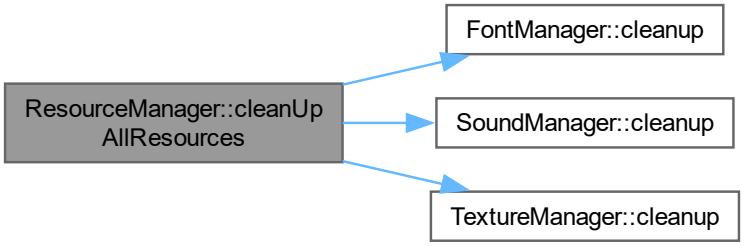
4.14.3 Dokumentacja funkcji składowych

4.14.3.1 cleanUpAllResources()

```
void ResourceManager::cleanUpAllResources ( ) [static]
```

Czyści pamięć wszystkich menedżerów zasobów.

Oto graf wywołań dla tej funkcji:



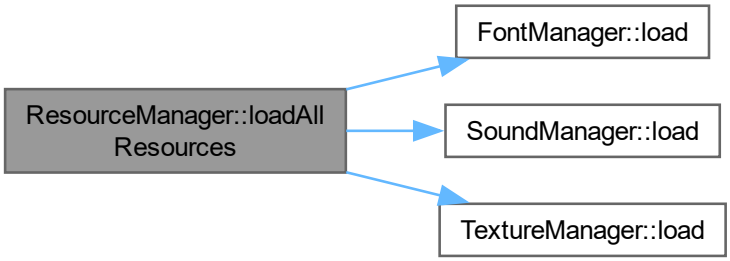
Oto graf wywoływań tej funkcji:

4.14.3.2 loadAllResources()

```
void ResourceManager::loadAllResources ( ) [static]
```

Ładuje wszystkie tekstury, czcionki i dźwięki zdefiniowane w DEFINITIONS.h. Powinna być wywołana na początku funkcji main().

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

Dokumentacja dla tej klasy została wygenerowana z plików:

- ResourceManager.h
- ResourceManager.cpp

4.15 Dokumentacja klasy SaveManager

Klasa statyczna obsługująca trwałość danych gry (system zapisu/odczytu).

```
#include <SaveManager.h>
```

Diagram współpracy dla SaveManager:

Statyczne metody publiczne

- static void init ()

  Inicjalizuje system zapisu. Sprawdza, czy plik zapisu istnieje; jeśli nie, tworzy domyślny.
- static int getUnlockedLevel ()

  Pobiera numer najwyższego odblokowanego poziomu przez gracza.
- static int getBestScore (int level)

  Pobiera najlepszy wynik (najniższa liczba uderzeń) dla danego poziomu.
- static void submitScore (int level, int score)

  Przesyła nowy wynik ukończenia poziomu. Aktualizuje najlepszy wynik, jeśli nowy jest lepszy, oraz odblokowuje kolejny poziom. Automatycznie zapisuje zmiany na dysku.

Statyczne metody prywatne

- static void save ()

  Zapisuje bieżące dane do pliku.
- static void load ()

  Wczytuje dane z pliku do pamięci.

Statyczne atrybuty prywatne

- static std::string fileName = "savegame.txt"

  Nazwa pliku zapisu.
- static int unlockedLevel = 1

  Indeks aktualnie odblokowanego poziomu.
- static std::vector< int > bestScores = {0, 0, 0, 0, 0}

  Lista najlepszych wyników dla każdego poziomu.

4.15.1 Opis szczegółowy

Klasa statyczna obsługująca trwałość danych gry (system zapisu/odczytu).

Zarządza odczytem i zapisem postępów gracza (odblokowane poziomy, najlepsze wyniki) do lokalnego pliku tekstowego.

4.15.2 Dokumentacja funkcji składowych

4.15.2.1 getBestScore()

```
int SaveManager::getBestScore (
            int level )  [static]
```

Pobiera najlepszy wynik (najniższa liczba uderzeń) dla danego poziomu.

Parametry

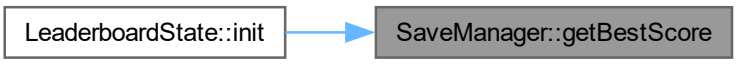
level	Numer poziomu do sprawdzenia.

Zwraca

Najlepszy wynik lub 0, jeśli brak zapisu.

Oto graf wywołań dla tej funkcji:

Oto graf wywoływań tej funkcji:



4.15.2.2 getUnlockedLevel()

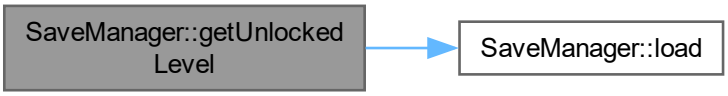
```
int SaveManager::getUnlockedLevel ( )  [static]
```

Pobiera numer najwyższego odblokowanego poziomu przez gracza.

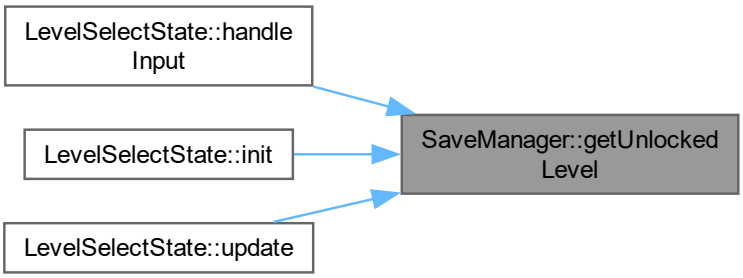
Zwraca

Numer poziomu (1-5).

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

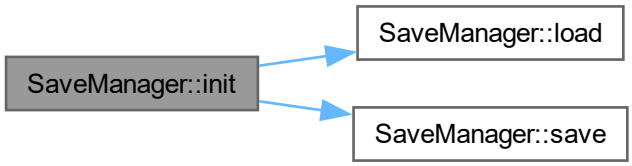


4.15.2.3 init()

```
void SaveManager::init ( )  [static]
```

Inicjalizuje system zapisu. Sprawdza, czy plik zapisu istnieje; jeśli nie, tworzy domyślny.

Oto graf wywołań dla tej funkcji:



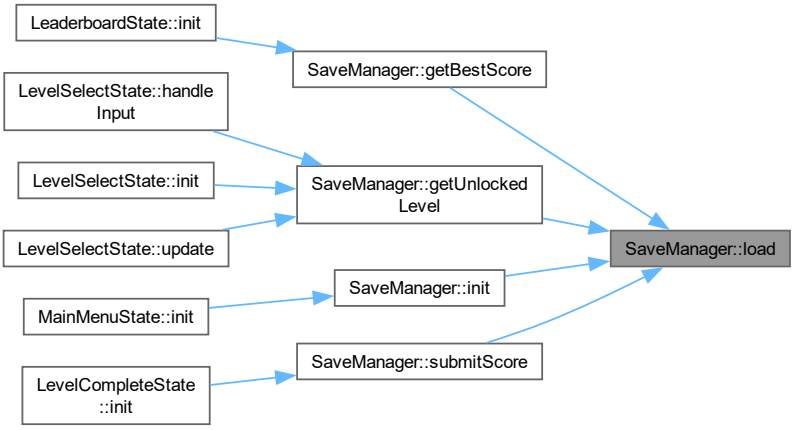
Oto graf wywoływań tej funkcji:

4.15.2.4 load()

```
void SaveManager::load ( )  [static], [private]
```

Wczytuje dane z pliku do pamięci.

Oto graf wywoływań tej funkcji:

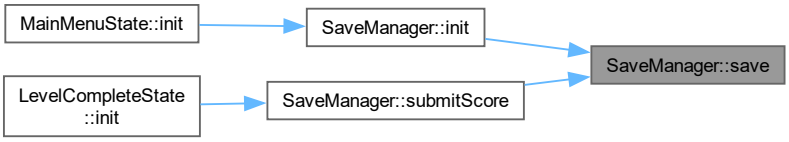


4.15.2.5 save()

```
void SaveManager::save ( ) [static], [private]
```

Zapisuje bieżące dane do pliku.

Oto graf wywoływań tej funkcji:



4.15.2.6 submitScore()

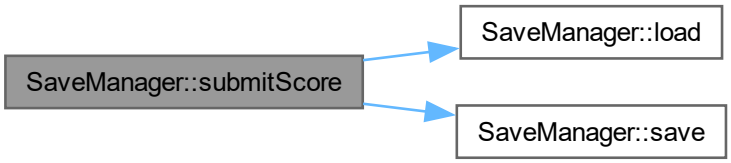
```
void SaveManager::submitScore (
            int level,
            int score ) [static]
```

Przesyła nowy wynik ukończenia poziomu. Aktualizuje najlepszy wynik, jeśli nowy jest lepszy, oraz odblokowuje kolejny poziom. Automatycznie zapisuje zmiany na dysku.

Parametry

level	Numer ukończonego poziomu.
score	Liczba wykonanych uderzeń.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

4.15.3 Dokumentacja atrybutów składowych

4.15.3.1 bestScores

```
std::vector< int > SaveManager::bestScores = {0, 0, 0, 0, 0}  [static], [private]
```

Lista najlepszych wyników dla każdego poziomu.

4.15.3.2 fileName

```
std::string SaveManager::fileName = "savegame.txt"  [static], [private]
```

Nazwa pliku zapisu.

4.15.3.3 unlockedLevel

```
int SaveManager::unlockedLevel = 1  [static], [private]
```

Indeks aktualnie odblokowanego poziomu.

Dokumentacja dla tej klasy została wygenerowana z plików:

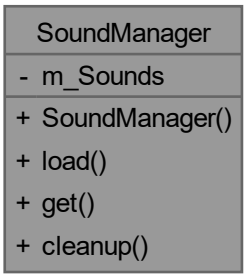
- SaveManager.h
- SaveManager.cpp

4.16 Dokumentacja klasy SoundManager

Statyczny menedżer zasobów dla buforów dźwiękowych.

```
#include <SoundManager.h>
```

Diagram współpracy dla SoundManager:



Metody publiczne

- SoundManager ()=delete

Statyczne metody publiczne

- static void load (const std::string &key, const std::string &path)

  Ładuje plik dźwiękowy do pamięci.
- static const sf::SoundBuffer & get (const std::string &key)

  Pobiera załadowany bufor dźwiękowy.
- static void cleanup ()

  Czyści wszystkie bufory dźwiękowe.

Statyczne atrybuty prywatne

- static std::map< std::string, sf::SoundBuffer > m_Sounds

4.16.1 Opis szczegółowy

Statyczny menedżer zasobów dla buforów dźwiękowych.

Zarządza obiektami sf::SoundBuffer, aby zapewnić jednokrotne ładowanie danych audio do pamięci.

4.16.2 Dokumentacja konstruktora i destruktora

4.16.2.1 SoundManager()

```
SoundManager::SoundManager ( ) [delete]
```

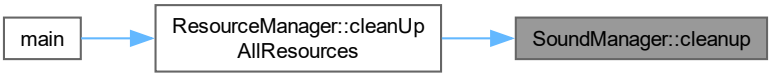
4.16.3 Dokumentacja funkcji składowych

4.16.3.1 cleanup()

```
void SoundManager::cleanup ( ) [static]
```

Czyści wszystkie bufory dźwiękowe.

Oto graf wywoływań tej funkcji:



4.16.3.2 get()

```
const sf::SoundBuffer & SoundManager::get (
            const std::string & key ) [static]
```

Pobiera załadowany bufor dźwiękowy.

Parametry

key	Unikalny identyfikator.

Zwraca

Stała referencja do sf::SoundBuffer.

Oto graf wywoływań tej funkcji:

4.16.3.3 load()

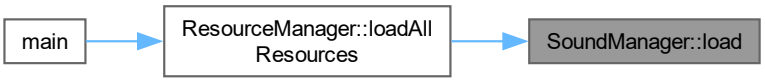
```
void SoundManager::load (
            const std::string & key,
            const std::string & path )  [static]
```

Ładuje plik dźwiękowy do pamięci.

Parametry

key	Unikalny identyfikator dźwięku.
path	Ścieżka do pliku audio (.wav, .ogg itp.).

Oto graf wywoływań tej funkcji:



4.16.4 Dokumentacja atrybutów składowych

4.16.4.1 m_Sounds

```
std::map< std::string, sf::SoundBuffer > SoundManager::m_Sounds  [static], [private]
```

Dokumentacja dla tej klasy została wygenerowana z plików:

- SoundManager.h
- SoundManager.cpp

4.17 Dokumentacja klasy State

Abstrakcyjna klasa bazowa reprezentująca pojedynczy stan gry (np. Menu, Gra, Pauza).

```
#include <State.h>
```

Diagram dziedziczenia dla State

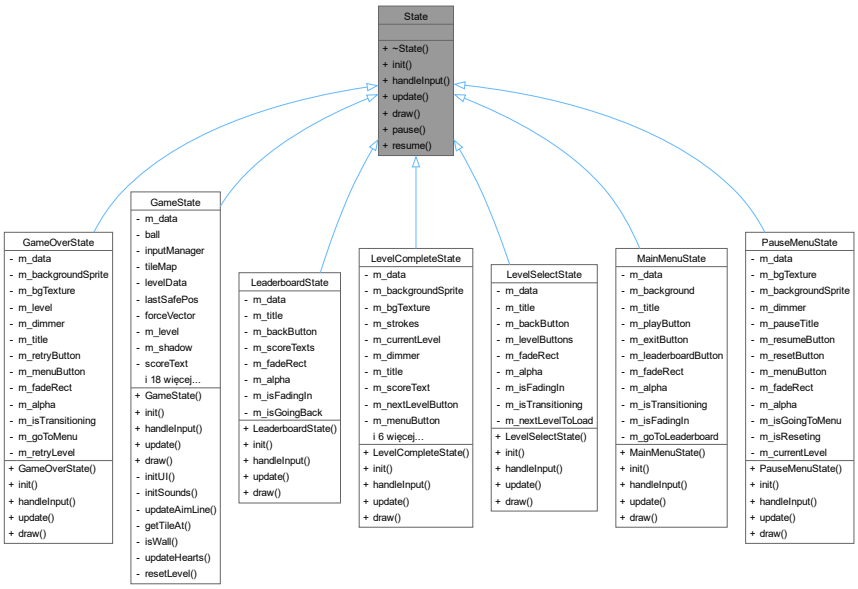


Diagram współpracy dla State:

Metody publiczne

- virtual ~State ()

  Wirtualny destruktor.
- virtual void init ()=0

  Inicjalizuje zasoby i zmienne stanu. Wywoływana raz, gdy stan jest dodawany do StateManagera.
- virtual void handleInput (sf::Event &event)=0

  Obsługuje wejście od użytkownika (klawiatura, mysz, zdarzenia okna).

- virtual void update (float dt)=0

  Aktualizuje logikę stanu (fizyka, animacje, reguły gry).
- virtual void draw (float dt)=0

  Renderuje grafikę stanu na ekranie.
- virtual void pause ()

  Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).
- virtual void resume ()

  Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

4.17.1 Opis szczegółowy

Abstrakcyjna klasa bazowa reprezentująca pojedynczy stan gry (np. Menu, Gra, Pauza).

Klasa ta definiuje interfejs, który muszą zaimplementować wszystkie konkretne stany gry. Wykorzystuje wzorzec projektowy "Stan" (State Pattern), pozwalając StateManagerowi na jednolite zarządzanie różnymi etapami rozgrywki.

4.17.2 Dokumentacja konstruktora i destruktora

4.17.2.1 ~State()

```
virtual State::~State ( )  [inline], [virtual]
```

Wirtualny destruktor.

4.17.3 Dokumentacja funkcji składowych

4.17.3.1 draw()

```
virtual void State::draw (
            float dt )  [pure virtual]
```

Renderuje grafikę stanu na ekranie.

Parametry

dt	Współczynnik interpolacji (używany do płynnego renderowania między krokami fizyki).

Implementowany w GameOverState, GameState, LeaderboardState, LevelCompleteState, LevelSelectState, MainMenuState i PauseMenuState.

4.17.3.2 handleInput()

```
virtual void State::handleInput (
            sf::Event & event )  [pure virtual]
```

Obsługuje wejście od użytkownika (klawiatura, mysz, zdarzenia okna).

Parametry

event	Zdarzenie SFML do przetworzenia.

Implementowany w GameOverState, GameState, LeaderboardState, LevelCompleteState, LevelSelectState, MainMenuState i PauseMenuState.

4.17.3.3 init()

```
virtual void State::init ( ) [pure virtual]
```

Inicjalizuje zasoby i zmienne stanu. Wywoływana raz, gdy stan jest dodawany do StateManagera.

Implementowany w GameOverState, GameState, LeaderboardState, LevelCompleteState, LevelSelectState, MainMenuState i PauseMenuState.

4.17.3.4 pause()

```
virtual void State::pause ( ) [inline], [virtual]
```

Wywoływana, gdy stan jest pauzowany (np. inny stan jest nakładany na wierzch).

4.17.3.5 resume()

```
virtual void State::resume ( ) [inline], [virtual]
```

Wywoływana, gdy stan jest wznawiany (np. stan powyżej został zdjęty ze stosu).

4.17.3.6 update()

```
virtual void State::update (
            float dt ) [pure virtual]
```

Aktualizuje logikę stanu (fizyka, animacje, reguły gry).

Parametry

dt	Czas, który upłynął od ostatniej klatki (Delta Time) w sekundach.

Implementowany w GameOverState, GameState, LeaderboardState, LevelCompleteState, LevelSelectState, MainMenuState i PauseMenuState.

Dokumentacja dla tej klasy została wygenerowana z pliku:

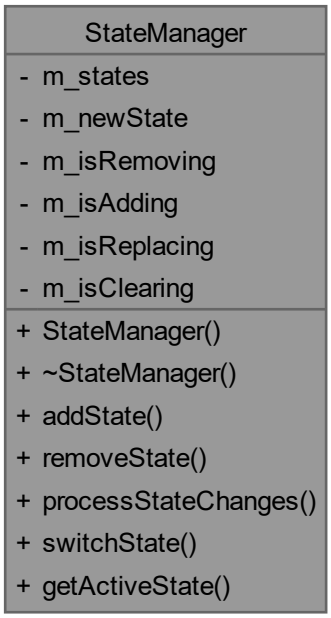
- State.h

4.18 Dokumentacja klasy StateManager

Zarządza stosem stanów gry.

```
#include <StateManager.h>
```

Diagram współpracy dla StateManager:



Metody publiczne

- StateManager ()
- ~StateManager ()
- void addState (StateRef newState, bool isReplacing=true)

  Planuje dodanie nowego stanu.
- void removeState ()

  Planuje usunięcie obecnego górnego stanu.
- void processStateChanges ()

  Przetwarza oczekujące zmiany stanów (dodawanie, usuwanie). Powinna być wywoływana na początku głównej pętli gry.
- void switchState (StateRef newState)

  Czyści cały stos i ustawia nowy stan (np. powrót do Menu Głównego).
- StateRef & getActiveState ()

  Zwraca referencję do aktywnego (górnego) stanu.

Atrybuty prywatne

- std::stack< StateRef > m_states

  Stos stanów gry.
- StateRef m_newState

  Tymczasowe przechowanie nowego stanu do dodania.
- bool m_isRemoving

  Flaga oznaczająca konieczność zdjęcia stanu ze stosu.
- bool m_isAdding

  Flaga oznaczająca konieczność dodania stanu na stos.
- bool m_isReplacing

  Flaga oznaczająca, czy dodanie ma zastąpić obecny stan.
- bool m_isClearing

  Flaga oznaczająca, czy stos ma zostać całkowicie wyczyszczony.

4.18.1 Opis szczegółowy

Zarządza stosem stanów gry.

Klasa odpowiedzialna za dodawanie, usuwanie i przełączanie między różnymi stanami gry. Obsługuje przejścia między stanami w bezpiecznym momencie (na początku pętli gry), aby uniknąć błędów pamięci podczas aktualizacji.

4.18.2 Dokumentacja konstruktora i destruktora

4.18.2.1 StateManager()

```
StateManager::StateManager ( ) [inline]
```

4.18.2.2 ~StateManager()

```
StateManager::~StateManager ( ) [inline]
```

4.18.3 Dokumentacja funkcji składowych

4.18.3.1 addState()

```
void StateManager::addState (
            StateRef newState,
            bool isReplacing = true )
```

Planuje dodanie nowego stanu.

Parametry

newState	Unikalny wskaźnik do nowego stanu.
isReplacing	Jeśli true, obecny górny stan zostanie usunięty przed dodaniem nowego.

4.18.3.2 getActiveState()

```
StateRef & StateManager::getActiveState ( )
```

Zwraca referencję do aktywnego (górnego) stanu.

Zwraca

Referencja do unikalnego wskaźnika obecnego stanu.

4.18.3.3 processStateChanges()

```
void StateManager::processStateChanges ( )
```

Przetwarza oczekujące zmiany stanów (dodawanie, usuwanie). Powinna być wywoływana na początku głównej pętli gry.

4.18.3.4 removeState()

```
void StateManager::removeState ( )
```

Planuje usunięcie obecnego górnego stanu.

4.18.3.5 switchState()

```
void StateManager::switchState (
            StateRef newState )
```

Czyści cały stos i ustawia nowy stan (np. powrót do Menu Głównego).

Parametry

newState	Nowy stan bazowy.

4.18.4 Dokumentacja atrybutów składowych

4.18.4.1 m_isAdding

```
bool StateManager::m_isAdding [private]
```

Flaga oznaczająca konieczność dodania stanu na stos.

4.18.4.2 m_isClearing

```
bool StateManager::m_isClearing [private]
```

Flaga oznaczająca, czy stos ma zostać całkowicie wyczyszczony.

4.18.4.3 m_isRemoving

```
bool StateManager::m_isRemoving [private]
```

Flaga oznaczająca konieczność zdjęcia stanu ze stosu.

4.18.4.4 m_isReplacing

```
bool StateManager::m_isReplacing [private]
```

Flaga oznaczająca, czy dodanie ma zastąpić obecny stan.

4.18.4.5 m_newState

```
StateRef StateManager::m_newState [private]
```

Tymczasowe przechowanie nowego stanu do dodania.

4.18.4.6 m_states

```
std::stack<StateRef> StateManager::m_states [private]
```

Stos stanów gry.

Dokumentacja dla tej klasy została wygenerowana z plików:

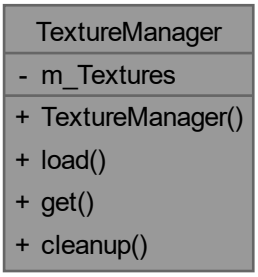
- StateManager.h
- StateManager.cpp

4.19 Dokumentacja klasy TextureManager

Statyczny menedżer zasobów dla tekstur SFML.

```
#include <TextureManager.h>
```

Diagram współpracy dla TextureManager:



Metody publiczne

- TextureManager ()=delete

Statyczne metody publiczne

- static void load (const std::string &key, const std::string &path)

  Ładuje teksturę z pliku i zapisuje ją pod kluczem tekstowym.
- static const sf::Texture & get (const std::string &key)

  Pobiera referencję do przechowywanej tekstury.
- static void cleanup ()

  Czyści wszystkie tekstury z pamięci.

Statyczne atrybuty prywatne

- static std::map< std::string, sf::Texture > m_Textures

  Kontener przechowujący tekstury.

4.19.1 Opis szczegółowy

Statyczny menedżer zasobów dla tekstur SFML.

Implementuje wzorzec Pyłek (Flyweight) poprzez przechowywanie załadowanych tekstur w mapie. Zapobiega to wielokrotnemu ładowaniu tego samego zasobu z dysku, oszczędzając pamięć.

4.19.2 Dokumentacja konstruktora i destruktora

4.19.2.1 TextureManager()

```
TextureManager::TextureManager ( )  [delete]
```

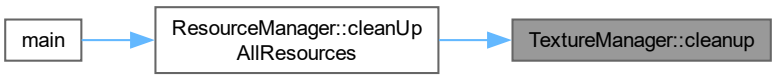
4.19.3 Dokumentacja funkcji składowych

4.19.3.1 cleanup()

```
void TextureManager::cleanup ( )  [static]
```

Czyści wszystkie tekstury z pamięci.

Oto graf wywoływań tej funkcji:



4.19.3.2 get()

```
const sf::Texture & TextureManager::get (
            const std::string & key )  [static]
```

Pobiera referencję do przechowywanej tekstury.

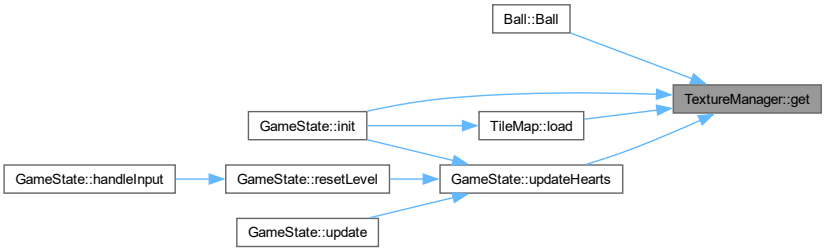
Parametry

key	Identyfikator tekstury do pobrania.

Zwraca

Stała referencja do sf::Texture.

Oto graf wywoływań tej funkcji:



4.19.3.3 load()

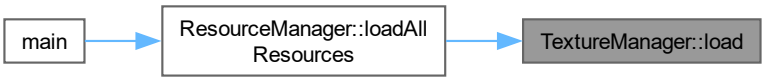
```
void TextureManager::load (
            const std::string & key,
            const std::string & path )  [static]
```

Ładuje teksturę z pliku i zapisuje ją pod kluczem tekstowym.

Parametry

key	Unikalny identyfikator tekstury.
path	Ścieżka do pliku graficznego.

Oto graf wywoływań tej funkcji:



4.19.4 Dokumentacja atrybutów składowych

4.19.4.1 m_Textures

```
std::map< std::string, sf::Texture > TextureManager::m_Textures  [static], [private]
```

Kontener przechowujący tekstury.

Dokumentacja dla tej klasy została wygenerowana z plików:

- TextureManager.h
- TextureManager.cpp

4.20 Dokumentacja klasy TileMap

Klasa odpowiedzialna za renderowanie poziomu gry przy użyciu Vertex Array.

```
#include <TileMap.h>
```

Diagram dziedziczenia dla TileMap

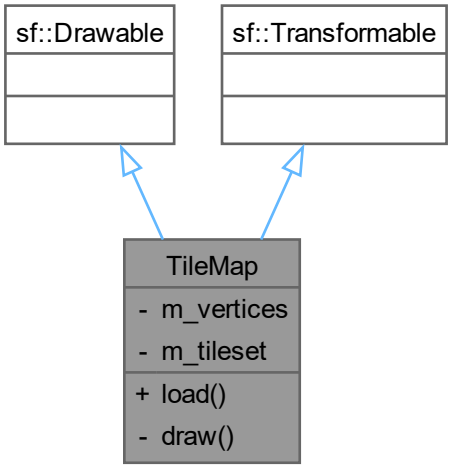
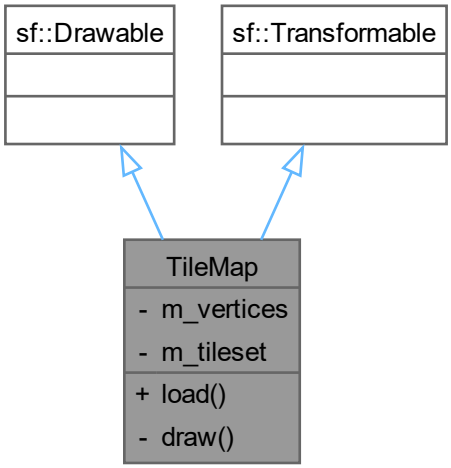


Diagram współpracy dla TileMap:



Metody publiczne

- bool load (const std::string &tileset, sf::Vector2u tileSize, std::vector< int >tiles, unsigned int width, unsigned int height)

  Wczytuje dane mapy i generuje tablicę wierzchołków.

Metody prywatne

- virtual void draw (sf::RenderTarget &target, sf::RenderStates states) const

  Nadpisana metoda draw z sf::Drawable. Obsługuje renderowanie tablicy wierzchołków z przypisaną teksturą.

Atrybuty prywatne

- sf::VertexArray m_vertices

  Kolekcja wierzchołków reprezentująca kafelki (Quads).
- const sf::Texture * m_tileset

  Wskaźnik do tekstury źródłowej (tileset).

4.20.1 Opis szczegółowy

Klasa odpowiedzialna za renderowanie poziomu gry przy użyciu Vertex Array.

Klasa ta optymalizuje wyświetlanie mapy poprzez rysowanie całego poziomu jako jednego obiektu (jedno wywołanie draw) przy użyciu sf::VertexArray. Jest to znacznie wydajniejsze niż rysowanie oddzielnych sprite'ów dla każdego kafelka.

4.20.2 Dokumentacja funkcji składowych

4.20.2.1 draw()

```
void TileMap::draw (
            sf::RenderTarget & target,
            sf::RenderStates states ) const  [private], [virtual]
```

Nadpisana metoda draw z sf::Drawable. Obsługuje renderowanie tablicy wierzchołków z przypisaną teksturą.

Parametry

target	Cel renderowania (okno).
states	Stany renderowania (transformacja, tekstura).

4.20.2.2 load()

```
bool TileMap::load (
            const std::string & tileset,
            sf::Vector2u tileSize,
            std::vector< int > tiles,
            unsigned int width,
            unsigned int height )
```

Wczytuje dane mapy i generuje tablicę wierzchołków.

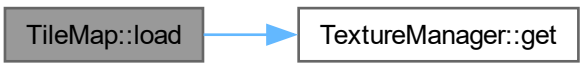
Parametry

tileset	Ścieżka do pliku tekstury (tileset) lub klucz w ResourceManagerze.
tileSize	Wymiary pojedynczego kafelka (np. 32x32).
tiles	Wektor zawierający identyfikatory kafelków (indeksy w tilesecie).
width	Szerokość mapy w kafelkach.
height	Wysokość mapy w kafelkach.

Zwraca

true jeśli wczytywanie się powiodło, false w przeciwnym razie.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

4.20.3 Dokumentacja atrybutów składowych

4.20.3.1 m_tileset

```
const sf::Texture* TileMap::m_tileset  [private]
```

Wskaźnik do tekstury źródłowej (tileset).

4.20.3.2 m_vertices

```
sf::VertexArray TileMap::m_vertices  [private]
```

Kolekcja wierzchołków reprezentująca kafelki (Quads).

Dokumentacja dla tej klasy została wygenerowana z plików:

- TileMap.h
- TileMap.cpp

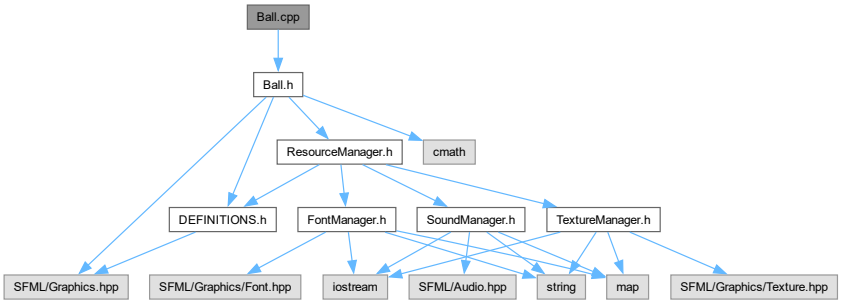
Rozdział 5

Dokumentacja plików

5.1 Dokumentacja pliku Ball.cpp

```
#include "Ball.h"
```

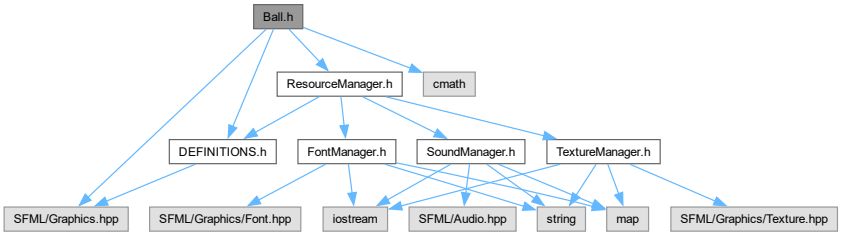
Wykres zależności załączania dla Ball.cpp:



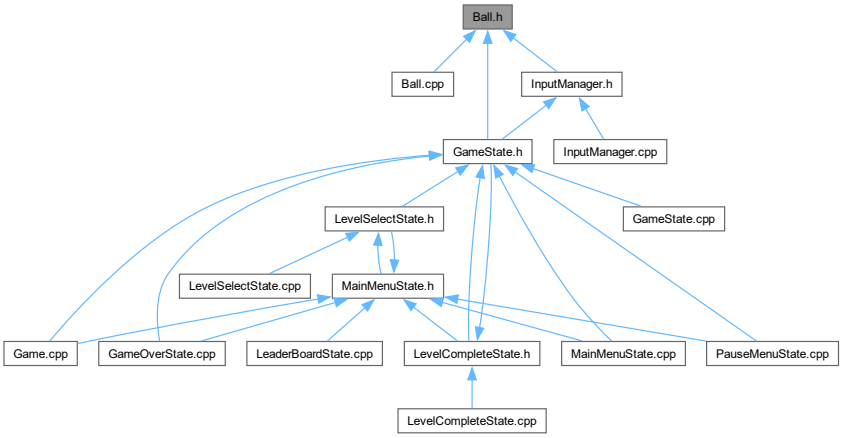
5.2 Dokumentacja pliku Ball.h

```
#include <SFML/Graphics.hpp>
#include "DEFINITIONS.h"
#include "ResourceManager.h"
#include <cmath>
```

Wykres zależności załączania dla Ball.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class Ball

  Klasa reprezentująca piłkę golfową (obiekt gracza).

5.3 Ball.h

Idź do dokumentacji tego pliku.

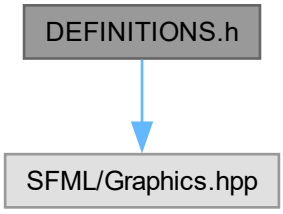
```
00001 #ifndef BALL_H
00002 #define BALL_H
00003
00004 #include <SFML/Graphics.hpp>
00005 #include "DEFINITIONS.h"
00006 #include "ResourceManager.h"
00007 #include <cmath>
00008
00016 class Ball {
00017 private:
00018     sf::Sprite player;
00019     sf::Vector2f velocity;
00020
00021 public:
00025     Ball();
00026
00032     void update(float dt, float friction);
00033
00038     void draw(sf::RenderWindow& window);
00039
00044     void applyImpulse(const sf::Vector2f& impulse);
00045
00051     bool contains(const sf::Vector2f& point) const;
00052
00057     sf::Vector2f getPosition() const;
00058
00063     void setPosition(const sf::Vector2f& pos);
00064
00068     void stop();
00069
00074     sf::Vector2f getScale() const { return player.getScale(); }
00075
00080     void bounceX();
00081
00086     void bounceY();
00087
00091     void invertVelocity();
00092
00097     sf::Vector2f getVelocity() const;
```

```
00098
00103     void setScale(float factor);
00104 };
00105
00106 #endif // BALL_H
```

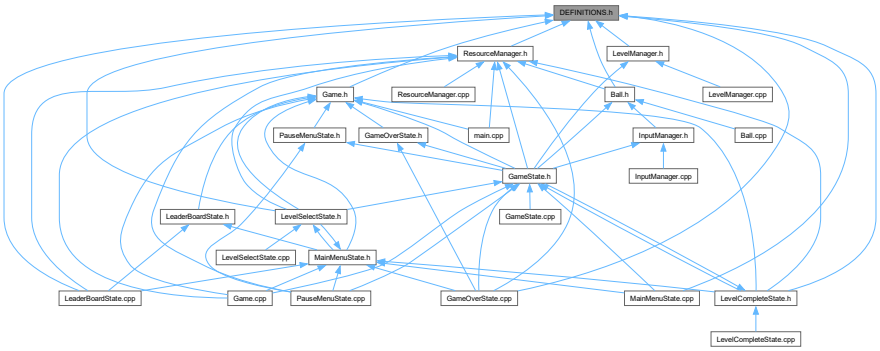
5.4 Dokumentacja pliku DEFINITIONS.h

Plik nagłówkowy zawierający globalne definicje, stałe oraz makra używane w projekcie.

`#include "SFML/Graphics.hpp"`
Wykres zależności załączania dla DEFINITIONS.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Definicje

Ustawienia Okna

Podstawowe parametry okna aplikacji.

- #define SCREEN_WIDTH 600

  Szerokość okna gry w pikselach.
- #define SCREEN_HEIGHT 800

  Wysokość okna gry w pikselach.
- #define TITLE "Mini Golf by Patryk Jozwiak"

Tytuł wyświetlany na belce okna.

Ścieżki do Tekstur

Lokalizacje plików graficznych.

- #define BALL_TEXTURE_PATH "Resources/textures/ball.png"
- #define TILESET_PATH "Resources/textures/tileset.png"
- #define HEART_WHITE_PATH "Resources/textures/heart_white.png"
- #define HEART_PATH "Resources/textures/heart.png"

Ścieżki do Czcionek

- #define FONT_PATH "Resources/font/ByteBounce.ttf"

Ścieżki do Dźwięków

Lokalizacje efektów dźwiękowych.

- #define WIN_PATH "Resources/sounds/win.wav"
- #define HIT_PATH "Resources/sounds/hit.wav"
- #define SPLASH_PATH "Resources/sounds/splash.wav"
- #define WALL_HIT_PATH "Resources/sounds/wall_hit.wav"

Konfiguracja Poziomów

Stałe dotyczące siatki i kolorów używanych w LevelManagerze.

- #define LEVEL_WIDTH 19

  Szerokość poziomu w kafelkach.
- #define LEVEL_HEIGHT 25

  Wysokość poziomu w kafelkach.
- #define TILE_SIZE sf::Vector2u(32, 32)

  Rozmiar pojedynczego kafelka w pikselach.
- const sf::Color COLOR_BALL (255, 255, 255)

  Kolor oznaczający pozycję startową piłki.
- const sf::Color COLOR_GRASS (0, 255, 0)

  Trawa (ID 0).
- const sf::Color COLOR_HOLE (255, 0, 0)

  Dołek (ID 1).
- const sf::Color COLOR_SAND (255, 255, 0)

  Piasek (ID 2).
- const sf::Color COLOR_WALL_L (0, 0, 0)

  Ściana lewa (ID 3).
- const sf::Color COLOR_WALL_R (2, 2, 2)

  Ściana prawa (ID 4).
- const sf::Color COLOR_WALL_UP (1, 1, 1)

  Ściana górna (ID 5).
- const sf::Color COLOR_WALL_DOWN (3, 3, 3)

  Ściana dolna (ID 6).
- const sf::Color COLOR_CORNER_LD (7, 7, 7)

  Narożnik lewy-dół (ID 7).
- const sf::Color COLOR_CORNER_RD (6, 6, 6)

Narożnik prawy-dół (ID 8).

- const sf::Color COLOR_CORNER_LU (4, 4, 4)

  Narożnik lewy-góra (ID 9).
- const sf::Color COLOR_CORNER_RU (5, 5, 5)

  Narożnik prawy-góra (ID 10).
- const sf::Color COLOR_WATER (0, 0, 255)

  Woda (ID 11).
- const sf::Color COLOR_CONCRETE (128, 128, 128)

  Beton (ID 12).
- const sf::Color COLOR_RAMP_UP (25, 25, 25)

  Rampa w górę (ID 13).
- const sf::Color COLOR_RAMP_DOWN (50, 50, 50)

  Rampa w dół (ID 14).
- const sf::Color COLOR_RAMP_R (100, 100, 100)

  Rampa w prawo (ID 15).
- const sf::Color COLOR_RAMP_L (150, 150, 150)

  Rampa w lewo (ID 16).

5.4.1 Opis szczegółowy

Plik nagłówkowy zawierający globalne definicje, stałe oraz makra używane w projekcie.

Plik ten przechowuje ścieżki do zasobów, parametry okna oraz definicje kolorów używanych do mapowania poziomów z plików graficznych.

5.4.2 Dokumentacja definicji

5.4.2.1 BALL_TEXTURE_PATH

```
#define BALL_TEXTURE_PATH "Resources/textures/ball.png"
```

5.4.2.2 FONT_PATH

```
#define FONT_PATH "Resources/font/ByteBounce.ttf"
```

5.4.2.3 HEART_PATH

```
#define HEART_PATH "Resources/textures/heart.png"
```

5.4.2.4 HEART_WHITE_PATH

```
#define HEART_WHITE_PATH "Resources/textures/heart_white.png"
```

5.4.2.5 HIT_PATH

```
#define HIT_PATH "Resources/sounds/hit.wav"
```

5.4.2.6 LEVEL_HEIGHT

```
#define LEVEL_HEIGHT 25
```

Wysokość poziomu w kafelkach.

5.4.2.7 LEVEL_WIDTH

```
#define LEVEL_WIDTH 19
```

Szerokość poziomu w kafelkach.

5.4.2.8 SCREEN_HEIGHT

```
#define SCREEN_HEIGHT 800
```

Wysokość okna gry w pikselach.

5.4.2.9 SCREEN_WIDTH

```
#define SCREEN_WIDTH 600
```

Szerokość okna gry w pikselach.

5.4.2.10 SPLASH_PATH

```
#define SPLASH_PATH "Resources/sounds/splash.wav"
```

5.4.2.11 TILE_SIZE

```
#define TILE_SIZE sf::Vector2u(32, 32)
```

Rozmiar pojedynczego kafelka w pikselach.

5.4.2.12 TILESET_PATH

```
#define TILESET_PATH "Resources/textures/tileset.png"
```

5.4.2.13 TITLE

```
#define TITLE "Mini Golf by Patryk Jozwiak"
```

Tytuł wyświetlany na belce okna.

5.4.2.14 WALL_HIT_PATH

```
#define WALL_HIT_PATH "Resources/sounds/wall_hit.wav"
```

5.4.2.15 WIN_PATH

```
#define WIN_PATH "Resources/sounds/win.wav"
```

5.4.3 Dokumentacja funkcji

5.4.3.1 COLOR_BALL()

```
const sf::Color COLOR_BALL (
            255 ,
            255 ,
            255  )
```

Kolor oznaczający pozycję startową piłki.

Oto graf wywoływań tej funkcji:

5.4.3.2 COLOR_CONCRETE()

```
const sf::Color COLOR_CONCRETE (
            128 ,
            128 ,
            128  )
```

Beton (ID 12).

Oto graf wywoływań tej funkcji:

5.4.3.3 COLOR_CORNER_LD()

```
const sf::Color COLOR_CORNER_LD (
            7 ,
            7 ,
            7  )
```

Narożnik lewy-dół (ID 7).

Oto graf wywoływań tej funkcji:

5.4.3.4 COLOR_CORNER_LU()

```
const sf::Color COLOR_CORNER_LU (
            4 ,
            4 ,
            4  )
```

Narożnik lewy-góra (ID 9).

Oto graf wywoływań tej funkcji:

5.4.3.5 COLOR_CORNER_RD()

```
const sf::Color COLOR_CORNER_RD (
            6 ,
            6 ,
            6  )
```

Narożnik prawy-dół (ID 8).

Oto graf wywoływań tej funkcji:

5.4.3.6 COLOR_CORNER_RU()

```
const sf::Color COLOR_CORNER_RU (
            5 ,
            5 ,
            5  )
```

Narożnik prawy-góra (ID 10).

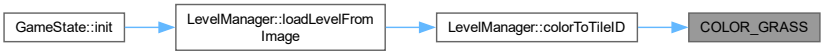
Oto graf wywoływań tej funkcji:

5.4.3.7 COLOR_GRASS()

```
const sf::Color COLOR_GRASS (
            0 ,
            255 ,
            0  )
```

Trawa (ID 0).

Oto graf wywoływań tej funkcji:

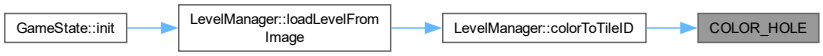


5.4.3.8 COLOR_HOLE()

```
const sf::Color COLOR_HOLE (
            255 ,
            0 ,
            0  )
```

Dołek (ID 1).

Oto graf wywoływań tej funkcji:



5.4.3.9 COLOR_RAMP_DOWN()

```
const sf::Color COLOR_RAMP_DOWN (
            50 ,
            50 ,
            50  )
```

Rampa w dół (ID 14).

Oto graf wywoływań tej funkcji:

5.4.3.10 COLOR_RAMP_L()

```
const sf::Color COLOR_RAMP_L (
            150 ,
            150 ,
            150  )
```

Rampa w lewo (ID 16).

Oto graf wywoływań tej funkcji:

5.4.3.11 COLOR_RAMP_R()

```
const sf::Color COLOR_RAMP_R (
            100 ,
            100 ,
            100  )
```

Rampa w prawo (ID 15).

Oto graf wywoływań tej funkcji:

5.4.3.12 COLOR_RAMP_UP()

```
const sf::Color COLOR_RAMP_UP (
            25 ,
            25 ,
            25  )
```

Rampa w górę (ID 13).

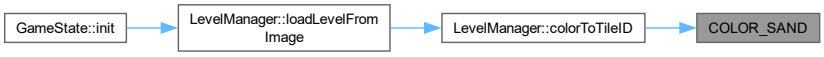
Oto graf wywoływań tej funkcji:

5.4.3.13 COLOR_SAND()

```
const sf::Color COLOR_SAND (
            255 ,
            255 ,
            0  )
```

Piasek (ID 2).

Oto graf wywoływań tej funkcji:



5.4.3.14 COLOR_WALL_DOWN()

```
const sf::Color COLOR_WALL_DOWN (
            3 ,
            3 ,
            3  )
```

Ściana dolna (ID 6).

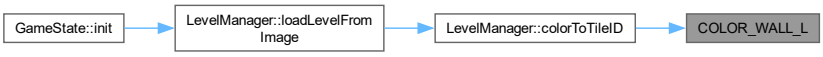
Oto graf wywoływań tej funkcji:

5.4.3.15 COLOR_WALL_L()

```
const sf::Color COLOR_WALL_L (
            0 ,
            0 ,
            0  )
```

Ściana lewa (ID 3).

Oto graf wywoływań tej funkcji:

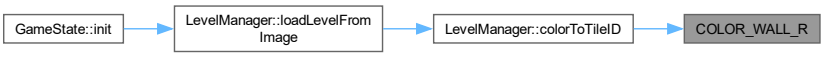


5.4.3.16 COLOR_WALL_R()

```
const sf::Color COLOR_WALL_R (
            2 ,
            2 ,
            2  )
```

Ściana prawa (ID 4).

Oto graf wywoływań tej funkcji:



5.4.3.17 COLOR_WALL_UP()

```
const sf::Color COLOR_WALL_UP (
            1 ,
            1 ,
            1  )
```

Ściana górna (ID 5).

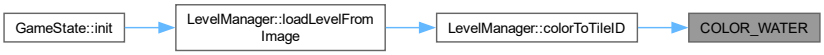
Oto graf wywoływań tej funkcji:

5.4.3.18 COLOR_WATER()

```
const sf::Color COLOR_WATER (
            0 ,
            0 ,
            255  )
```

Woda (ID 11).

Oto graf wywoływań tej funkcji:



5.5 DEFINITIONS.h

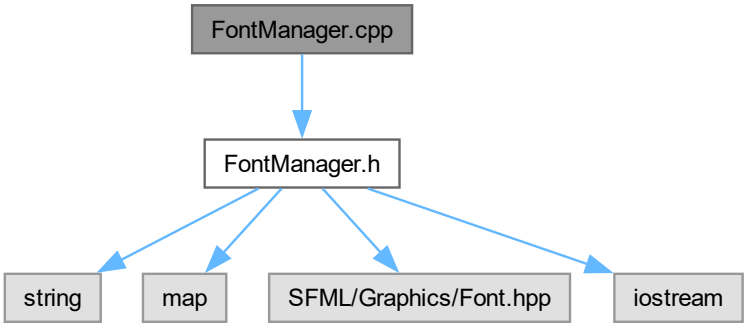
Idź do dokumentacji tego pliku.

```
00001 #ifndef MINI_GOLF_DEFINITIONS_H
00002 #define MINI_GOLF_DEFINITIONS_H
00003
00012 #include "SFML/Graphics.hpp"
00013
00018 #define SCREEN_WIDTH 600
00019 #define SCREEN_HEIGHT 800
00020 #define TITLE "Mini Golf by Patryk Jozwiak"
00022
00027 #define BALL_TEXTURE_PATH "Resources/textures/ball.png"
00028 #define TILESET_PATH "Resources/textures/tileset.png"
00029 #define HEART_WHITE_PATH "Resources/textures/heart_white.png"
00030 #define HEART_PATH "Resources/textures/heart.png"
00032
00035 #define FONT_PATH "Resources/font/ByteBounce.ttf"
00037
00042 #define WIN_PATH "Resources/sounds/win.wav"
00043 #define HIT_PATH "Resources/sounds/hit.wav"
00044 #define SPLASH_PATH "Resources/sounds/splash.wav"
00045 #define WALL_HIT_PATH "Resources/sounds/wall_hit.wav"
00047
00052 #define LEVEL_WIDTH  19
00053 #define LEVEL_HEIGHT  25
00054 #define TILE_SIZE sf::Vector2u(32, 32)
00055
00056 // Definicje kolorów do parsowania mapy z obrazka
00057 const sf::Color COLOR_BALL (255,255,255);
00058 const sf::Color COLOR_GRASS(0, 255, 0);
00059 const sf::Color COLOR_HOLE(255, 0, 0);
00060 const sf::Color COLOR_SAND(255, 255, 0);
00061 const sf::Color COLOR_WALL_L(0,0,0);
00062 const sf::Color COLOR_WALL_R(2, 2, 2);
00063 const sf::Color COLOR_WALL_UP(1, 1, 1);
00064 const sf::Color COLOR_WALL_DOWN(3, 3, 3);
00065 const sf::Color COLOR_CORNER_LD(7, 7, 7);
00066 const sf::Color COLOR_CORNER_RD( 6, 6, 6 );
00067 const sf::Color COLOR_CORNER_LU( 4, 4, 4);
00068 const sf::Color COLOR_CORNER_RU( 5, 5, 5);
00069 const sf::Color COLOR_WATER(0, 0, 255);
00070 const sf::Color COLOR_CONCRETE(128, 128, 128);
00071 const sf::Color COLOR_RAMP_UP(25, 25, 25);
00072 const sf::Color COLOR_RAMP_DOWN( 50, 50, 50);
00073 const sf::Color COLOR_RAMP_R( 100, 100, 100);
00074 const sf::Color COLOR_RAMP_L( 150, 150, 150);
00076
00077 #endif //MINI_GOLF_DEFINITIONS_H
```

5.6 Dokumentacja pliku FontManager.cpp

```
#include "FontManager.h"
```

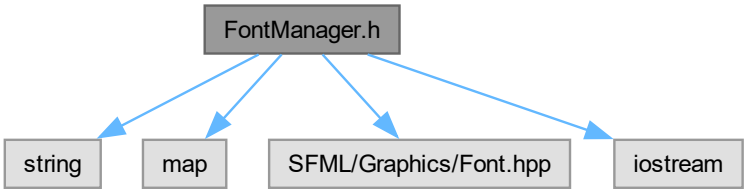
Wykres zależności załączania dla FontManager.cpp:



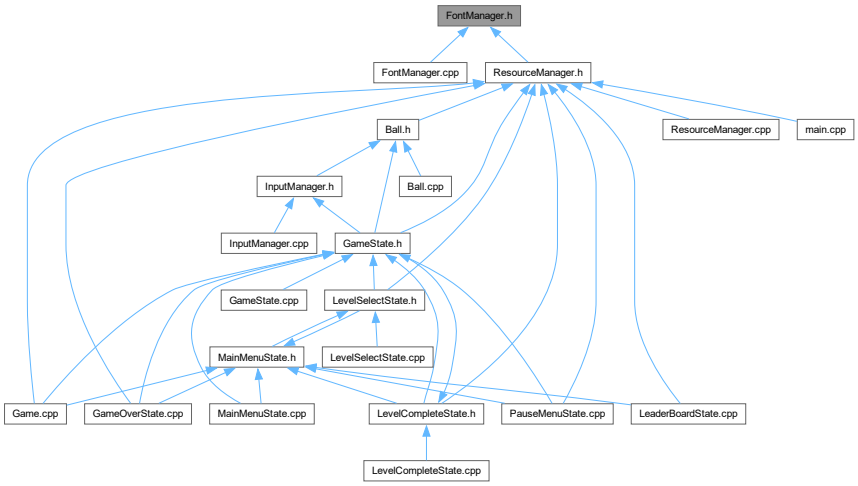
5.7 Dokumentacja pliku FontManager.h

```
#include <string>
#include <map>
#include "SFML/Graphics/Font.hpp"
#include <iostream>
```

Wykres zależności załączania dla FontManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class FontManager

  Klasa statyczna zarządzająca zasobami czcionek w grze.

5.8 FontManager.h

Idź do dokumentacji tego pliku.

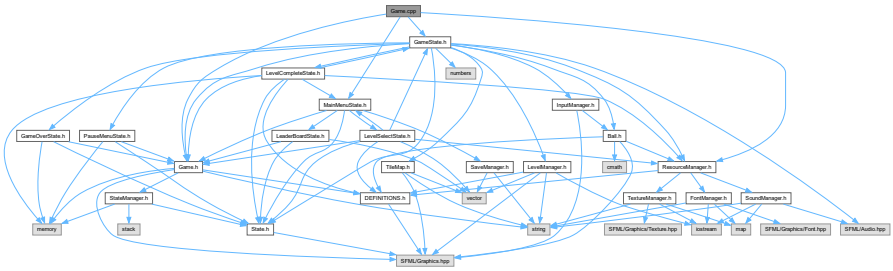
```
00001 //
00002 // Created by patry on 04.11.2025.
00003 //
00004
00005 #ifndef MINI_GOLF_FONTMANAGER_H
00006 #define MINI_GOLF_FONTMANAGER_H
00007
00008 #include <string>
00009 #include <map>
00010 #include "SFML/Graphics/Font.hpp"
00011 #include <iostream>
00012
00020 class FontManager {
00021 public:
00025     FontManager() = delete;
00026
00032     static void load(const std::string& key, const std::string& path);
00033
00040     static const sf::Font& get(const std::string& key);
00041
00045     static void cleanup();
00046
00047 private:
00051     static std::map<std::string, sf::Font> m_Font;
00052 };
00053
00054 #endif //MINI_GOLF_FONTMANAGER_H
```

5.9 Dokumentacja pliku Game.cpp

```
#include "Game.h"
#include "GameState.h"
```

```
#include "ResourceManager.h"
#include "MainMenuState.h"
```

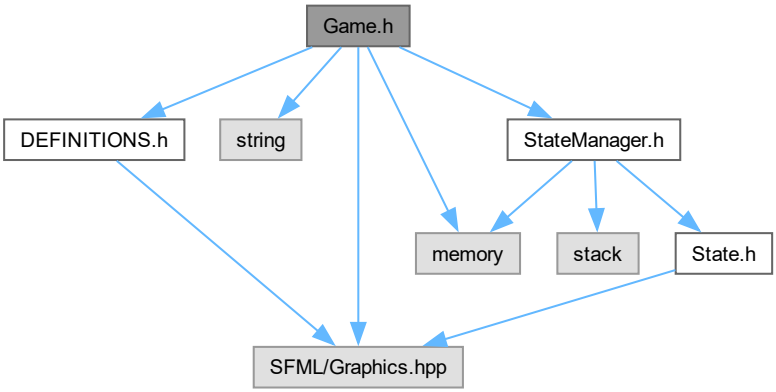
Wykres zależności załączania dla Game.cpp:



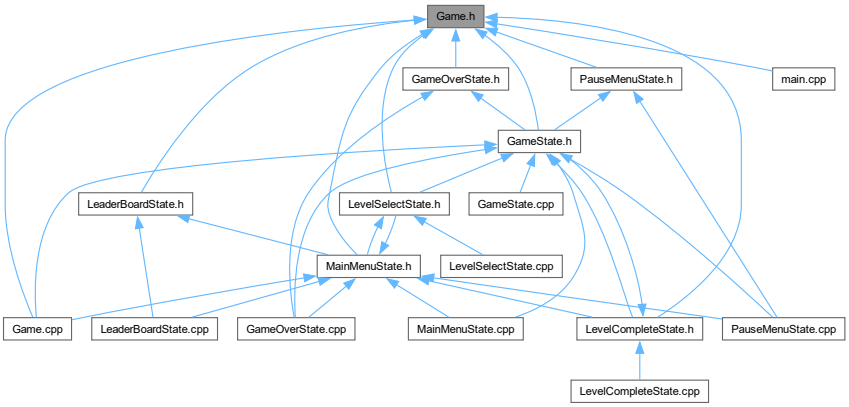
5.10 Dokumentacja pliku Game.h

```
#include <memory>
#include <string>
#include <SFML/Graphics.hpp>
#include "StateManager.h"
#include "DEFINITIONS.h"
```

Wykres zależności załączania dla Game.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- struct GameData

  Struktura przechowująca współdzielone dane gry.
- class Game

  Główna klasa silnika gry.

Definicje typów

- typedef std::shared_ptr< GameData > GameDataRef

  Alias dla std::shared_ptr<GameData>. Ułatwia przekazywanie danych gry.

5.10.1 Dokumentacja definicji typów

5.10.1.1 GameDataRef

`GameDataRef`

Alias dla std::shared_ptr<GameData>. Ułatwia przekazywanie danych gry.

5.11 Game.h

Idź do dokumentacji tego pliku.

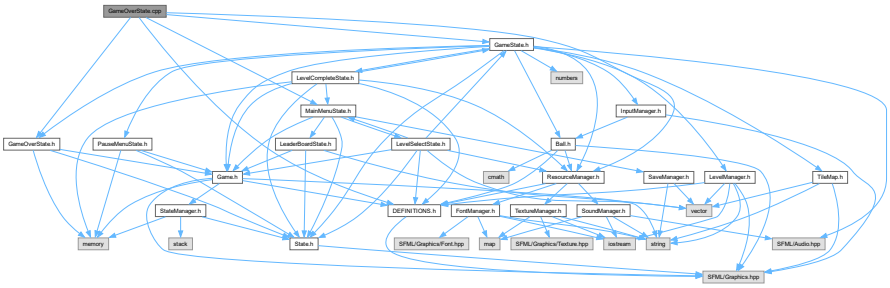
```
00001 #ifndef GAME_H
00002 #define GAME_H
00003
00004 #include <memory>
00005 #include <string>
00006 #include <SFML/Graphics.hpp>
00007 #include "StateManager.h"
00008 #include "DEFINITIONS.h"
00009
00017 struct GameData {
00018     StateManager machine;
00019     sf::RenderWindow window;
```

```
};

typedef std::shared_ptr<GameData> GameDataRef;

class Game {
public:
    Game(int width, int height, std::string title);

    void run();

private:
    const sf::Time timePerFrame = sf::seconds(1.f / 60.f);

    sf::Clock clock;

    GameDataRef m_data = std::make_shared<GameData>();
};

#endif // GAME_H
```

5.12 Dokumentacja pliku GameOverState.cpp

```
#include "GameOverState.h"
#include "GameState.h"
#include "MainMenuState.h"
#include "ResourceManager.h"
#include "DEFINITIONS.h"
```

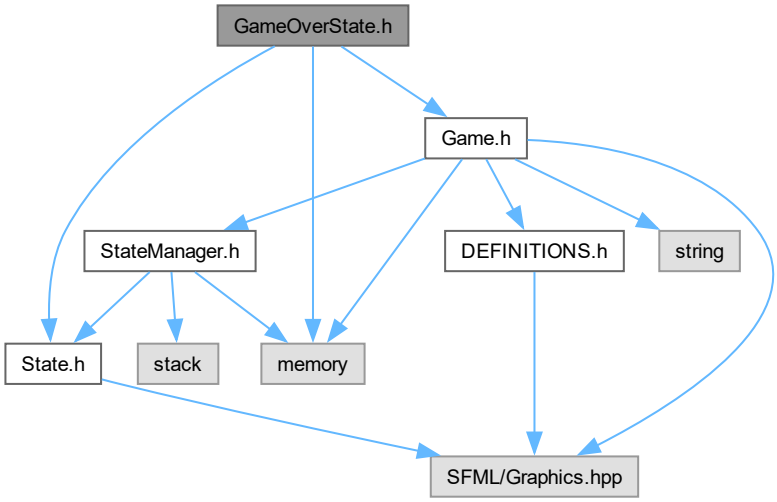
Wykres zależności załączania dla GameOverState.cpp:



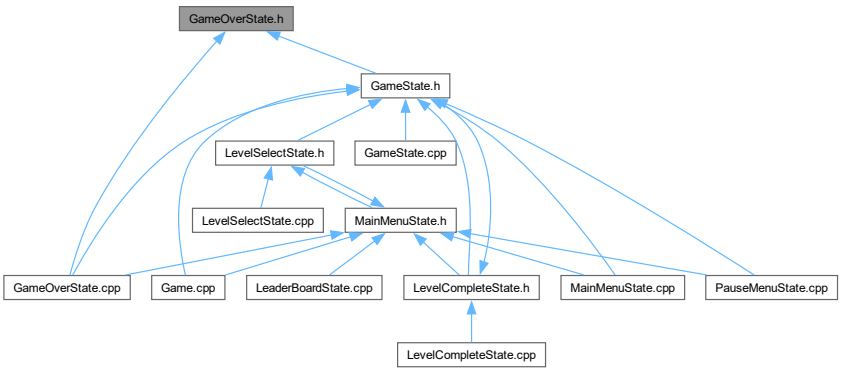
5.13 Dokumentacja pliku GameOverState.h

```
#include "State.h"
#include "Game.h"
#include <memory>
```

Wykres zależności załączania dla GameOverState.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class GameOverState

  Stan gry wyświetlany po przegranej (Game Over).

5.14 GameOverState.h

Idź do dokumentacji tego pliku.

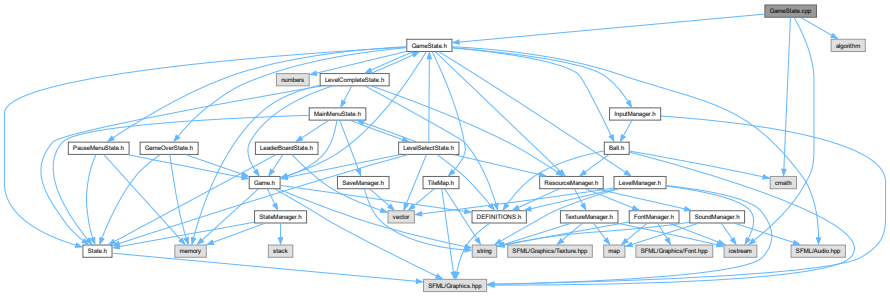
```
00001 #ifndef MINI_GOLF_GAMEOVERTSTATE_H
00002 #define MINI_GOLF_GAMEOVERTSTATE_H
00003
00004 #include "State.h"
```

```
#include "Game.h"
#include <memory>

class GameOverState : public State {
public:
    GameOverState(GameDataRef data, sf::Sprite bgSprite, std::shared_ptr<sf::Texture> bgTexture, int level);

    void init() override;

    void handleInput(sf::Event& event) override;

    void update(float dt) override;

    void draw(float dt) override;

private:
    GameDataRef m_data;
    sf::Sprite m_backgroundSprite;
    std::shared_ptr<sf::Texture> m_bgTexture;
    int m_level;

    sf::RectangleShape m_dimmer;

    sf::Text m_title;
    sf::Text m_retryButton;
    sf::Text m_menuButton;

    // Zmienne do efektu przejścia (Fader)
    sf::RectangleShape m_fadeRect;
    float m_alpha;
    bool m_isTransitioning;
    bool m_goToMenu;
    bool m_retryLevel;
};

#endif //MINI_GOLF_GAMEOVERTSTATE_H
```

5.15 Dokumentacja pliku GameState.cpp

```
#include "GameState.h"
#include <cmath>
#include <iostream>
#include <algorithm>
```

Wykres zależności załączania dla GameState.cpp:

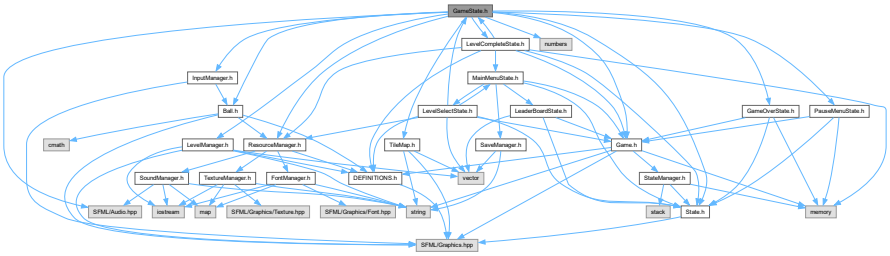


5.16 Dokumentacja pliku GameState.h

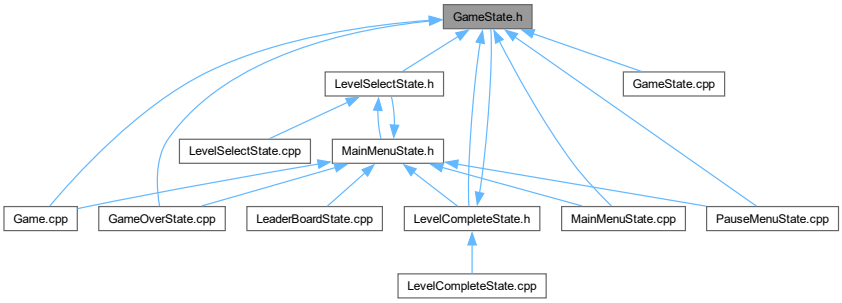
```
#include "State.h"
#include "Game.h"
#include "Ball.h"
#include "TileMap.h"
```

```
#include "InputManager.h"
#include "ResourceManager.h"
#include "PauseMenuState.h"
#include <SFML/Audio.hpp>
#include <numbers>
#include "LevelManager.h"
#include "LevelCompleteState.h"
#include "GameOverState.h"
```

Wykres zależności załączania dla GameState.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class GameState

  Główny stan rozgrywki.

5.17 GameState.h

Idź do dokumentacji tego pliku.

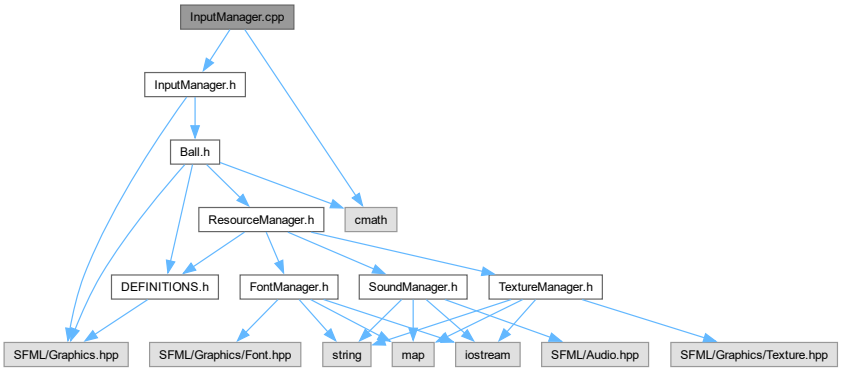
```
00001 #ifndef MINI_GOLF_GAMESTATE_H
00002 #define MINI_GOLF_GAMESTATE_H
00003
00004 #include "State.h"
00005 #include "Game.h"
00006 #include "Ball.h"
00007 #include "TileMap.h"
00008 #include "InputManager.h"
00009 #include "ResourceManager.h"
00010 #include "PauseMenuState.h"
00011 #include <SFML/Audio.hpp>
00012 #include <numbers>
```

```
#include "LevelManager.h"
#include "LevelCompleteState.h"
#include "GameOverState.h"

class GameState : public State {
public:
    GameState(GameDataRef data, int level);

    void init() override;

    void handleInput(sf::Event& event) override;

    void update(float dt) override;

    void draw(float dt) override;

private:
    GameDataRef m_data;
    Ball ball;
    InputManager inputManager;
    TileMap tileMap;
    std::vector<int> levelData;
    sf::Vector2f lastSafePos;
    sf::Vector2f forceVector;
    int m_level;
    sf::CircleShape m_shadow;

    // UI Elements
    sf::Text scoreText;
    sf::Text winText;
    sf::RectangleShape aimLine;

    // Zmienne logiczne stanu gry
    int strokes;
    bool ballInHole;
    bool wasInWater;
    bool isSinking;
    bool isJumping;
    float jumpTimer;
    float levelFinishTimer;
    float maxJumpTime;

    // System Audio
    sf::Sound m_hitSound;
    sf::Sound m_splashSound;
    sf::Sound m_wallHitSound;
    sf::Sound m_winSound;

    // Metody pomocnicze (Helper Methods)
    void initUI();
    void initSounds();
    void updateAimLine();

    int getTileAt(sf::Vector2f position);

    bool isWall(int tileId);

    // Efekt przejścia (Fader)
    sf::RectangleShape m_fadeRect;
    float m_alpha;

    // System Żyć
    int lives;
    std::vector<sf::Sprite> heartSprites;
    void updateHearts();
    void resetLevel();
};

#endif //MINI_GOLF_GAMESTATE_H
```

5.18 Dokumentacja pliku InputManager.cpp

```
#include "InputManager.h"
#include <cmath>
```

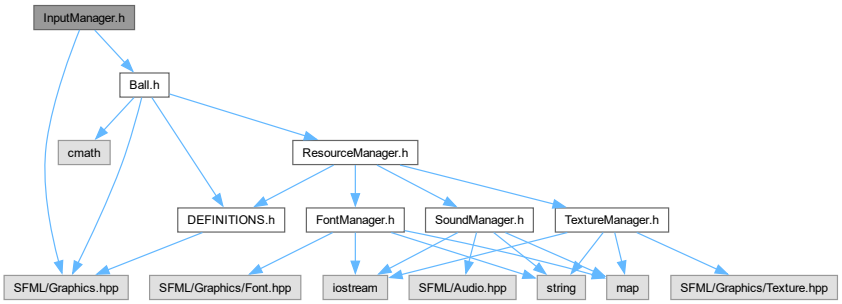
Wykres zależności załączania dla InputManager.cpp:



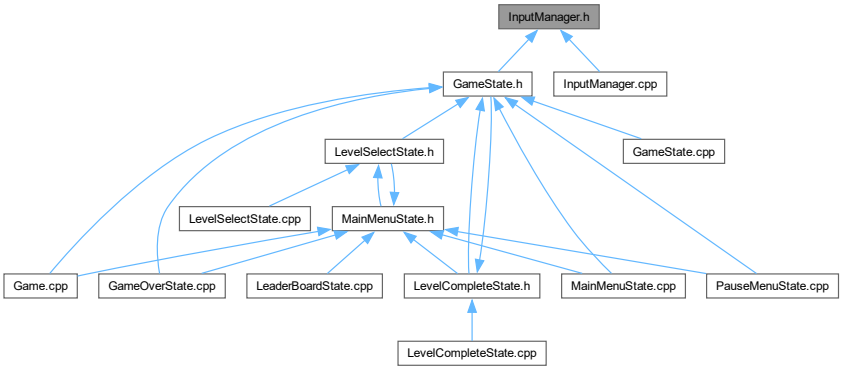
5.19 Dokumentacja pliku InputManager.h

```
#include <SFML/Graphics.hpp>
#include "Ball.h"
```

Wykres zależności załączania dla InputManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class InputManager

  Klasa obsługująca sterowanie (mechanika Drag & Shoot).

5.20 InputManager.h

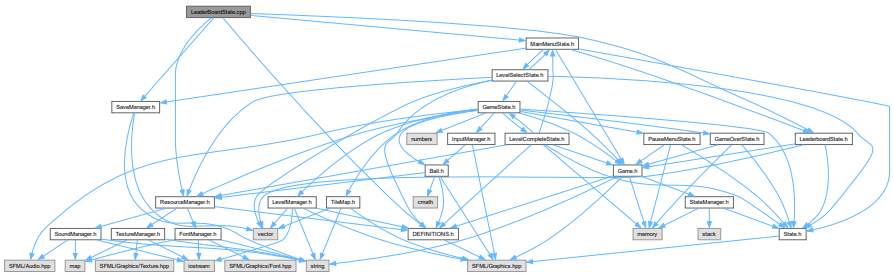
Idź do dokumentacji tego pliku.

```
00001 #ifndef INPUTMANAGER_H
00002 #define INPUTMANAGER_H
00003
00004 #include <SFML/Graphics.hpp>
00005 #include "Ball.h"
00006
00014 class InputManager {
00015 private:
00016     bool isGrabbed;
00017     bool ready;
00018     sf::Vector2f ballStartPos;
00019     sf::Vector2f currentMousePos;
00020     float force;
00021
00022 public:
00026     InputManager();
00027
00038     void handleEvent(const sf::Event& event, Ball& ball, sf::RenderWindow& window);
00039
00044     bool isReady() const;
00045
00055     sf::Vector2f getForceVector();
00056
00064     sf::Vector2f getCurrentForce(const sf::RenderWindow& window) const;
00065
00070     bool isDragging() const;
00071 };
00072
00073 #endif // INPUTMANAGER_H
```

5.21 Dokumentacja pliku LeaderBoardState.cpp

```
#include "LeaderboardState.h"
#include "MainMenuState.h"
#include "ResourceManager.h"
#include "SaveManager.h"
#include "DEFINITIONS.h"
```

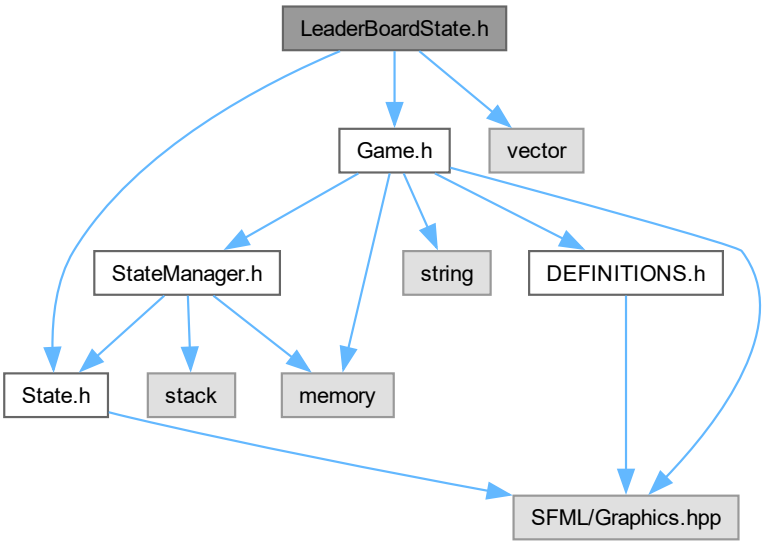
Wykres zależności załączania dla LeaderBoardState.cpp:



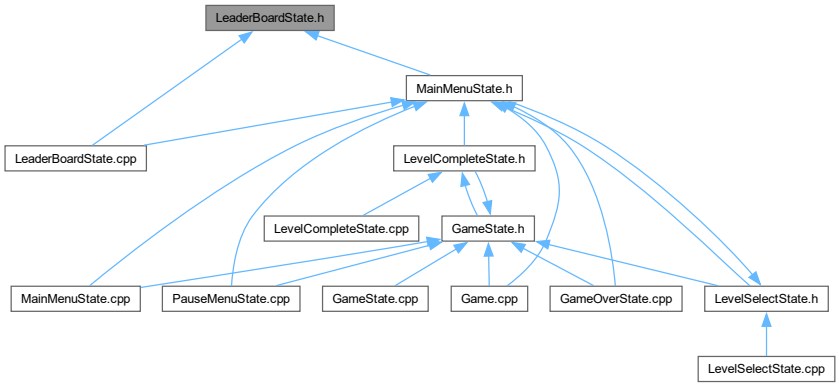
5.22 Dokumentacja pliku LeaderBoardState.h

```
#include "State.h"
#include "Game.h"
#include <vector>
```

Wykres zależności załączania dla LeaderBoardState.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class LeaderboardState

  Stan wyświetlający tabelę najlepszych wyników.

5.23 LeaderBoardState.h

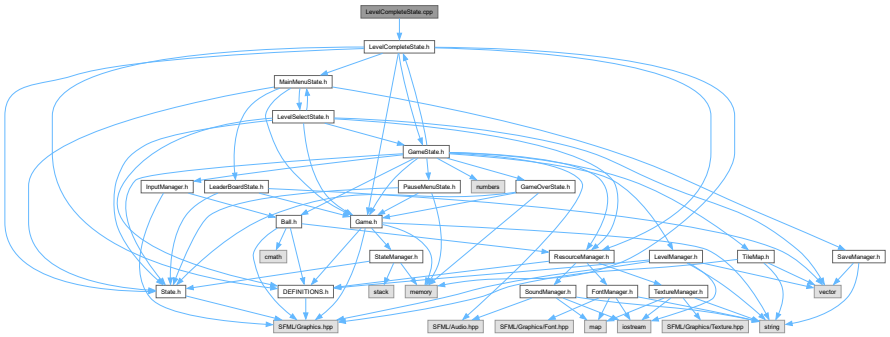
Idź do dokumentacji tego pliku.

```
00001 //
00002 // Created by patry on 17.01.2026.
00003 //
00004
00005 #ifndef MINI_GOLF_LEADERBOARDSTATE_H
00006 #define MINI_GOLF_LEADERBOARDSTATE_H
00007
00008 #include "State.h"
00009 #include "Game.h"
00010 #include <vector>
00011
00018 class LeaderboardState : public State {
00019 public:
00024     LeaderboardState(GameDataRef data);
00025
00029     void init() override;
00030
00034     void handleInput(sf::Event& event) override;
00035
00039     void update(float dt) override;
00040
00044     void draw(float dt) override;
00045
00046 private:
00047     GameDataRef m_data;
00048     sf::Text m_title;
00049     sf::Text m_backButton;
00050     std::vector<sf::Text> m_scoreTexts;
00051
00052     sf::RectangleShape m_fadeRect;
00053     float m_alpha;
00054     bool m_isFadingIn;
00055     bool m_isGoingBack;
00056 };
00057
00058 #endif //MINI_GOLF_LEADERBOARDSTATE_H
```

5.24 Dokumentacja pliku LevelCompleteState.cpp

```
#include "LevelCompleteState.h"
```

Wykres zależności załączania dla LevelCompleteState.cpp:

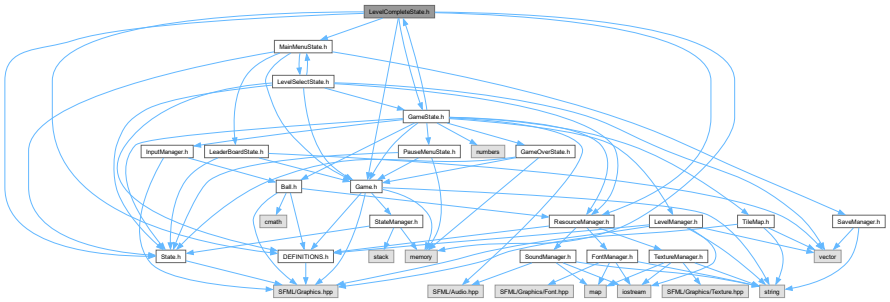


5.25 Dokumentacja pliku LevelCompleteState.h

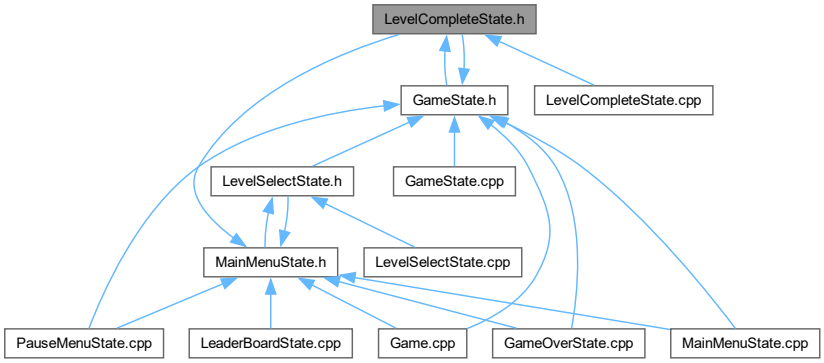
```
#include "State.h"
#include "Game.h"
#include "GameState.h"
```

```
#include "MainMenuState.h"
#include "ResourceManager.h"
#include "DEFINITIONS.h"
#include <memory>
```

Wykres zależności załączania dla LevelCompleteState.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class LevelCompleteState

  Stan gry wyświetlany po pomyślnym ukończeniu poziomu.

5.26 LevelCompleteState.h

Idź do dokumentacji tego pliku.

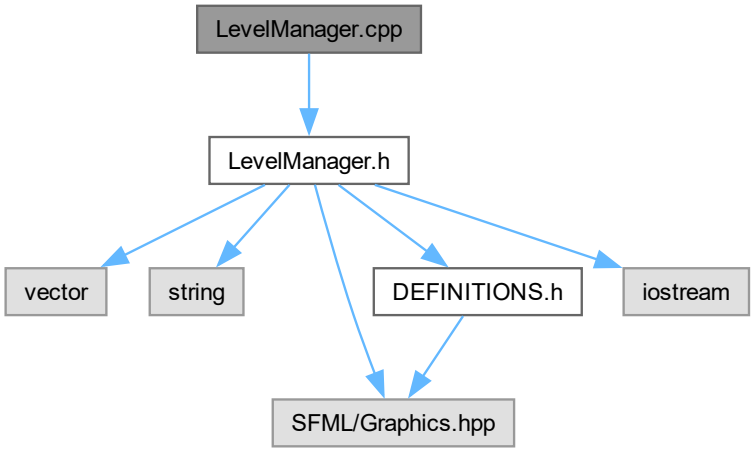
```
00001 #ifndef MINI_GOLF_LEVELCOMPLETESTATE_H
00002 #define MINI_GOLF_LEVELCOMPLETESTATE_H
00003
00004 #include "State.h"
00005 #include "Game.h"
00006 #include "GameState.h"
00007 #include "MainMenuState.h"
00008 #include "ResourceManager.h"
00009 #include "DEFINITIONS.h"
00010 #include <memory>
00011
00021 class LevelCompleteState : public State {
```

```
public:
    LevelCompleteState(GameDataRef data, sf::Sprite bgSprite, std::shared_ptr<sf::Texture> bgTexture,
int strokes, int currentLevel);

    void init() override;

    void handleInput(sf::Event& event) override;

    void update(float dt) override;

    void draw(float dt) override;

private:
    GameDataRef m_data;
    sf::Sprite m_backgroundSprite;
    std::shared_ptr<sf::Texture> m_bgTexture;

    int m_strokes;
    int m_currentLevel;

    sf::RectangleShape m_dimmer;

    sf::Text m_title;
    sf::Text m_scoreText;

    sf::Text m_nextLevelButton;
    sf::Text m_menuButton;
    sf::Text m_retryButton;

    // Zmienne do efektu przejścia (Fader)
    sf::RectangleShape m_fadeRect;
    float m_alpha;
    bool m_isTransitioning;
    bool m_goToMenu;
    bool m_retryLevel;
};

#endif //MINI_GOLF_LEVELCOMPLETESTATE_H
```

5.27 Dokumentacja pliku LevelManager.cpp

`#include "LevelManager.h"`

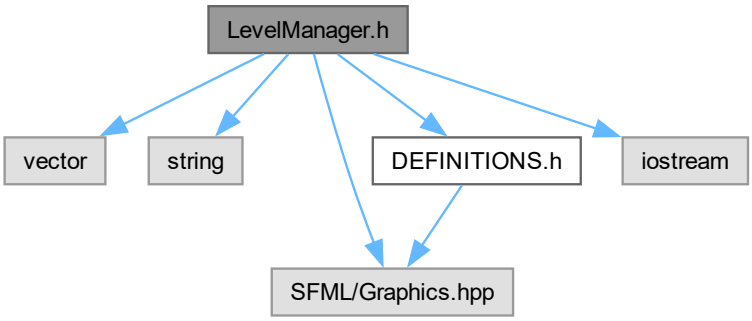
Wykres zależności załączania dla LevelManager.cpp:



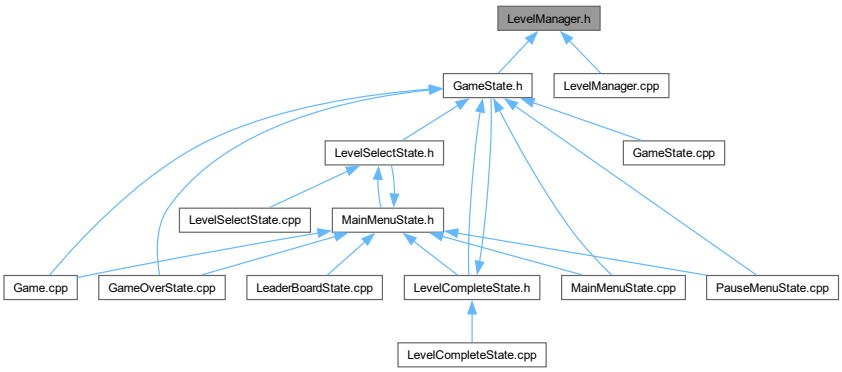
5.28 Dokumentacja pliku LevelManager.h

```
#include <vector>
#include <string>
#include <SFML/Graphics.hpp>
#include "DEFINITIONS.h"
#include <iostream>
```

Wykres zależności załączania dla LevelManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class LevelManager

  Klasa statyczna odpowiedzialna za wczytywanie poziomów.

5.29 LevelManager.h

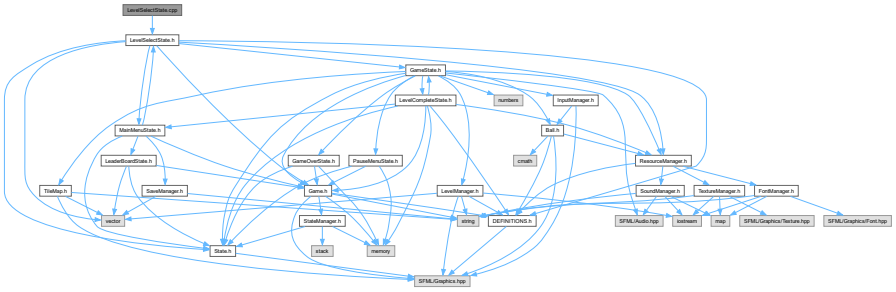
Idź do dokumentacji tego pliku.

```
#ifndef MINI_GOLF_LEVELMANAGER_H
#define MINI_GOLF_LEVELMANAGER_H

#include <vector>
#include <string>
#include <SFML/Graphics.hpp>
#include "DEFINITIONS.h"
#include <iostream>

class LevelManager {
public:
    static std::vector<int> loadLevelFromImage(int levelNumber);

    static sf::Vector2f startPosition;

private:
    static int colorToTileID(sf::Color color);
};

#endif //MINI_GOLF_LEVELMANAGER_H
```

5.30 Dokumentacja pliku LevelSelectState.cpp

`#include "LevelSelectState.h"`

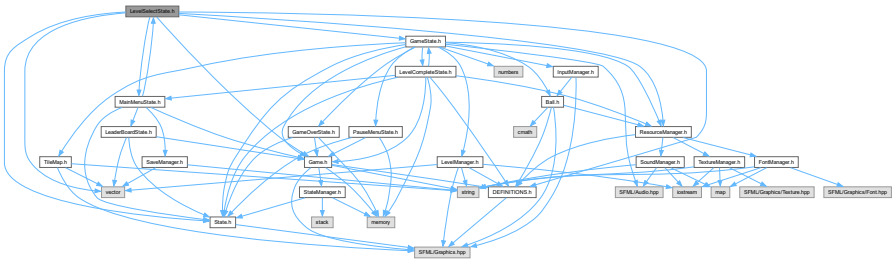
Wykres zależności załączania dla LevelSelectState.cpp:



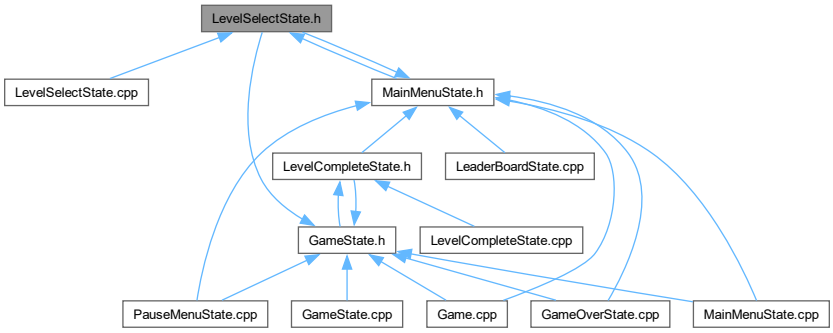
5.31 Dokumentacja pliku LevelSelectState.h

```
#include "State.h"
#include "Game.h"
#include <vector>
#include "GameState.h"
#include "MainMenuState.h"
#include "ResourceManager.h"
#include "DEFINITIONS.h"
```

Wykres zależności załączania dla LevelSelectState.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class LevelSelectState

  Stan wyboru poziomu.

5.32 LevelSelectState.h

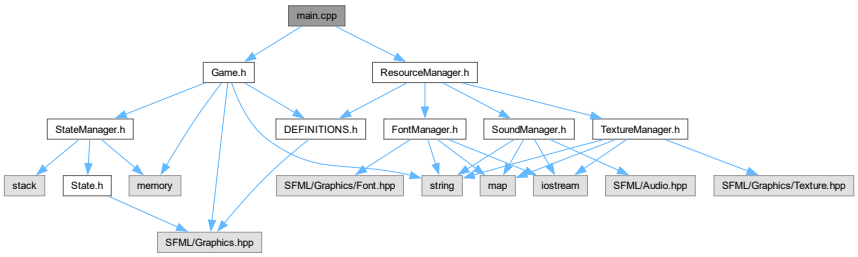
Idź do dokumentacji tego pliku.

```
00001 #ifndef MINI_GOLF_LEVELSELECTSTATE_H
00002 #define MINI_GOLF_LEVELSELECTSTATE_H
00003
00004 #include "State.h"
00005 #include "Game.h"
00006 #include <vector>
00007 #include "GameState.h"
00008 #include "MainMenuState.h"
00009 #include "ResourceManager.h"
00010 #include "DEFINITIONS.h"
00011
00019 class LevelSelectState : public State {
00020 public:
00025     LevelSelectState(GameDataRef data);
00026
00030     void init() override;
00031
00037     void handleInput(sf::Event& event) override;
00038
00043     void update(float dt) override;
00044
00049     void draw(float dt) override;
00050
00051 private:
00052     GameDataRef m_data;
00053
00054     sf::Text m_title;
00055     sf::Text m_backButton;
00056
00057     std::vector<sf::Text> m_levelButtons;
00058
00059     // Zmienne do przejścia (Fader)
00060     sf::RectangleShape m_fadeRect;
00061     float m_alpha;
00062     bool m_isFadingIn;
00063     bool m_isTransitioning;
00064     int m_nextLevelToLoad;
00065 };
00066
00067 #endif //MINI_GOLF_LEVELSELECTSTATE_H
```

5.33 Dokumentacja pliku main.cpp

```
#include "Game.h"
#include "ResourceManager.h"
```

Wykres zależności załączania dla main.cpp:



Funkcje

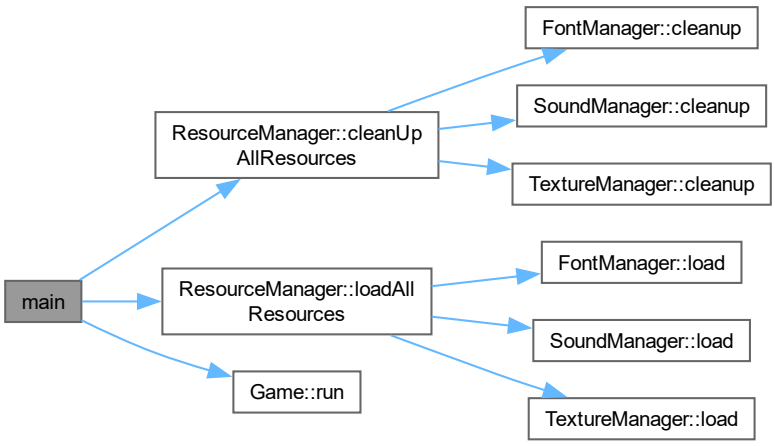
- int main ()

5.33.1 Dokumentacja funkcji

5.33.1.1 main()

```
int main ( )
```

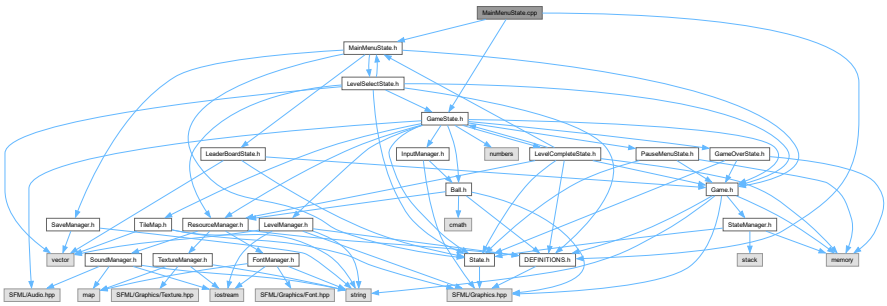
Oto graf wywołań dla tej funkcji:



5.34 Dokumentacja pliku MainMenuState.cpp

```
#include "MainMenuState.h"
#include "GameState.h"
#include "DEFINITIONS.h"
```

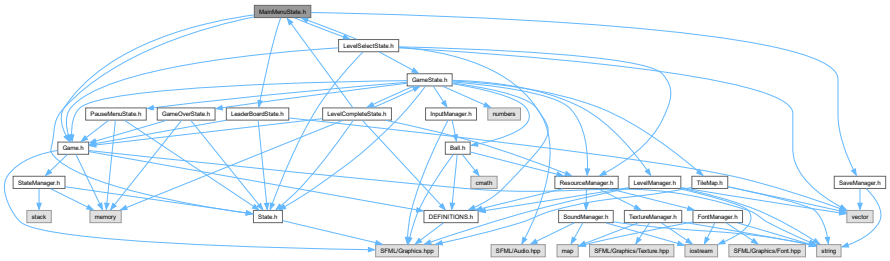
Wykres zależności załączania dla MainMenuState.cpp:



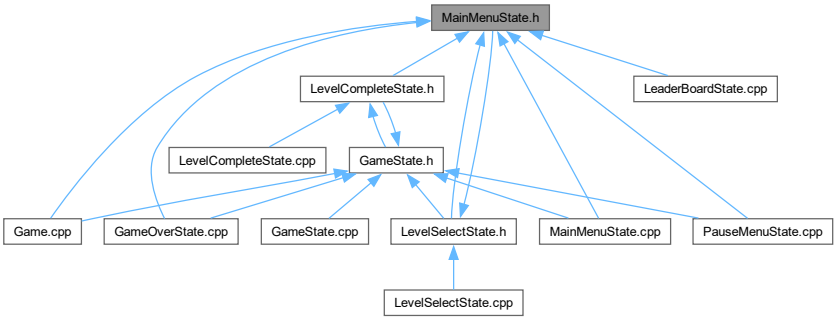
5.35 Dokumentacja pliku MainMenuState.h

```
#include "State.h"
#include "Game.h"
#include "LevelSelectState.h"
#include "SaveManager.h"
#include "LeaderBoardState.h"
```

Wykres zależności załączania dla MainMenuState.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class MainMenuState

  Stan początkowy gry (Menu Główne).

5.36 MainMenuState.h

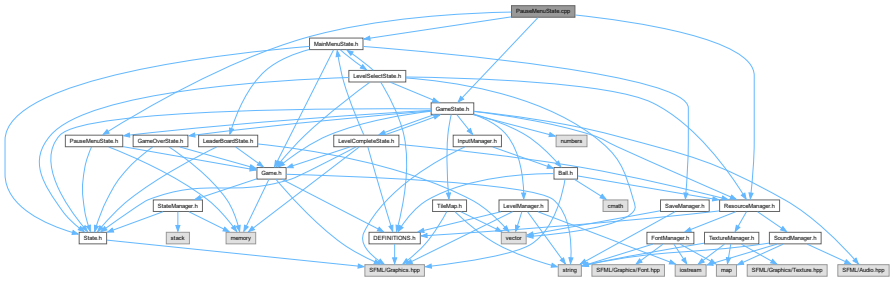
Idź do dokumentacji tego pliku.

```
00001 #ifndef MINI_GOLF_MAINMENUSTATE_H
00002 #define MINI_GOLF_MAINMENUSTATE_H
00003
00004 #include "State.h"
00005 #include "Game.h"
00006 #include "LevelSelectState.h"
00007 #include "SaveManager.h"
00008 #include "LeaderBoardState.h"
00009
00020 class MainMenuState : public State {
00021 public:
00026     MainMenuState(GameDataRef data);
00027
00032     void init() override;
00033
00038     void handleInput(sf::Event& event) override;
00039
00044     void update(float dt) override;
00045
00050     void draw(float dt) override;
00051
00052 private:
00053     GameDataRef m_data;
00054
00055     sf::Sprite m_background;
00056
00057     sf::Text m_title;
00058     sf::Text m_playButton;
00059     sf::Text m_exitButton;
00060     sf::Text m_leaderboardButton;
00061
00062     // Zmienne do efektu przejścia (Fader)
00063     sf::RectangleShape m_fadeRect;
00064     float m_alpha;
00065     bool m_isTransitioning;
00066     bool m_isFadingIn;
00067     bool m_goToLeaderboard;
00068 };
00069
00070 #endif //MINI_GOLF_MAINMENUSTATE_H
```

5.37 Dokumentacja pliku PauseMenuState.cpp

```
#include "PauseMenuState.h"
#include "MainMenuState.h"
#include "ResourceManager.h"
#include "GameState.h"
```

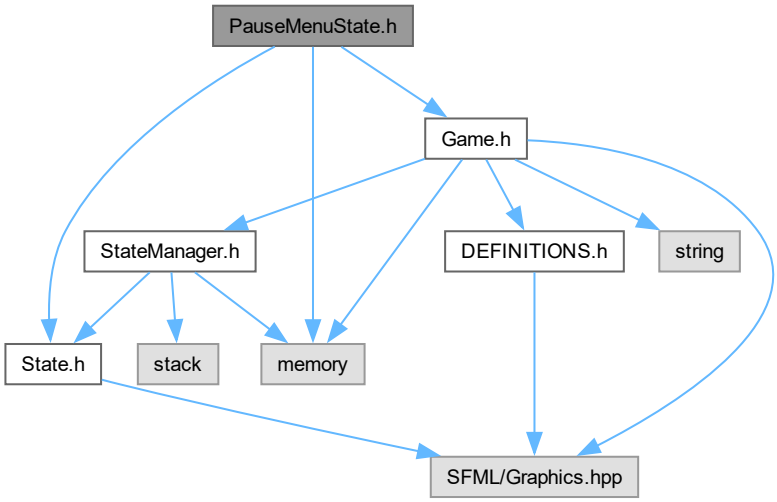
Wykres zależności załączania dla PauseMenuState.cpp:



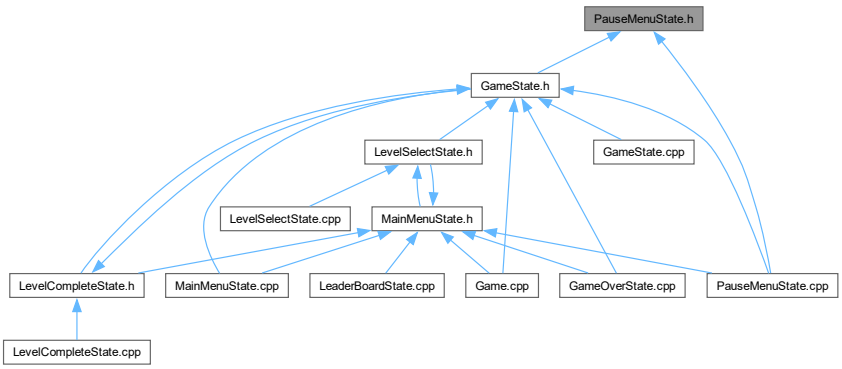
5.38 Dokumentacja pliku PauseMenuState.h

```
#include "State.h"
#include "Game.h"
#include <memory>
```

Wykres zależności załączania dla PauseMenuState.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class PauseMenuState

  Stan pauzy nakładany na stan gry.

5.39 PauseMenuState.h

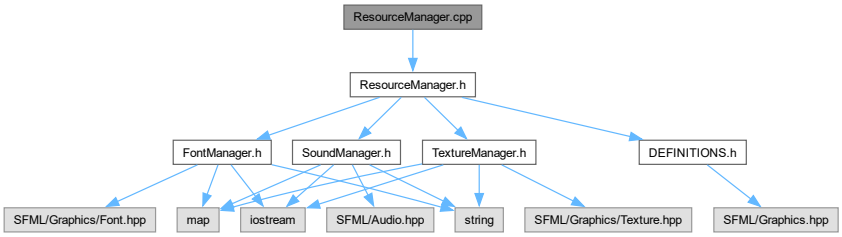
Idź do dokumentacji tego pliku.

```
#ifndef MINI_GOLF_PAUSEMENUSTATE_H
#define MINI_GOLF_PAUSEMENUSTATE_H

#include "State.h"
#include "Game.h"
#include <memory>

class PauseMenuState : public State {
public:
    PauseMenuState(GameDataRef data, sf::Sprite bgSprite, std::shared_ptr<sf::Texture> bgTexture, int levelNumber);

    void init() override;

    void handleInput(sf::Event& event) override;

    void update(float dt) override;

    void draw(float dt) override;

private:
    GameDataRef m_data;

    std::shared_ptr<sf::Texture> m_bgTexture;
    sf::Sprite m_backgroundSprite;
    sf::RectangleShape m_dimmer;

    sf::Text m_pauseTitle;
    sf::Text m_resumeButton;
    sf::Text m_resetButton;
    sf::Text m_menuButton;

    sf::RectangleShape m_fadeRect;
    float m_alpha;
    bool m_isGoingToMenu;
    bool m_isReseting;
    int m_currentLevel;
};

#endif //MINI_GOLF_PAUSEMENUSTATE_H
```

5.40 Dokumentacja pliku ResourceManager.cpp

`#include "ResourceManager.h"`

Wykres zależności załączania dla ResourceManager.cpp:

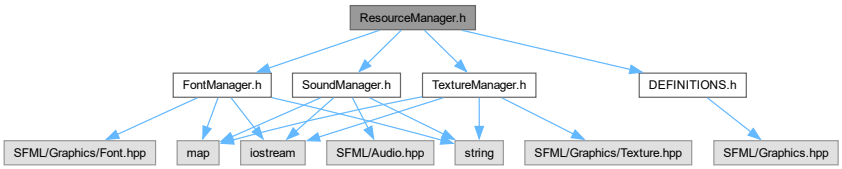


5.41 Dokumentacja pliku ResourceManager.h

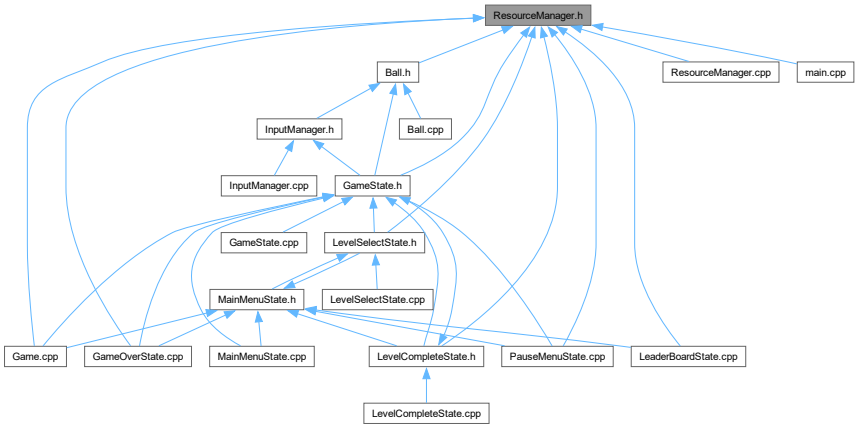
```
#include "TextureManager.h"
#include "FontManager.h"
```

```
#include "SoundManager.h"
#include "DEFINITIONS.h"
```

Wykres zależności załączania dla ResourceManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class ResourceManager

  Fasada zarządzająca ładowaniem wszystkich zasobów aplikacji.

5.42 ResourceManager.h

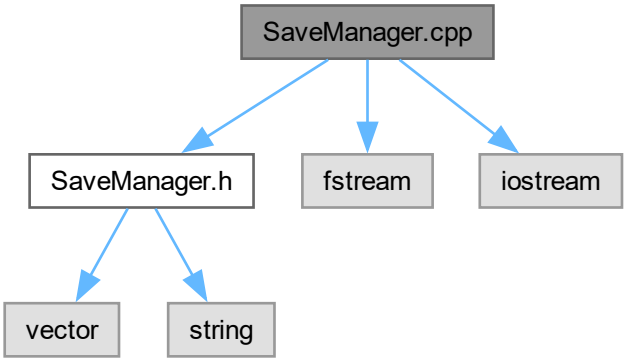
Idź do dokumentacji tego pliku.

```
00001 #ifndef MINI_GOLF_RESOURCEMANAGER_H
00002 #define MINI_GOLF_RESOURCEMANAGER_H
00003
00004 #include "TextureManager.h"
00005 #include "FontManager.h"
00006 #include "SoundManager.h"
00007 #include "DEFINITIONS.h"
00008
00017 class ResourceManager {
00018 public:
00019     ResourceManager() = delete;
00020
00025     static void loadAllResources();
00026
00030     static void cleanUpAllResources();
00031 };
00032
00033 #endif //MINI_GOLF_RESOURCEMANAGER_H
```

5.43 Dokumentacja pliku SaveManager.cpp

```
#include "SaveManager.h"
#include <fstream>
#include <iostream>
```

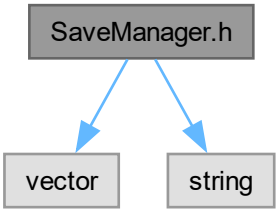
Wykres zależności załączania dla SaveManager.cpp:



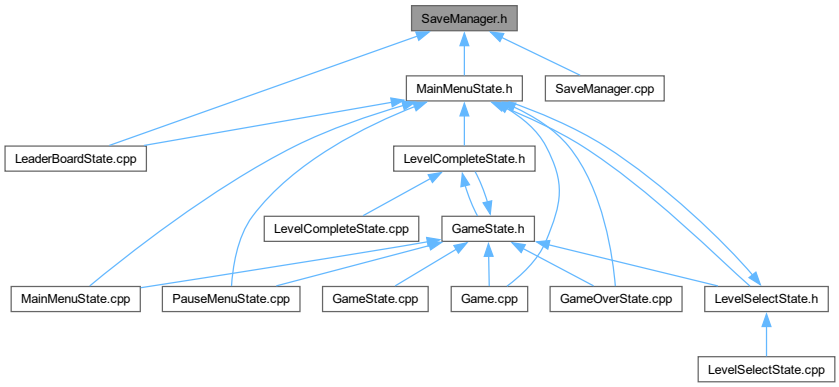
5.44 Dokumentacja pliku SaveManager.h

```
#include <vector>
#include <string>
```

Wykres zależności załączania dla SaveManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class SaveManager

  Klasa statyczna obsługująca trwałość danych gry (system zapisu/odczytu).

5.45 SaveManager.h

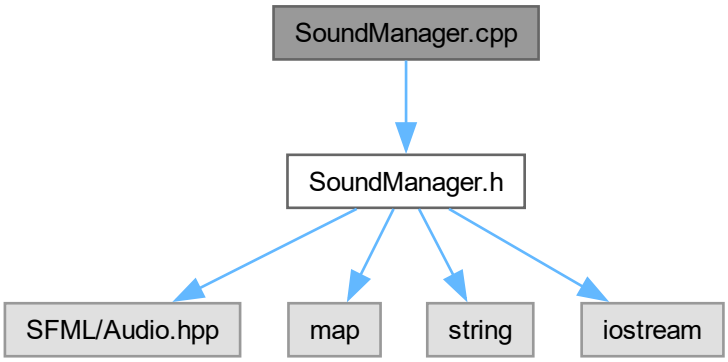
Idź do dokumentacji tego pliku.

```
#ifndef MINI_GOLF_SAVEMANAGER_H
#define MINI_GOLF_SAVEMANAGER_H

#include <vector>
#include <string>

class SaveManager {
public:
    static void init();

    static int getUnlockedLevel();

    static int getBestScore(int level);

    static void submitScore(int level, int score);

private:
    static void save();

    static void load();

    static std::string fileName;
    static int unlockedLevel;
    static std::vector<int> bestScores;
};

#endif //MINI_GOLF_SAVEMANAGER_H
```

5.46 Dokumentacja pliku SoundManager.cpp

```
#include "SoundManager.h"
```

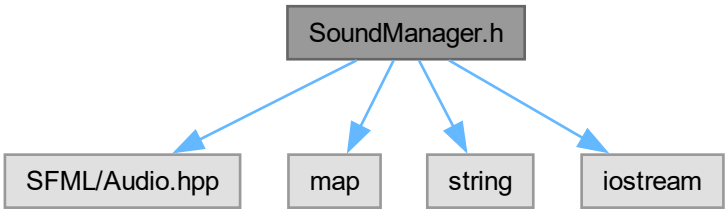
Wykres zależności załączania dla SoundManager.cpp:



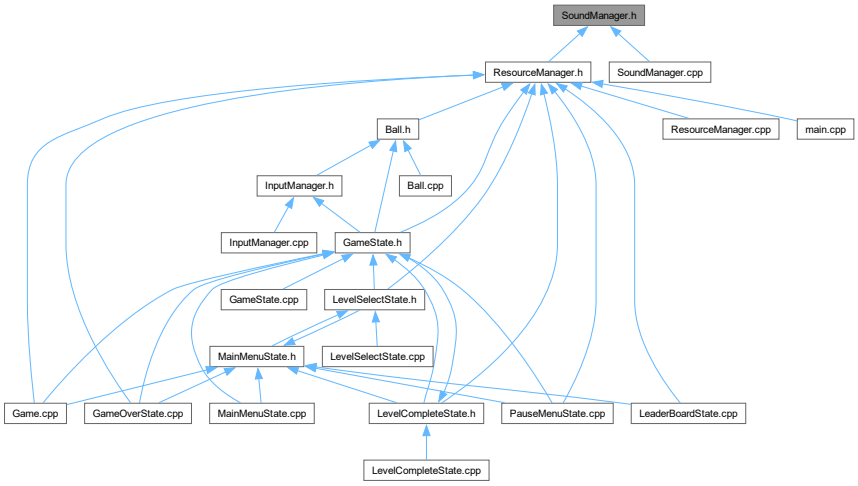
5.47 Dokumentacja pliku SoundManager.h

```
#include "SFML/Audio.hpp"
#include <map>
#include <string>
#include <iostream>
```

Wykres zależności załączania dla SoundManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class SoundManager

  Statyczny menedżer zasobów dla buforów dźwiękowych.

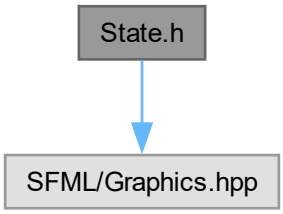
5.48 SoundManager.h

Idź do dokumentacji tego pliku.

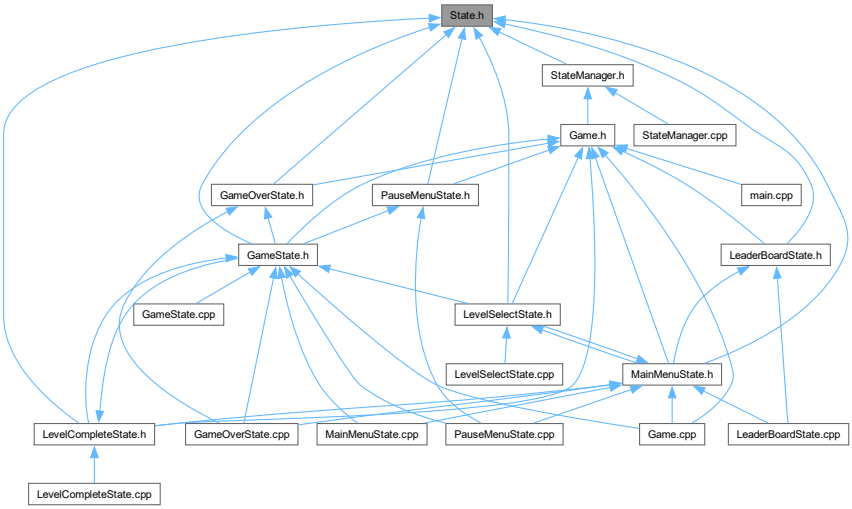
```
00001 #ifndef MINI_GOLF_SOUNDMANAGER_H
00002 #define MINI_GOLF_SOUNDMANAGER_H
00003 #include "SFML/Audio.hpp"
00004 #include <map>
00005 #include <string>
00006 #include <iostream>
00007
00015 class SoundManager {
00016 public:
00017     SoundManager() = delete;
00018
00024     static void load(const std::string& key, const std::string& path);
00025
00031     static const sf::SoundBuffer& get(const std::string&key);
00032
00036     static void cleanup();
00037
00038 private:
00039     static std::map<std::string, sf::SoundBuffer> m_Sounds;
00040 };
00041
00042 #endif //MINI_GOLF_SOUNDMANAGER_H
```

5.49 Dokumentacja pliku State.h

`#include <SFML/Graphics.hpp>`
Wykres zależności załączania dla State.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class State

  Abstrakcyjna klasa bazowa reprezentująca pojedynczy stan gry (np. Menu, Gra, Pauza).

5.50 State.h

Idź do dokumentacji tego pliku.

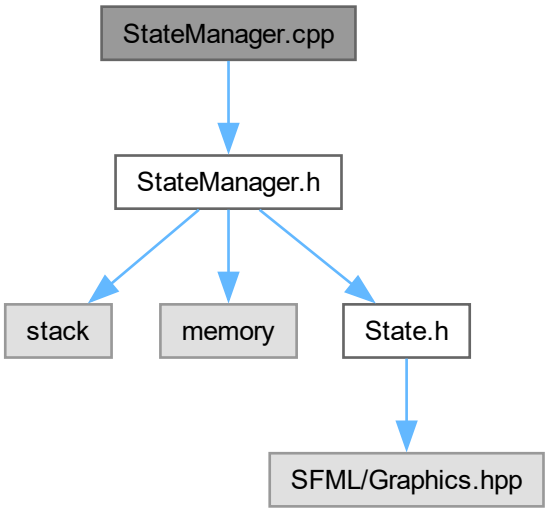
```
00001 #ifndef MINI_GOLF_STATE_H
00002 #define MINI_GOLF_STATE_H
00003
```

```
#include <SFML/Graphics.hpp>

class State {
public:
    virtual ~State() {}

    virtual void init() = 0;

    virtual void handleInput(sf::Event& event) = 0;

    virtual void update(float dt) = 0;

    virtual void draw(float dt) = 0;

    virtual void pause() {}

    virtual void resume() {}
};

#endif //MINI_GOLF_STATE_H
```

5.51 Dokumentacja pliku StateManager.cpp

`#include "StateManager.h"`

Wykres zależności załączania dla StateManager.cpp:

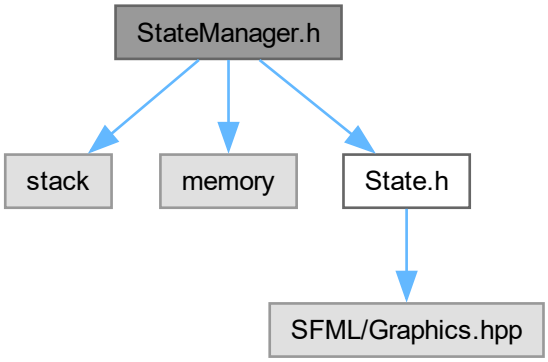


5.52 Dokumentacja pliku StateManager.h

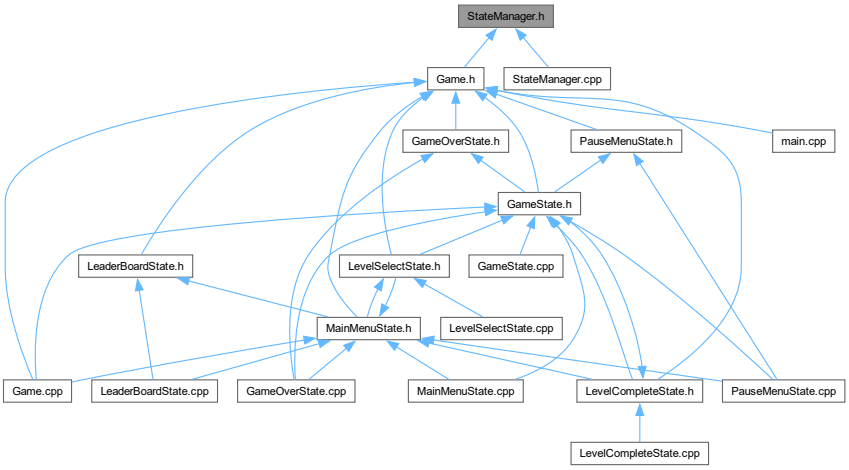
```
#include <stack>
#include <memory>
```

```
#include "State.h"
```

Wykres zależności załączania dla StateManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class StateManager

  Zarządza stosem stanów gry.

Definicje typów

- typedef std::unique_ptr< State > StateRef

  Alias dla unikalnego wskaźnika na obiekt klasy State.

5.52.1 Dokumentacja definicji typów

5.52.1.1 StateRef

`StateRef`

Alias dla unikalnego wskaźnika na obiekt klasy State.

5.53 StateManager.h

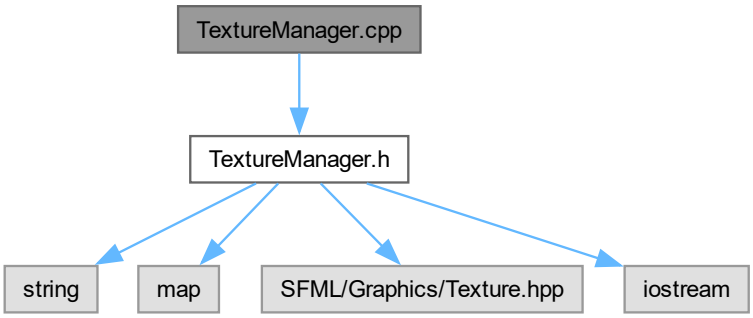
Idź do dokumentacji tego pliku.

```
#ifndef MINI_GOLF_STATEMANAGER_H
#define MINI_GOLF_STATEMANAGER_H

#include <stack>
#include <memory>
#include "State.h"

typedef std::unique_ptr<State> StateRef;

class StateManager {
public:
    StateManager() : m_isRemoving(false), m_isAdding(false), m_isReplacing(false) {}
    ~StateManager() {}

    void addState(StateRef newState, bool isReplacing = true);

    void removeState();

    void processStateChanges();

    void switchState(StateRef newState);

    StateRef& getActiveState();

private:
    std::stack<StateRef> m_states;
    StateRef m_newState;

    bool m_isRemoving;
    bool m_isAdding;
    bool m_isReplacing;
    bool m_isClearing;
};

#endif //MINI_GOLF_STATEMANAGER_H
```

5.54 Dokumentacja pliku TextureManager.cpp

```
#include "TextureManager.h"
```

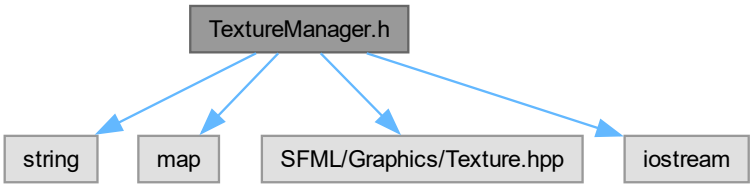
Wykres zależności załączania dla TextureManager.cpp:



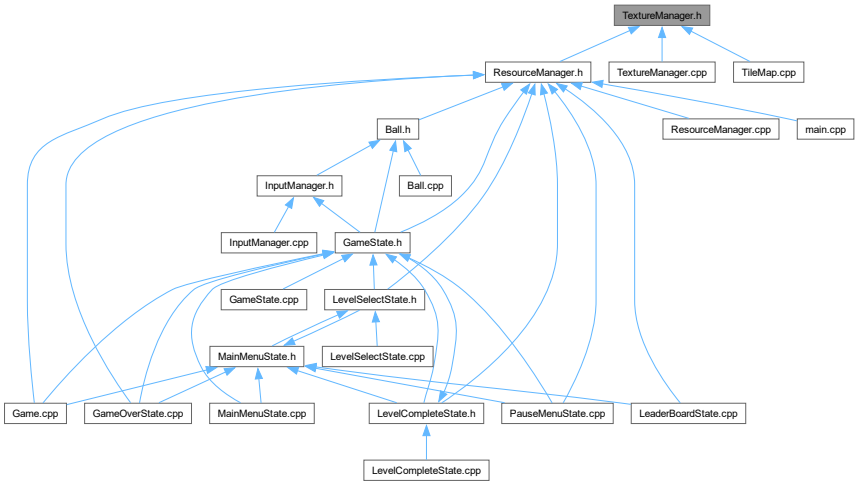
5.55 Dokumentacja pliku TextureManager.h

```
#include <string>
#include <map>
#include "SFML/Graphics/Texture.hpp"
#include <iostream>
```

Wykres zależności załączania dla TextureManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class TextureManager

  Statyczny menedżer zasobów dla tekstur SFML.

5.56 TextureManager.h

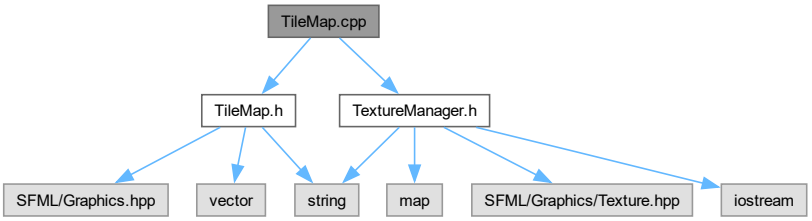
Idź do dokumentacji tego pliku.

```
00001 #ifndef MINI_GOLF_TEXTUREMANAGER_H
00002 #define MINI_GOLF_TEXTUREMANAGER_H
00003
00004 #include <string>
00005 #include <map>
00006 #include "SFML/Graphics/Texture.hpp"
00007 #include <iostream>
00008
00016 class TextureManager {
00017 public:
00018     TextureManager() = delete; // Klasa statyczna, brak możliwości instancjonowania
00019
00025     static void load(const std::string& key, const std::string& path);
00026
00032     static const sf::Texture& get(const std::string&key);
00033
00037     static void cleanup();
00038
00039 private:
00040     static std::map<std::string, sf::Texture> m_Textures;
00041 };
00042
00043 #endif //MINI_GOLF_TEXTUREMANAGER_H
```

5.57 Dokumentacja pliku TileMap.cpp

```
#include "TileMap.h"
#include "TextureManager.h"
```

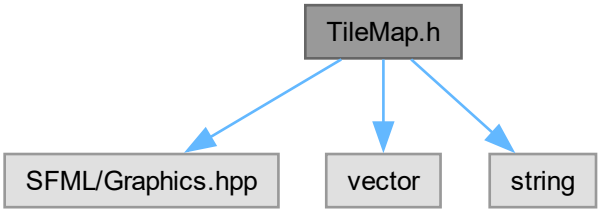
Wykres zależności załączania dla TileMap.cpp:



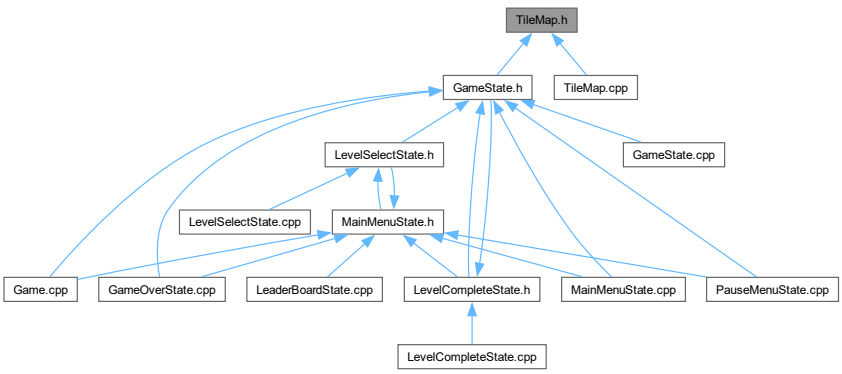
5.58 Dokumentacja pliku TileMap.h

```
#include <SFML/Graphics.hpp>
#include <vector>
#include <string>
```

Wykres zależności załączania dla TileMap.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class TileMap

  Klasa odpowiedzialna za renderowanie poziomu gry przy użyciu Vertex Array.

5.59 TileMap.h

Idź do dokumentacji tego pliku.

```
00001 #ifndef TILEMAP_H
00002 #define TILEMAP_H
00003
00004 #include <SFML/Graphics.hpp>
00005 #include <vector>
00006 #include <string>
00007
00016 class TileMap : public sf::Drawable, public sf::Transformable
00017 {
00018 public:
00028     bool load(const std::string& tileset, sf::Vector2u tileSize, std::vector<int>tiles, unsigned int
      width, unsigned int height);
00029
00030 private:
00038     virtual void draw(sf::RenderTarget& target, sf::RenderStates states) const;
00039
00040     sf::VertexArray m_vertices;
00041     const sf::Texture* m_tileset;
00042 };
00043
00044 #endif // TILEMAP_H
```

Skorowidz

~State
State, 103
~StateManager
StateManager, 106

addState
StateManager, 106
aimLine
GameState, 41
applyImpulse
Ball, 9

Ball, 7
applyImpulse, 9
Ball, 9
bounceX, 9
bounceY, 10
contains, 10
draw, 11
getPosition, 11
getScale, 12
getVelocity, 12
invertVelocity, 13
player, 15
setPosition, 13
setScale, 13
stop, 14
update, 14
velocity, 15
ball
GameState, 41
Ball.cpp, 115
Ball.h, 115
BALL_TEXTURE_PATH
DEFINITIONS.h, 119
ballInHole
GameState, 41
ballStartPos
InputManager, 49
bestScores
SaveManager, 98
bounceX
Ball, 9
bounceY
Ball, 10

cleanup
FontManager, 16
SoundManager, 100
TextureManager, 109
cleanUpAllResources
ResourceManager, 91
clock
Game, 20
COLOR_BALL
DEFINITIONS.h, 121
COLOR_CONCRETE
DEFINITIONS.h, 121
COLOR_CORNER_LD
DEFINITIONS.h, 121
COLOR_CORNER_LU
DEFINITIONS.h, 122
COLOR_CORNER_RD
DEFINITIONS.h, 122
COLOR_CORNER_RU
DEFINITIONS.h, 122
COLOR_GRASS
DEFINITIONS.h, 123
COLOR_HOLE
DEFINITIONS.h, 123
COLOR_RAMP_DOWN
DEFINITIONS.h, 123
COLOR_RAMP_L
DEFINITIONS.h, 124
COLOR_RAMP_R
DEFINITIONS.h, 124
COLOR_RAMP_UP
DEFINITIONS.h, 124
COLOR_SAND
DEFINITIONS.h, 125
COLOR_WALL_DOWN
DEFINITIONS.h, 125
COLOR_WALL_L
DEFINITIONS.h, 125
COLOR_WALL_R
DEFINITIONS.h, 126
COLOR_WALL_UP
DEFINITIONS.h, 126
COLOR_WATER
DEFINITIONS.h, 126
colorToTileID
LevelManager, 65
contains
Ball, 10
currentMousePos
InputManager, 49

DEFINITIONS.h, 117
BALL_TEXTURE_PATH, 119
COLOR_BALL, 121

COLOR_CONCRETE, 121
COLOR_CORNER_LD, 121
COLOR_CORNER_LU, 122
COLOR_CORNER_RD, 122
COLOR_CORNER_RU, 122
COLOR_GRASS, 123
COLOR_HOLE, 123
COLOR_RAMP_DOWN, 123
COLOR_RAMP_L, 124
COLOR_RAMP_R, 124
COLOR_RAMP_UP, 124
COLOR_SAND, 125
COLOR_WALL_DOWN, 125
COLOR_WALL_L, 125
COLOR_WALL_R, 126
COLOR_WALL_UP, 126
COLOR_WATER, 126
FONT_PATH, 119
HEART_PATH, 119
HEART_WHITE_PATH, 119
HIT_PATH, 119
LEVEL_HEIGHT, 120
LEVEL_WIDTH, 120
SCREEN_HEIGHT, 120
SCREEN_WIDTH, 120
SPLASH_PATH, 120
TILE_SIZE, 120
TILESET_PATH, 120
TITLE, 120
WALL_HIT_PATH, 121
WIN_PATH, 121
draw
Ball, 11
GameOverState, 26
GameState, 34
LeaderboardState, 54
LevelCompleteState, 60
LevelSelectState, 73
MainMenuState, 79
PauseMenuState, 87
State, 103
TileMap, 113

fileName
SaveManager, 98
FONT_PATH
DEFINITIONS.h, 119
FontManager, 15
cleanup, 16
FontManager, 16
get, 16
load, 17
m_Font, 18
FontManager.cpp, 128
FontManager.h, 128
force
InputManager, 49
forceVector
GameState, 41
Game, 18
clock, 20
Game, 19
m_data, 20
run, 20
timePerFrame, 20
Game.cpp, 129
Game.h, 130
GameDataRef, 131
GameData, 21
machine, 22
window, 22
GameDataRef
Game.h, 131
GameOverState, 22
draw, 26
GameOverState, 26
handleInput, 26
init, 27
m_alpha, 27
m_backgroundSprite, 27
m_bgTexture, 27
m_data, 28
m_dimmer, 28
m_fadeRect, 28
m_goToMenu, 28
m_isTransitioning, 28
m_level, 28
m_menuButton, 28
m_retryButton, 28
m_retryLevel, 29
m_title, 29
update, 27
GameOverState.cpp, 132
GameOverState.h, 132
GameState, 29
aimLine, 41
ball, 41
ballInHole, 41
draw, 34
forceVector, 41
GameState, 34
getTileAt, 34
handleInput, 35
heartSprites, 41
init, 35
initSounds, 36
initUI, 37
inputManager, 41
isJumping, 42
isSinking, 42
isWall, 37
jumpTimer, 42
lastSafePos, 42
levelData, 42
levelFinishTimer, 42
lives, 42
m_alpha, 42

m_data, 43
m_fadeRect, 43
m_hitSound, 43
m_level, 43
m_shadow, 43
m_splashSound, 43
m_wallHitSound, 43
m_winSound, 43
maxJumpTime, 44
resetLevel, 38
scoreText, 44
strokes, 44
tileMap, 44
update, 38
updateAimLine, 39
updateHearts, 40
wasInWater, 44
winText, 44
GameState.cpp, 134
GameState.h, 134
get
FontManager, 16
SoundManager, 100
TextureManager, 109
getActiveState
StateManager, 107
getBestScore
SaveManager, 94
getCurrentForce
InputManager, 46
getForceVector
InputManager, 47
getPosition
Ball, 11
getScale
Ball, 12
getTileAt
GameState, 34
getUnlockedLevel
SaveManager, 94
getVelocity
Ball, 12

handleEvent
InputManager, 47
handleInput
GameOverState, 26
GameState, 35
LeaderboardState, 54
LevelCompleteState, 60
LevelSelectState, 73
MainMenuState, 79
PauseMenuState, 87
State, 103
HEART_PATH
DEFINITIONS.h, 119
HEART_WHITE_PATH
DEFINITIONS.h, 119
heartSprites
GameState, 41
HIT_PATH
DEFINITIONS.h, 119

init
GameOverState, 27
GameState, 35
LeaderboardState, 54
LevelCompleteState, 61
LevelSelectState, 73
MainMenuState, 79
PauseMenuState, 87
SaveManager, 95
State, 104
initSounds
GameState, 36
initUI
GameState, 37
InputManager, 45
ballStartPos, 49
currentMousePos, 49
force, 49
getCurrentForce, 46
getForceVector, 47
handleEvent, 47
InputManager, 46
isDragging, 48
isGrabbed, 49
isReady, 48
ready, 49
inputManager
GameState, 41
InputManager.cpp, 136
InputManager.h, 137
invertVelocity
Ball, 13
isDragging
InputManager, 48
isGrabbed
InputManager, 49
isJumping
GameState, 42
isReady
InputManager, 48
isSinking
GameState, 42
isWall
GameState, 37

jumpTimer
GameState, 42

lastSafePos
GameState, 42
LeaderboardState, 50
draw, 54
handleInput, 54
init, 54
LeaderboardState, 53

m_alpha, 55
m_backButton, 55
m_data, 55
m_fadeRect, 55
m_isFadingIn, 55
m_isGoingBack, 55
m_scoreTexts, 55
m_title, 55
update, 54
LeaderBoardState.cpp, 138
LeaderBoardState.h, 139
LEVEL_HEIGHT
DEFINITIONS.h, 120
LEVEL_WIDTH
DEFINITIONS.h, 120
LevelCompleteState, 56
draw, 60
handleInput, 60
init, 61
LevelCompleteState, 60
m_alpha, 62
m_backgroundSprite, 62
m_bgTexture, 62
m_currentLevel, 62
m_data, 62
m_dimmer, 62
m_fadeRect, 62
m_goToMenu, 62
m_isTransitioning, 63
m_menuButton, 63
m_nextLevelButton, 63
m_retryButton, 63
m_retryLevel, 63
m_scoreText, 63
m_strokes, 63
m_title, 63
update, 61
LevelCompleteState.cpp, 140
LevelCompleteState.h, 140
levelData
GameState, 42
levelFinishTimer
GameState, 42
LevelManager, 64
colorToTileID, 65
loadLevelFromImage, 67
startPosition, 69
LevelManager.cpp, 142
LevelManager.h, 143
LevelSelectState, 69
draw, 73
handleInput, 73
init, 73
LevelSelectState, 72
m_alpha, 74
m_backButton, 74
m_data, 74
m_fadeRect, 75
m_isFadingIn, 75
m_isTransitioning, 75
m_levelButtons, 75
m_nextLevelToLoad, 75
m_title, 75
update, 74
LevelSelectState.cpp, 144
LevelSelectState.h, 144
lives
GameState, 42
load
FontManager, 17
SaveManager, 96
SoundManager, 100
TextureManager, 110
TileMap, 113
loadAllResources
ResourceManager, 91
loadLevelFromImage
LevelManager, 67

m_alpha
GameOverState, 27
GameState, 42
LeaderboardState, 55
LevelCompleteState, 62
LevelSelectState, 74
MainMenuState, 80
PauseMenuState, 88
m_backButton
LeaderboardState, 55
LevelSelectState, 74
m_background
MainMenuState, 80
m_backgroundSprite
GameOverState, 27
LevelCompleteState, 62
PauseMenuState, 88
m_bgTexture
GameOverState, 27
LevelCompleteState, 62
PauseMenuState, 88
m_currentLevel
LevelCompleteState, 62
PauseMenuState, 88
m_data
Game, 20
GameOverState, 28
GameState, 43
LeaderboardState, 55
LevelCompleteState, 62
LevelSelectState, 74
MainMenuState, 80
PauseMenuState, 88
m_dimmer
GameOverState, 28
LevelCompleteState, 62
PauseMenuState, 89
m_exitButton

MainMenuState, 80
m_fadeRect
GameOverState, 28
GameState, 43
LeaderboardState, 55
LevelCompleteState, 62
LevelSelectState, 75
MainMenuState, 81
PauseMenuState, 89
m_Font
FontManager, 18
m_goToLeaderboard
MainMenuState, 81
m_goToMenu
GameOverState, 28
LevelCompleteState, 62
m_hitSound
GameState, 43
m_isAdding
StateManager, 107
m_isClearing
StateManager, 107
m_isFadingIn
LeaderboardState, 55
LevelSelectState, 75
MainMenuState, 81
m_isGoingBack
LeaderboardState, 55
m_isGoingToMenu
PauseMenuState, 89
m_isRemoving
StateManager, 107
m_isReplacing
StateManager, 108
m_isReseting
PauseMenuState, 89
m_isTransitioning
GameOverState, 28
LevelCompleteState, 63
LevelSelectState, 75
MainMenuState, 81
m_leaderboardButton
MainMenuState, 81
m_level
GameOverState, 28
GameState, 43
m_levelButtons
LevelSelectState, 75
m_menuButton
GameOverState, 28
LevelCompleteState, 63
PauseMenuState, 89
m_newState
StateManager, 108
m_nextLevelButton
LevelCompleteState, 63
m_nextLevelToLoad
LevelSelectState, 75
m_pauseTitle
PauseMenuState, 89
m_playButton
MainMenuState, 81
m_resetButton
PauseMenuState, 89
m_resumeButton
PauseMenuState, 89
m_retryButton
GameOverState, 28
LevelCompleteState, 63
m_retryLevel
GameOverState, 29
LevelCompleteState, 63
m_scoreText
LevelCompleteState, 63
m_scoreTexts
LeaderboardState, 55
m_shadow
GameState, 43
m_Sounds
SoundManager, 101
m_splashSound
GameState, 43
m_states
StateManager, 108
m_strokes
LevelCompleteState, 63
m_Textures
TextureManager, 111
m_tileset
TileMap, 114
m_title
GameOverState, 29
LeaderboardState, 55
LevelCompleteState, 63
LevelSelectState, 75
MainMenuState, 81
m_vertices
TileMap, 114
m_wallHitSound
GameState, 43
m_winSound
GameState, 43
machine
GameData, 22
main
main.cpp, 146
main.cpp, 146
main, 146
MainMenuState, 76
draw, 79
handleInput, 79
init, 79
m_alpha, 80
m_background, 80
m_data, 80
m_exitButton, 80

m_fadeRect, 81
m_goToLeaderboard, 81
m_isFadingIn, 81
m_isTransitioning, 81
m_leaderboardButton, 81
m_playButton, 81
m_title, 81
MainMenuState, 79
update, 80
MainMenuState.cpp, 147
MainMenuState.h, 147
maxJumpTime
GameState, 44

pause
State, 104
PauseMenuState, 82
draw, 87
handleInput, 87
init, 87
m_alpha, 88
m_backgroundSprite, 88
m_bgTexture, 88
m_currentLevel, 88
m_data, 88
m_dimmer, 89
m_fadeRect, 89
m_isGoingToMenu, 89
m_isReseting, 89
m_menuButton, 89
m_pauseTitle, 89
m_resetButton, 89
m_resumeButton, 89
PauseMenuState, 86
update, 88
PauseMenuState.cpp, 148
PauseMenuState.h, 149
player
Ball, 15
processStateChanges
StateManager, 107

ready
InputManager, 49
removeState
StateManager, 107
resetLevel
GameState, 38
ResourceManager, 90
cleanUpAllResources, 91
loadAllResources, 91
ResourceManager, 90
ResourceManager.cpp, 150
ResourceManager.h, 150
resume
State, 104
run
Game, 20

save
SaveManager, 96
SaveManager, 92
bestScores, 98
fileName, 98
getBestScore, 94
getUnlockedLevel, 94
init, 95
load, 96
save, 96
submitScore, 97
unlockedLevel, 98
SaveManager.cpp, 152
SaveManager.h, 152
scoreText
GameState, 44
SCREEN_HEIGHT
DEFINITIONS.h, 120
SCREEN_WIDTH
DEFINITIONS.h, 120
setPosition
Ball, 13
setScale
Ball, 13
SoundManager, 99
cleanup, 100
get, 100
load, 100
m_Sounds, 101
SoundManager, 100
SoundManager.cpp, 154
SoundManager.h, 154
SPLASH_PATH
DEFINITIONS.h, 120
startPosition
LevelManager, 69
State, 101
~State, 103
draw, 103
handleInput, 103
init, 104
pause, 104
resume, 104
update, 104
State.h, 156
StateManager, 105
~StateManager, 106
addState, 106
getActiveState, 107
m_isAdding, 107
m_isClearing, 107
m_isRemoving, 107
m_isReplacing, 108
m_newState, 108
m_states, 108
processStateChanges, 107
removeState, 107
StateManager, 106

switchState, 107
StateManager.cpp, 157
StateManager.h, 157
StateRef, 159
StateRef
StateManager.h, 159
stop
Ball, 14
strokes
GameState, 44
submitScore
SaveManager, 97
switchState
StateManager, 107

TextureManager, 108
cleanup, 109
get, 109
load, 110
m_Textures, 111
TextureManager, 109
TextureManager.cpp, 160
TextureManager.h, 160
TILE_SIZE
DEFINITIONS.h, 120
TileMap, 111
draw, 113
load, 113
m_tileset, 114
m_vertices, 114
tileMap
GameState, 44
TileMap.cpp, 161
TileMap.h, 162
TILESET_PATH
DEFINITIONS.h, 120
timePerFrame
Game, 20
TITLE
DEFINITIONS.h, 120

unlockedLevel
SaveManager, 98
update
Ball, 14
GameOverState, 27
GameState, 38
LeaderboardState, 54
LevelCompleteState, 61
LevelSelectState, 74
MainMenuState, 80
PauseMenuState, 88
State, 104
updateAimLine
GameState, 39
updateHearts
GameState, 40

velocity
Ball, 15

WALL_HIT_PATH
DEFINITIONS.h, 121
wasInWater
GameState, 44
WIN_PATH
DEFINITIONS.h, 121
window
GameData, 22
winText
GameState, 44